East China Normal University

School of Software Engineering

Acceptance Safety: A Conjunctive Release Framework for LLM-Generated Guard Specifications

HSP-Agile: A SOFL/FSF Instantiation

Research Report — East China Normal University

Ye Xujun

School of Software Engineering
East China Normal University
Shanghai, China

July 2026

Keywords: specification-guided defect prevention, formal specification, LLM synthesis, SOFL, counterexample-guided repair, SMT verification, mutation testing

Abstract

Large language models synthesise code that often *looks* correct under everyday unit tests yet still mishandles thin specification regions—especially catch-all residuals and ordered overlaps. Consider a tier classifier whose `OTHERWISE` branch must return `Review`: an LLM candidate can pass every Reject/Pass sample while silently promoting the mid-band to `Pass`. A release dashboard that tracks mean pass rate alone will ship that defect.

We reframe the goal as **acceptance safety**: release only when every specification-derived witness matches *and* a structural screen clears (Accept $\Leftrightarrow$ Conf=1$\wedge$Screen=*pass*). The vehicle is a **conjunctive release decision framework** realised by HSP-Agile on SOFL/FSF: first-match SMT witnesses, a typed Semantic Feedback IR for repair under headroom, and a pluggable screen. Here *FAR* (False Accept Rate) is the fraction of faulty mutants that a mode incorrectly releases; *PDR* is the complementary rejection rate.

At comparable mean conformance, conjunctive Accept cuts escaped defects (E2: M FAR 5.0% vs. B2 8.8%; PDR 95.0% vs. 91.2%; bootstrap CIs on the gaps exclude 0). When repair still has headroom, structured feedback rescues catch-all failures that unstructured dumps miss (E6: semantic_ir 86.9% vs. test_only 79.1%, +7.7 pp; W/L/T 14/4/102; 95% CI [2.3, 13.6] pp; p=0.018; all 14 Conf. wins are `others` rescues, none are ordering repairs). At matched budget K=3, equal-K Conf. places `M_eq` at 100.0% vs. B2 97.5% (+2.5 pp; 3/0/117). Prefer test-feedback (B2) by default; escalate to full mode M when release requires Accept=1 or FAR$\leq$5% and headroom remains. Table 1.1 summarises the evidence ladder. Artifacts released.

Keywords. Acceptance safety; conjunctive release; specification-guided repair; false accept rate; LLM synthesis; SOFL; SMT witnesses; catch-all rescue.

Contents

1 Introduction

A release dashboard can look healthy while a residual specification region is already broken. That is the defect this report studies—not whether a model can write more tests that pass, but whether a team can *refuse* to ship LLM-generated code when the specification still has a thin slice that everyday suites never exercise.

Acceptance safety is that refusal rule. Release requires every specification-derived witness to match *and* a structural screen to clear: Accept $\Leftrightarrow$ Conf=1 $\wedge$ Screen=*pass*. A candidate that "looks right" on busy inputs, yet mishandles a catch-all or overlap region, must not sail through on mean pass rate alone. We use a direct **dual-gate acceptance rule**: release requires both complete witness conformance and a clear static safety screen. HSP-Agile makes the rule executable for SOFL/FSF through SpecIR regions, structured scenario-aware repair feedback, and a configurable Screen. The rule is a bounded operational check over generated witnesses and the configured screen; it is not a proof of whole-program safety.

Conjunctive Accept cuts escaped defects at comparable mean conformance (E2); structured feedback rescues catch-all failures when repair still has headroom (E6); prefer test-feedback (B2) by default and escalate to full mode M when release requires Accept=1 or FAR$\leq$5% and headroom remains.

1.1 Primary Vignette: Catch-All That Ships

Start from the failure mode that drives our strongest Conf. evidence.

Imagine a tier classifier for incoming support tickets (Listing 1.1). Two named guards handle the easy cases: reject negative scores; pass scores at or above a floor. Everything else falls into `OTHERWISE` and must return `Review`—a human queue, not an automatic green light. An LLM candidate that looks "logically smooth" collapses that residual into `Pass` (Listing 1.2). Unit tests drawn from Reject and Pass bands all succeed. The dashboard reports a high pass rate. In production, the mid-band—the catch-all—never reaches a human; the bug is an escaped defect, not a failed build.

That is false acceptance in miniature. A pass/fail dump tells the repair model only that "something failed" if a rare mid-band test ever appears; it does not name scenario S_3 (`others`), the expected constant, or why the residual matters. A typed feedback record does. Under controlled E6, all 14 Conf. wins for the structured scenario-aware feedback

record (implemented as Semantic Feedback IR) over test-only concentrate on this catch-all regime (Tables 7.4, 7.5, 7.8); none are ordering-precedence repairs on BENCH-120.

```
# classify_tier(score: int, floor: int) -> tier: str
#
# S1: WHEN score < 0
#         THEN tier = "Reject"
#
# S2: WHEN score >= floor
#         THEN tier = "Pass"
#
# S3: OTHERWISE                 -- catch-all others
#         THEN tier = "Review"  -- arithmetic/output must match oracle
```

Listing 1.1: Primary vignette: catch-all `others` must return `Review`. Busy Reject/Pass tests miss the residual—the E6-aligned repair regime.

```
def classify_tier(score: int, floor: int) -> str:
    if score < 0:
        return "Reject"
    if score >= floor:
        return "Pass"
    # BUG: catch-all others returns wrong constant / wrong expression
    return "Pass"  # should be "Review"
```

Listing 1.2: Looks coherent, ships the wrong catch-all: mid-band returns `Pass` instead of `Review`.

1.2 Problem Statement

1.2.1 Setting

SOFL[40,42] provides the Functional Specification Format (FSF): an ordered list of scenarios, each pairing a guard with a postcondition under first-match precedence. Agile-SOFL[36] embeds those artefacts in sprint workflows as design documents, oracles, and acceptance criteria. LLM-assisted generation accelerates that workflow[11,52], but synthesises by pattern completion rather than by respecting specification structure[15,57]. Two failure modes matter: (i) catch-all / output mismatches on residual regions that unstructured dumps fail to localise (primary vignette above), and (ii) overlap slices where first-match order is decisive (secondary vignette below). Random suites undersample both[69]; when they fail, pass/fail bits still omit scenario identity.

1.2.2 High Pass Rate, Weak Acceptance Safety

A specification-conformance defect is a bug that unit tests rarely catch—or catch without enough structure to repair. On the tier classifier, a candidate can clear every Reject/Pass sample and still mis-route every mid-band ticket. A release gate that only looks at mean pass rate may *accept* that candidate anyway.

We write FAR for the **False Accept Rate**: among injected faulty implementations (mutants), the fraction that a mode incorrectly releases. PDR (Prevention Detection Rate) is the complementary rejection rate on that set. Strong mean conformance—agreement on a finite witness suite—does not imply low FAR.

1.2.3 Acceptance Predicate

Given task t with ordered scenarios $\mathcal{S}_t = (S_1, \ldots, S_n)$ and budget K, is candidate c safe to release? Every specification-derived witness must match, *and* no critical structural anti-pattern may remain:

$$\text{Accept}(c,t) \iff \text{Conf}(c,t) \geq \tau \ \wedge \ P^H(c) = 0, \tag{1.1}$$

where $\text{Conf}(c,t) \in [0,1]$ is the fraction of witnesses on which c matches the oracle, $\tau = 1.0$ here, and $P^H(c) = 0$ asserts that the pluggable high-severity Screen is clear. A witness for scenario S_i must land in the *priority region*

$$\Phi_i(\mathbf{x}) \ \triangleq \ G_i(\mathbf{x}) \wedge \bigwedge_{j<i} \neg G_j(\mathbf{x}) \tag{1.2}$$

—including, for catch-alls, an input that satisfies no earlier guard. Figure 1.1 contrasts a busy test cloud with a surgical SMT cut; the geometry applies to residual regions as well as overlaps.

1.2.4 Secondary Vignette: Ordering / Overlap

Overlap explains a second way tests miss bugs and motivates Φ_i witnesses. Listing 1.3 is an ordered tax-bracket contract: on $(income, is\_foreign) = (150000, 1)$, both a foreign-high surcharge and a domestic mid bracket hold, yet first-match requires `ForeignHigh`. Listing 1.4 tests mid first—wrong, but often invisible to domestic-heavy suites. This vignette anchors witness design and E2 structural mutants; it is *not* the dominant E6 win regime (Table 7.6).

```
1 # compute_tax(income: int, is_foreign: int) -> bracket: str
2 #
```

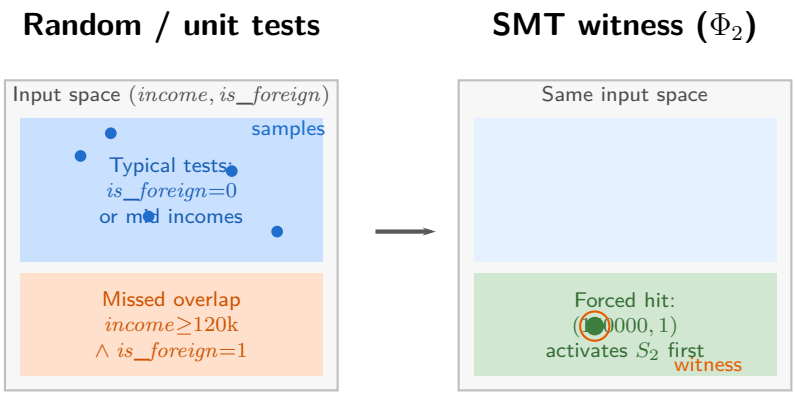

Fig. 1.1. Acceptance safety needs witnesses, not denser random tests. Left: unit tests concentrate on busy bands, so a catch-all or overlap bug can still look like a high pass rate. Right: an SMT model of a thin Φ_i (Equation (1.2)) forces the region the release gate must clear—why FAR, not mean Conf. alone, is the release metric. Catch-all rescue under structured feedback is the primary E6 Conf. mechanism (Listings 1.1–1.2); ordering/overlap motivates witnesses and E2.

```
# Ordered FSF scenarios (first-match): the first true guard wins.
# S2 and S3 overlap when income >= 120000 and is_foreign = 1;
# precedence requires ForeignHigh before DomesticMid.
#
# S1: WHEN income < 0
#         THEN bracket = "Invalid"
#
# S2: WHEN income >= 120000 AND is_foreign = 1
#         THEN bracket = "ForeignHigh"
#
# S3: WHEN income >= 50000
#         THEN bracket = "DomesticMid"
#
# S4: OTHERWISE                  -- catch-all others
#         THEN bracket = "Low"
```

Listing 1.3: Secondary vignette: ordered tax brackets. First-match: S_2 preempts S_3 on the foreign-high overlap.

```python
def compute_tax(income: int, is_foreign: int) -> str:
    # BUG: evaluates the mid-bracket guard before the foreign-high
    # surcharge. On (150000, 1) both guards hold; FSF first-match
    # requires "ForeignHigh", but this candidate returns "DomesticMid".
    if income < 0:
        return "Invalid"
    if income >= 50000:
```

```
        return "DomesticMid"
    if income >= 120000 and is_foreign == 1:
        return "ForeignHigh"
    return "Low"
```

Listing 1.4: Bracket-order bug: mid guard before foreign-high surcharge—witness/E2 motivation, not E6 win coding.

1.3 Research Challenges

1.3.1 Name the Failed Region

Checking scenario S_i requires an input in Φ_i (Equation (1.2))—including catch-alls that random tests miss[7]. The check must preserve first-match partitions without rewriting the pipeline per surface notation (Chapter 3).

1.3.2 Typed Failures for Repair

Test-feedback loops lack specification-level structure[57]: the model may patch one counterexample without knowing whether a catch-all output or an ordering clash failed. Repair needs a record that names the violated scenario *before* any natural-language prompt is rendered (Chapter 4).

1.3.3 Conjunctive Release under Budget K

Full verification is impractical for unannotated LLM output[35,43]. What is needed is calibrated acceptance safety: stronger than pass-rate gates[11], weaker than deductive proof, and fast enough for sprint budgets[8,36]—Equation (1.1) under hard K (K=3 in controlled E6).

1.4 Proposed Approach

The same specification that exposes the catch-all defect can drive prevention if three ideas hold: shared semantic regions, typed repair feedback, and conjunctive release. SGDP (Specification-guided Defect Prevention) does three jobs: (1) carve first-match regions with an SMT solver; (2) tell the LLM *which* region failed; (3) refuse release unless every region and the pluggable screen clear. Figure 1.2 is that loop. Adapters and SMT lowering are necessary engineering, not separate scientific claims. Chapter 4 walks the decision path end to end; Chapter 7 leads with mechanism and prevention, not aggregate pass-

rate crowns.

1.5 Contributions

This work contributes a **dual-gate acceptance rule** and evidence for when scenario-aware repair and static screening justify their cost (Figure 1.3; Table 1.1). The headline is acceptance safety (C3): the gate that refuses to ship a lucky-pass residual when mean Conf. against test-feedback looks fine. SpecIR adapters, SMT lowering, and the hard benchmark are engineering that makes the decision rule runnable.

Engineering premise (SpecIR). Prevention must depend on *semantic regions*, not surface syntax—the Φ_i cut in Figure 1.1. Overlap tertiles on E3 are non-monotone: denser overlap does not by itself prescribe deploying M (a negative bound on C4).

C3. Release must be *conjunctive*: every specification-derived witness matches *and* no high-severity structural pattern remains (Equation (1.1)). Pass-rate alone is not enough—as the catch-all vignette shows, a candidate can look strong on busy tests and still ship a broken residual. On E2 impl-screening (n=852), M reaches PDR 95.0% / FAR 5.0% vs. B2 91.2% / 8.8% (bootstrap CIs on the gaps exclude zero). Ordering and preemption mutants motivate the gate beyond E6 Conf. gains. Limitation: near-ceiling mean-Conf. ranking across corpora is secondary appendix material; C3 is scored by Accept/FAR, not by Conf. crowns.

C2. When repair has headroom, it needs *specification-indexed structured feedback*, not raw pass/fail bits. E6 isolates feedback under full M, equal K=3: semantic_ir reaches 86.9% vs. test_only 79.1% (+7.7 pp; paired W/L/T 14/4/102; 95% CI [2.3, 13.6] pp; Wilcoxon p=0.018). Qualitative coding of all 14 wins (Table 7.8): **14/14 catch-all (`others`) rescues**; **0/14 ordering-precedence repairs**. Field structure (Table 7.14): FULL beats NL-only dumps of the same facts (+28.0 pp; CI excludes 0). Limitation: E14 shows length-matched structured traces can beat bare Semantic Feedback IR on aggregate Conf.; the E6 win profile on BENCH-120 is catch-all rescue, not ordering-precedence repair.

C4. Prefer test-feedback (B2) by default for mean conformance and latency; escalate to full HSP-Agile (M) when release requires Accept=1 or target FAR≤5% *and* repair headroom remains (Table 8.1). When contracts are simple enough that baselines already saturate, B2 is enough—that is a deployment boundary, not a failure of the decision rule. Equal-K `M_eq` is +2.5 pp vs. B2 (100.0% vs. 97.5%; 3/0/117); external pilots often tie on mean Conf.; a public ACL-style case (Section 7.4) shows the gate intercepting a lucky-pass candidate. Limitation: live vendor CI replication remains future work; present pilots support the deployment heuristic, not production KPIs.

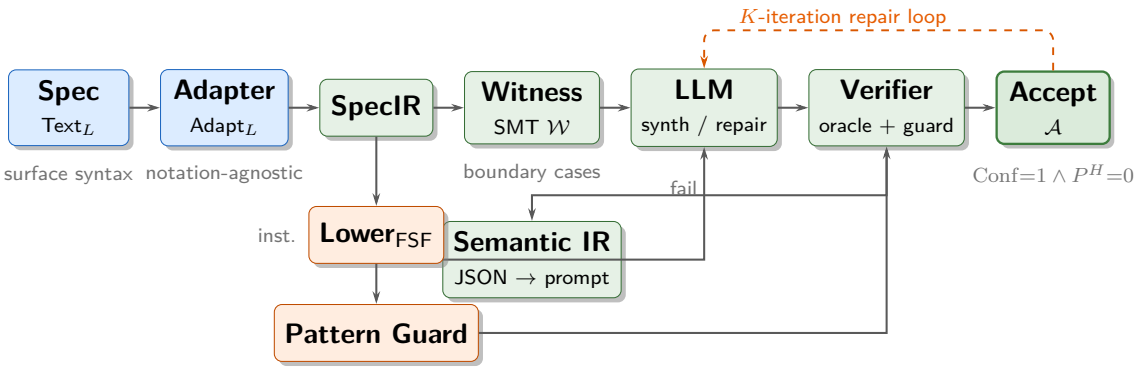

Fig. 1.2. Three ideas, one release loop: SpecIR regions, Semantic Feedback IR, conjunctive Accept under budget K. On the catch-all vignette, the loop forces the residual Φ_i, names the others/output violation, and releases only when Accept holds (Equation (1.1)). Left to right for one attempt; dashed orange arc is repair; dashed branch is FSF lowering and the pluggable pattern screen.

Table 1.1. Evidence ladder for the conjunctive release decision framework (read top-down). Lead sell is *acceptance safety* (C3: FAR/PDR), not a higher pass-rate headline. Near-ceiling E1/E10/E12 ranking lives in the appendix only.

Rung	Claim	Authoritative evidence
1	C3 acceptance safety	E2 PDR/FAR (M 95.0%/5.0% vs. B2 91.2%/8.8%); defect map 7.6
2	C2 repair under headroom	E6 +7.7 pp vs. test-only (CI excl. 0); qualitative coding Table 7.8 (14/14 catch-all; 0/14 ordering)
3	C2 scope	E14 (not unique among structured); field table 7.14 (structure ≫ NL; single-field CIs include 0)
4	C4 deployment	Equal-K Table 7.20; ACL interception case; pilots often B2=`M_eq` (escalate for Accept/FAR)
—	Appendix only	E1/E10/E12 near-ceiling Conf. stress-tests

Primary evidence is E2, E6, equal-K Conf. ranking, and the deployment table. Near-ceiling fixed-oracle ranking runs (E1/E10/E12) are appendix material: E1 places B1/B2/M at 84.2%/98.3%/100.0% under a K=5 bundle for M; E12 shows B2 at 100% every seed and M at 99.7%—M does not lead every seed.

1.6 Research Questions

Evaluation is mechanism- and safety-first; aggregate ranking is secondary.

RQ1—does specification-indexed Semantic Feedback IR improve repair beyond unstructured test-only feedback? (E6; qualitative win coding; field probe); **RQ2**—does the M–baseline gap grow monotonically with guard-overlap density? (E3, E10; negative bound on C4); **RQ3**—does conjunctive Accept improve acceptance safety (PDR/FAR) vs. mean conformance alone? (E2); **RQ4**—under what conditions should practitioners deploy M vs. B2? (deployment boundary; equal-K; ACL case); **RQ5**—quality–overhead trade-off (appendix ranking runs at comparable Conf.).

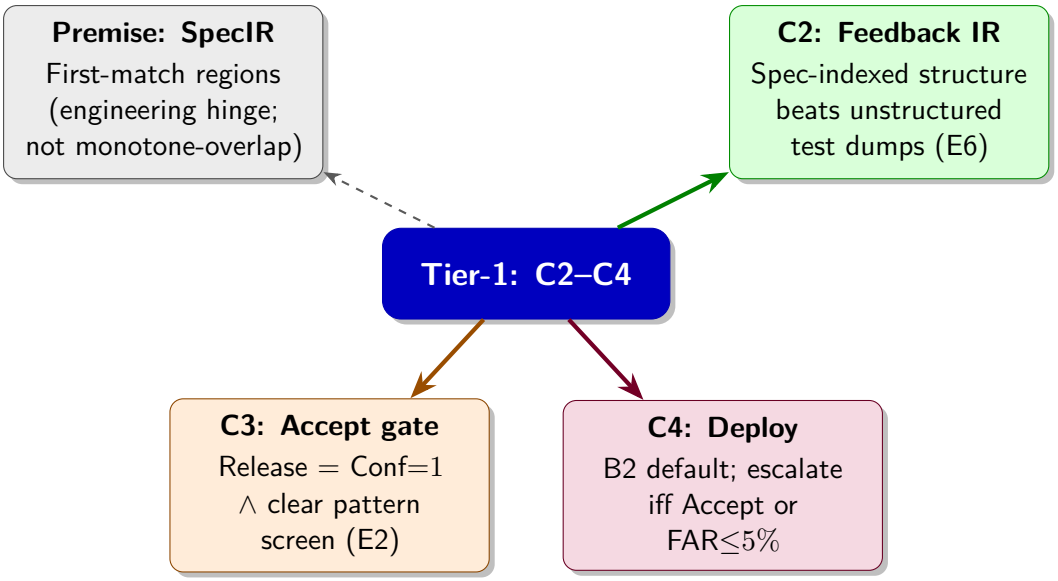

Fig. 1.3. Tier-1 claims are C2–C4, with acceptance safety (C3) as the release headline. SpecIR is the shared engineering hinge, not a monotone-overlap ranking claim. C3 gates release; C2 enables rescue under headroom; C4 chooses when to pay for the full loop.

1.7 Report Organisation

Chapter 2 surveys background and related work (including LLM-as-judge structured feedback). Chapter 3 develops the mathematical core. Chapter 4 designs SpecIR, typed IR, and the pluggable Screen. Chapter 5 is supporting engineering (Screen pluggability over module taxonomy). Chapters 6–7 lead with E6, E2, equal-K, and C4; near-ceiling corpora appear in the appendix. Chapter 8 states scope when baselines saturate; Chapter 9 restates the decision framework. Appendices cover reproducibility, benchmark construction, the pattern catalogue, and near-ceiling ranking runs (Appendix D).

2 Background and Related Work

Specification-conformance prevention draws on several mature ingredients: ordered FSF contracts, LLM synthesis, CEGIS-style repair, SMT witnesses, requirement-level pattern screens, and mutation-based scoring. Prior systems typically assemble only a subset of these for specification-conformance defects. This chapter introduces the ingredients in concept order—SOFL and FSF first, then the synthesis and verification mechanisms that operate on them—and closes with a single positioning of SGDP relative to adjacent lines of work. Formal definitions are deferred to Chapter 3.

2.1 SOFL and Agile-SOFL

SOFL combines Z-style predicate logic, data-flow diagrams, and object-oriented structuring under stepwise refinement[5,42]. Its guarded process scenarios are the historical ancestor of the Functional Specification Format (FSF) used throughout this report (Section 2.2).

Agile-SOFL[36] places that rigour inside sprint workflows: FSF contracts serve as design artefacts, oracles, and acceptance criteria, with automatic conformance checking and SMT-directed test generation before review[8,37]. HSP-Agile builds an LLM-driven synthesis–repair loop on that toolchain, addressing the workflow-integration barrier identified by Woodcock et al.[62]: formal artefacts help only when they participate in everyday delivery, not when they remain specialist sidelines.

2.2 The Functional Specification Format (FSF)

FSF is the primary sprint artefact of Agile-SOFL[36,40]. A task declares parameters, readable and writable externals, and an *ordered* sequence of scenario clauses. Each clause pairs a Boolean guard with an assignment block; the final clause may be an `others` catch-all and must appear last. Guards are not a flat case statement: *order is part of the contract.*

2.2.1 First-match semantics and residual regions

The defining rule is *first-match*: the first satisfied guard determines the output. Two concrete failure modes follow from that rule; both recur under LLM synthesis.

Primary vignette: catch-all residual. On the tier classifier of Chapter 1 (Listing 1.1), named guards handle Reject and Pass bands; everything else falls into `OTHERWISE` and must return `Review`. An LLM candidate that collapses the residual into `Pass` can satisfy every busy-band unit test while shipping a wrong mid-band behaviour—false acceptance in miniature. The catch-all is thin: random suites drawn from Reject/Pass bands often miss it entirely.

Secondary vignette: overlapping guards. On `classify_signal`, input $(level, threshold) = (-3, -10)$ satisfies both S_1 ($level < 0$) and S_2 ($level \geq threshold$), yet FSF requires `"Error"` because S_1 has priority. An implementation that tests the threshold first returns `"Critical"`—a *scenario-order violation* invisible to checkers that treat guards as an unordered Boolean formula.

Chapter 3 defines the oracle g_t and the precedence partition formally (Figure 3.2). Unlike Dijkstra's guarded commands[19], FSF is deterministic priority, not nondeterministic choice.

2.3 LLM-Assisted Code Generation

Contest-style generation optimises for pass@k on hidden tests[11,38,52]. FSF synthesis needs something stricter: conformance to an ordered scenario contract, including residual regions that coarse suites undersample. Deployed assistants often produce *semantically plausible* code that fails at boundaries and multi-condition logic[15]—precisely what FSF guards encode.

Without additional guidance, three failure modes recur[15,57]: (i) *catch-all / residual blindness*—hard-coded defaults that pass busy-band tests (pattern RF07; Section 2.6); (ii) *scenario-order blindness*—overlapping guards bias the model toward a "natural" `if` order rather than mandated precedence; (iii) *boundary hallucination*—confused relational operators at scenario edges (e.g. $<$ vs. $\leq$). Test-feedback repair[11] helps only when tests hit the violated region; fixed suites often miss thin catch-alls and overlap slices. HSP-Agile therefore synthesises region witnesses with Z3 rather than relying on sampled unit tests alone.

2.4 Counterexample-Guided Synthesis (CEGIS)

Repair needs counterexamples that name the violated region, not only a failing assertion. CEGIS splits synthesis into a synthesiser and a verifier that returns counterexamples until success or budget exhaustion[56]. LLM-augmented variants replace templates with free-form generation while retaining formal checks[43,57]; those pipelines target Dafny-style proof obligations rather than FSF first-match regions or requirements-level pattern screens.

HSP-Agile instantiates the SGDP loop with an LLM synthesiser, a Z3-based verifier over suite $\mathcal{D}(t)$, and a feedback aggregator that packages counterexamples $(x^*, g_t(x^*), f_{c_k}(x^*))$ together with high-severity pattern hits (Figure 2.1). Budget K caps iterations; on exhaustion the highest-Conf candidate is returned as best-effort $(\arg\max_k \mathrm{Conf}(c_k, t))$[7]. Experimental budgets and mode definitions are given in Chapter 6.

2.5 SMT Solving and Formal Verification

SMT extends SAT with background theories; a model is a concrete test input[7,16]. For each scenario s_i, HSP-Agile solves the *effective* guard $\varphi_i = g_i(x) \wedge \bigwedge_{j<i} \neg g_j(x)$, guaranteeing at least one witness per first-match region—including catch-alls encoded as the negation of all prior guards. That coverage is targeted rather than random: property-based testing (e.g. QuickCheck) samples broad input families from executable properties[13,69], but does not by itself emit region-typed failure records for ordered FSF partitions.

The resulting oracle is weaker than full deductive verification (e.g. Dafny[35]) yet requires no developer annotations and fits Agile-SOFL sprint velocity. Prevention guarantees remain bounded by suite coverage (Chapter 3).

2.6 Requirement-Level Error Pattern Detection

Static analysers encode recurring structural bug patterns[14,26]. HSP-Agile's pattern guard lifts the idea to *requirements-level* violations: signatures that cannot conform to FSF regardless of tested inputs. Catalogue RF01–RF14[37] (Appendix C) includes missing preconditions, wrong defaults, swapped comparisons, and unconditional success (RF07); high-severity hits ($\mathcal{P}^H$) block acceptance and enter CEGIS repair context even when all formal tests pass. Conjunctive release—witness agreement *and* a clear pattern screen—is the Accept predicate developed in later chapters.

2.7 Mutation Testing for Specification Assessment

Mutation testing injects syntactic faults; survivors indicate inadequate coverage[1,18,49]. HSP-Agile uses two operator families (details in Chapter 6): specification-level SNO, ORO, SPO; implementation-level ICO, WRO, BCO. Prevention quality is scored as *Prevention Detection Rate* (PDR: fraction of injected faults correctly rejected) and *False Accept Rate* (FAR: escape rate among faulty mutants), both over the faulty-mutant set (Definitions 3.23–3.24). These metrics complement Conf. / pass-rate: a candidate may clear a fixed suite yet still accept faulty mutants that encode residual or ordering defects.

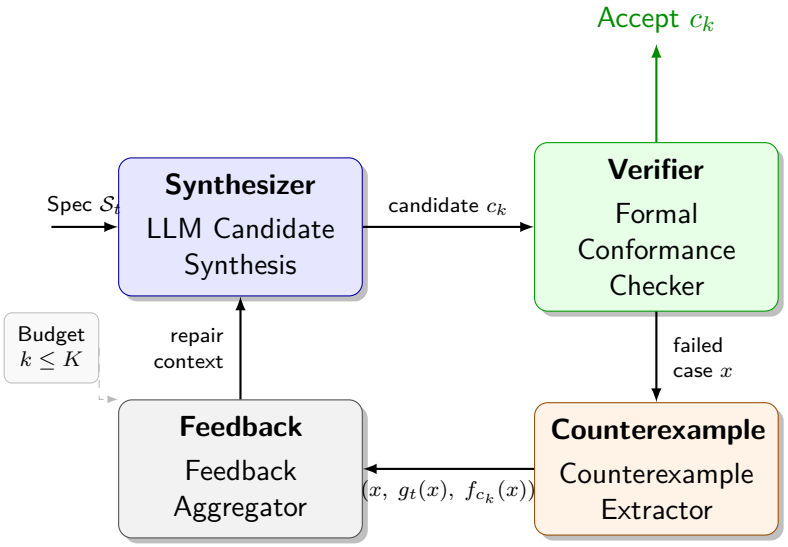

Fig. 2.1. The HSP-Agile CEGIS loop. The LLM synthesiser produces candidate c_k from FSF specification $\mathcal{S}_t$ and repair context. The conformance checker verifies c_k against the Z3-generated test suite; failures become counterexamples. The feedback aggregator packages counterexamples and anti-patterns for the next iteration. If all tests pass and pattern guards are clear, c_k is accepted.

2.8 Related Work

What is missing is not another synthesiser, but a prevention loop that treats ordered guards as first-class evidence and scores release by prevention, not only pass@k. Table 2.2 summarises the comparison on the axes that matter for this report.

Successful formal-method deployments automate narrow, high-value layers inside existing workflows[53,62]; Agile-SOFL follows that prescription, and HSP-Agile automates synthesis and repair without manual annotation. Empirical SE guidance on construct and external validity[1,61] shapes how we separate mechanism claims (typed feedback under E6) from deployment heuristics (when to escalate from a lighter loop to the full Accept gate).

2.8.1 LLM synthesis, agents, and automated program repair

Contest-style generation optimises for pass@k on hidden tests[3,11,38,44,52]. Repository agents (OpenHands, SWE-agent, Agentless, AutoCodeRover) and multi-agent coders target GitHub-scale tasks[27,31,59,64,66,68] without a first-match specification oracle g_t or PDR/FAR protocols. Classical and LLM-based APR (GenProg, TBar, ChatRepair, RepairAgent, and related studies) patch failing tests or buggy snippets[9,21,30,34,39,51,55,60,63,67]; our loop instead repairs against *ordered-guard* witnesses derived from SpecIR and scores prevention, not only pass@k. Verbal self-repair (Self-Refine, Self-Debug, Reflexion) raises generation success via critique or memory[12,47,54] but does not emit region-typed failure records keyed to scenario identity.

2.8.2 LLM-as-judge and verbal structured feedback

A natural alternative to typed IR is to ask another LLM to critique the candidate in natural language—LLM-as-judge / self-refine style feedback[12,47,54]. Those loops can raise generation success, but the critique is not bound to an executable first-match oracle g_t: the judge may invent a plausible story, omit the catch-all scenario index, or disagree with Φ_i witnesses. Semantic Feedback IR is the opposite design choice: structure is *derived from* SMT witnesses and scenario identity, then rendered to text—not elicited as free-form opinion. Baselines B3–B5 instantiate verbal/self-critique styles under equal K on BENCH-120 (Chapter 6); they are related feedback channels, not substitutes for oracle-grounded region records.

2.8.3 Oracles, property-based testing, and symbolic suites

The oracle problem remains central whenever tests are the only gate[6,25,69]. Property-based testing (QuickCheck, Hypothesis) samples broad input families from executable properties[13,46]; search-based and symbolic generators (EvoSuite, KLEE, Pex) raise structural coverage[2,10,23,58]. HSP-Agile complements those lines with solver-directed witnesses for *first-match* regions Φ_i, then packages failures as typed repair records rather than coverage maximisation alone. Mutation testing surveys and operators inform our PDR/FAR protocol[1,18,29].

2.8.4 Contracts, proof-oriented LLM loops, and adjacent notations

Design-by-Contract and embedded contract languages provide runtime oracles[20,48] but do not encode FSF precedence partitions. DafnyBench and Clover-style verifier-in-the-loop systems target proof obligations[17,22,32,43,57]; they are fair references for closed-loop repair, not drop-in FSF comparators. Alloy and TLA+ encode structured constraints[28,33]; SpecIR adapters are the intended bridge when guards are first-match and SMT-encodable. Prior SOFL fault-prevention work assumed human-written implementations[36,37]; LLM synthesis introduces residual and ordering failure modes that neither human-process prevention nor test-only post-processing alone addresses.

Closest LLM+verifier systems differ on three axes: whether witnesses respect first-match order, whether repair carries scenario structure, and whether prevention (not only pass@k) is measured. Our controlled baselines isolate those ingredients: B1 (one-shot LLM), B2 (test-feedback without SMT witnesses), B6 (VerifierLoop-FSF: SMT counterexamples without typed Semantic Feedback IR or a hard pattern screen), and M (full SGDP with IR and Accept gate). Primary ranking and mechanism results appear in Chapter 7; the comparison axes below are design distinctions, not a substitute for those tables.

Table 2.2. Structured comparison of related approaches to LLM-assisted specification-conformant code generation. Checkmarks (✓) indicate full support; dashes (–) absence; "partial" indicates limited support. Columns emphasise the axes that matter for this paper: first-match witnesses, typed repair IR, prevention metrics, and release gate.

Approach	Ordered Φ_i	Typed IR	Prev.	Gate	Primary metric
DafnyBench [43]	–	–	–	proof	pass@k
Sun et al. (Clover) [57]	–	partial	–	proof	correctness
LLM-APR / agents [63,64]	–	–	–	tests	pass@k
Self-Refine / Self-Debug / Reflexion [12,47,54]	–	–	–	verbal	pass@k
Standard LLM (B1)	–	–	–	–	pass rate
Test-feedback (B2)	–	–	–	tests	Conf. / pass
B6 VerifierLoop-FSF	✓	–	–	cex	Conf.
HSP-Agile (ours)	✓	✓	✓	Accept	**Conf, PDR, FAR**

Relative to these lines of work, SGDP's increment is the *combination* of FSF first-match witnesses, specification-indexed Semantic Feedback IR, and conjunctive Accept under PDR/FAR evaluation [11,34,43,57]. Versus B6, the design delta is typed IR plus the Accept/-pattern gate—SMT counterexamples alone do not constitute the full loop. Versus APR and verbal self-debug, the delta is SpecIR-region witnesses and prevention metrics rather than pass@k alone. Versus Clover-style verifier loops, success here is witness agreement under first-match partitions plus a release predicate, not theorem discharge. Property-based testing and symbolic suites raise coverage without emitting region-typed repair records [10,13]; metamorphic and oracle-problem surveys frame why executable SpecIR oracles matter when tests alone undersample thin guards [6,25].

2.9 Gap Summary

Table 2.2 leaves open three design commitments that Chapter 3 makes precise—with SpecIR as the shared engineering hinge:

Premise. Prevention must depend on *semantic regions* (first-match partitions), not surface syntax—realised by SpecIR.

C2. Repair needs *specification-indexed structured feedback*, not raw pass/fail bits—realised by the Semantic Feedback IR (controlled contrast: scenario-indexed IR vs. test-only dumps).

C3. Release must be *conjunctive*: every witness matches *and* no high-severity structural pattern remains (Accept $\Leftrightarrow$ Conf=1 $\wedge$ Screen=pass).

Deployment guidance (C4) then chooses when to pay for the full loop versus a lighter test-feedback baseline (B2): escalate when Accept/FAR headroom or residual-region risk

warrants it. The catch-all vignette of Chapter 1 remains the conceptual anchor for those claims; overlapping-guard tasks are a secondary stress on ordering. Empirical ranking of baselines, equal-K Conf. comparisons, and prevention (PDR/FAR) appear in Chapters 6–7.

3 Formal Problem Definition

Release under a conjunctive gate needs a small set of precise objects: a typed task, a notation-agnostic SpecIR, a first-match oracle with specification-derived witnesses, and an acceptance predicate under budget K. This chapter defines those objects; Chapter 4 turns them into a runnable loop. Notation: $\mathbb{N}$, $\mathbb{Z}$, and $\mathbf{1}[\cdot]$ for the Iverson bracket.

Throughout, the **primary** running example is the catch-all tier classifier of Chapter 1 (Listings 1.1–1.2): busy Reject/Pass tests can clear while the residual `OTHERWISE` band still returns `Pass` instead of `Review`. The industrial-monitoring contract `classify_signal` and the ordered tax brackets of Listing 1.3 appear only as **secondary** illustrations of guard overlap and priority witnesses Φ_i.

3.1 Running Example Roadmap

Before any synthesis loop is built, four commitments must be fixed for a task: its typed signature, its ordered oracle, a finite witness set that respects first-match priority (including residual `others` regions), and a release predicate under budget K. Figure 3.1 shows the SpecIR hinge on the *secondary* overlap vignette `classify_signal`; the same chain applies to the primary catch-all, where the residual case is encoded as $\Phi_n \equiv \neg \bigvee_{j<n} g_j$.

3.2 Formal Task Model

A release decision needs a precise unit of work: one function whose behaviour is described entirely by an ordered scenario specification. That unit is a *formal specification task*. HSP-Agile instantiates the model over SOFL/FSF[36,40]; every later stage consumes one such task.

Definition 3.1 (Formal Specification Task)**.** A *formal specification task* is a triple

$$t \;=\; \big(\mathcal{S}_t,\; \mathcal{X}_t,\; \mathcal{Y}_t\big),$$

where

- $\mathcal{S}_t = \langle s_1, s_2, \ldots, s_n \rangle$ is a finite, ordered sequence of n FSF scenarios (the *specification*);
- $\mathcal{X}_t \subseteq \mathbb{Z}^d$ is the *input domain*, a d-dimensional integer vector space; and

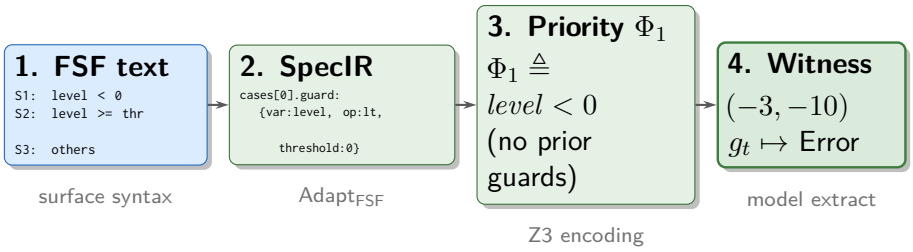

Fig. 3.1. From FSF text to witness (SpecIR hinge). Illustrated on the secondary overlap vignette `classify_signal`: the adapter lowers surface syntax into SpecIR; the witness generator encodes first-match priority as Φ_i; Z3 returns a model such as $(-3, -10)$ that activates S_1. The primary catch-all tier classifier uses the identical chain, with Φ_3 forcing an input that satisfies neither Reject nor Pass.

- $\mathcal{Y}_t \subseteq \mathbb{Z}^m$ is the *output domain*, an m-dimensional integer vector space.

Primary example (catch-all tier). For `classify_tier`, $d = 2$, $m = 1$ (string-valued tier encoded as a discrete label), $\mathcal{S}_t = \langle s_1, s_2, s_3 \rangle$, and $\mathcal{X}_t \subseteq \mathbb{Z}^2$ with coordinates $(score, floor)$. Benchmark tasks in later chapters often use $d = 5$ and $m = 3$; the same definitions apply.

3.3 Specification Intermediate Representation (SpecIR)

The prevention pipeline should not be rewritten for every surface notation. What is needed is a shared hinge: a notation-agnostic form that still preserves ordered guarded cases—**semantic regions**, not surface syntax. That hinge is the **Specification Intermediate Representation** (**SpecIR**). Adapters (Chapter 1; implementation in Chapter 4) map SOFL/FSF, Mini-Z, or Mini-StateMachine text into SpecIR; after that step the shared pipeline never sees the original syntax. Witness generation, conformance, and acceptance all consume SpecIR cases as first-match regions.

Definition 3.2 (Specification Intermediate Representation)**.** For task t, SpecIR$(t) = (\Sigma_t,\ \Gamma_t,\ C_t,\ M_t)$, where Σ_t is the typed I/O signature, Γ_t the variable domain, $C_t = \langle c_1, \ldots, c_n \rangle$ an ordered list of guarded cases (first-match semantics), and M_t notation-specific metadata. The abstract syntax is written SpecIR $\triangleq \langle \Sigma_t, \Gamma_t, C_t, M_t \rangle$ or, equivalently, SpecIR ::= $Signature\ Cases\ [Metadata]$.

Primary example. For `classify_tier`, Σ_t declares inputs `score:int`, `floor:int` and output `tier:string`; C_t holds three guarded cases, ending in an `others` residual whose postcondition is the constant `Review`. Each case is one FSF scenario $s_i = (g_i, \phi_i)$: a Boolean guard $g_i : \mathcal{X}_t \to \{0, 1\}$ and a total output mapping $\phi_i : \mathcal{X}_t \to \mathcal{Y}_t$ (Definition 3.7).

Definition 3.3 (SpecIR Surface Grammar (EBNF))**.** The JSON serialisation (`schemas/spec_ir.schema.json`, `src/ir/spec_ir.py`) is $SpecIR$ ::= $task\_id\ notation\ name\ Signature\ Cases\ [Metadata]$

with $GuardedCase ::= index\ guard\ postcondition$ and $GuardAtom ::= var\ op\ [threshold]$ ($op \in \{eq, ne, lt, le, gt, ge, others\}$).

Definition 3.4 (SpecIR Well-formedness)**.** A SpecIR document d is *well-formed* iff: (i) $notation \in \{sofl, mini\_z, mini\_statemachine, real\_derived\}$; (ii) case indices are consecutive $1, \ldots, n$ with pairwise distinct guards; (iii) every guard *var* (except *others*) is declared in `inputs`, and every postcondition key in `outputs`; and (iv) the final case is *others* or equivalent to $\neg \bigvee_{i<n} g_i$. Enforced by `src/ir/schema_validate.py` (Appendix A).

Definition 3.5 (Specification Adapter)**.** $\text{Adapt}_L : \text{Text}_L \longrightarrow \text{SpecIR}$ (Chapter 4).

Definition 3.6 (FSF Lowering (HSP-Agile Instantiation))**.** $\text{Lower}_{\text{FSF}} : \text{SpecIR} \longrightarrow \text{TaskSpec}_{\text{FSF}}$ projects SpecIR onto an FSF-shaped view for screening; the pipeline core depends only on SpecIR.

Definition 3.7 (FSF Scenario)**.** $s_i = (g_i,\ \phi_i)$ with $g_i : \mathcal{X}_t \to \{0, 1\}$ and $\phi_i : \mathcal{X}_t \to \mathcal{Y}_t$.

Priority is part of the specification, not an implementation detail[19]. The oracle is the function that enforces it.

Definition 3.8 (Scenario Ordering and Specification Oracle)**.** Scenario s_i has *higher priority* than s_j whenever $i < j$. For any input $x \in \mathcal{X}_t$, the *specification oracle* $g_t : \mathcal{X}_t \to \mathcal{Y}_t$ is

$$g_t(x) \;=\; \phi_i(x), \qquad \text{where } i = \min\big\{j : g_j(x) = 1\big\}.$$

The final scenario s_n is the *"others" case* with $g_n(x) = \neg \bigvee_{j=1}^{n-1} g_j(x)$, so every input is handled by exactly one scenario.

Primary example. On $(score, floor) = (3, 10)$, neither g_1 ($score < 0$) nor g_2 ($score \geq floor$) holds, so $g_t(3, 10) = \phi_3(3, 10) = Review$. An implementation that collapses the residual to `Pass` (Listing 1.2) disagrees with the oracle on every mid-band input, while still matching on busy Reject and Pass samples.

Secondary example (ordering / overlap). On `classify_signal` at $(-3, -10)$, both g_1 ($level < 0$) and g_2 ($level \geq threshold$) hold, so $g_t(-3, -10) = \phi_1(-3, -10) = \text{"}Error\text{"}$. An implementation that checks g_2 first returns `"Critical"` and fails the oracle wherever both guards hold—correct on many disjoint inputs, wrong on the overlap.

3.4 The Bounded Acceptance Problem

The oracle answers what a correct implementation must return. Release is stronger: match every specification-derived witness *and* clear a structural screen, within a fixed

LLM budget. We state the acceptance objects first; Section 3.5 then constructs the witnesses that make conformance operational.

Definition 3.9 (Candidate Implementation)**.** Any computable $c : \mathcal{X}_t \to \mathcal{Y}_t$; write f_c for its observable behaviour.

Definition 3.10 (Strict Formal Conformance)**.** Given a finite case set $\mathcal{D}(t) \subseteq \mathcal{X}_t \times \mathcal{Y}_t$ (Definition 3.16),

$$\mathrm{Conf}(c,\, t) \;=\; \frac{1}{|\mathcal{D}(t)|} \sum_{(x,\, y)\, \in\, \mathcal{D}(t)} \mathbf{1}\big[f_c(x) = y\big] \;\in\; [0, 1].$$

Strict success (Definition 3.11) is $\mathrm{Conf}(c, t) \geq \tau$ with $\tau = 1$; a lower τ would mask residual or ordering violations that pass most cases where the omitted region is thin[69].

Primary example. If $\mathcal{D}(t)$ has three witnesses—one Reject, one Pass, one `others`—and the buggy `classify_tier` fails only on the residual, then $\mathrm{Conf}(c, t) = 2/3$.

Definition 3.11 (Strict Success)**.** $\mathrm{Conf}(c,\, t) \geq \tau$ with $\tau = 1$.

Conformance alone is still insufficient: structural anti-patterns can clear every witness. The high-severity screen $\mathcal{P}^H$ (Table 4.5; Appendix C) blocks those cases.

Definition 3.12 (High-Severity Pattern Violation)**.** c is *pattern-safe* iff $\forall p \in \mathcal{P}^H : p(c,\, t) = 0$.

Release is the conjunction of the two checks—neither alone is enough. This is the lead claim of the report: Accept $\Leftrightarrow$ Conf=1 $\wedge$ Screen=*pass*.

Definition 3.13 (Hybrid Acceptance Predicate)**.**

$$\mathrm{Accept}(c,\, t) \;=\; \mathbf{1}\big[\mathrm{Conf}(c,\, t) \geq 1 \;\wedge\; \forall p \in \mathcal{P}^H : p(c,\, t) = 0\big]\,.$$

Primary example. A repaired `classify_tier` with Conf $= 1$ still fails acceptance if RF07 (*unconditional success*) fires on a hard-coded `return "Pass"`. Conversely, clearing the pattern screen while failing the `others` witness also fails Accept. Mean conformance can therefore be high while acceptance stays selective (Chapter 7).

Those pieces assemble into the computational problem: find an acceptable candidate within budget K, or return the best-effort candidate when the budget is exhausted. Semantic feedback on attempt $k{+}1$ is a structured JSON record (counterexamples, pattern hits, attempt index); Chapter 4 details its fields. The search is CEGIS-shaped[56,57] with a stochastic synthesiser—hence the explicit fallback.

Definition 3.14 (Bounded Acceptance Problem)**.** Given task t, budget $K \in \mathbb{N}^+$, and synthesis oracle $\mathrm{LLM}(\cdot)$, find $c^* = \mathrm{LLM}(t, \mathrm{feedback}_1, \ldots, \mathrm{feedback}_{k-1})$ with at most K calls such that $\mathrm{Accept}(c^*, t) = 1$; otherwise return $c^* = \arg\max_{c \in C_K} \mathrm{Conf}(c,\, t)$ where C_K is the set of candidates generated in the K attempts.

3.5 Witnesses and Scenario Semantics

The oracle in Definition 3.8 is a mathematical object. Conformance (Definition 3.10) needs concrete inputs on which to execute candidates. Those inputs are SMT-generated witnesses rather than random samples—one representative per strict first-match region, including the residual `others` band.

Definition 3.15 (Concrete Case)**.** A *concrete case* is $(x,\, g_t(x))$ for $x \in \mathcal{X}_t$; the *case set* is $\mathcal{D}(t)$.

Definition 3.16 (SMT-Generated Witness Suite)**.** For each scenario $s_i = (g_i, \phi_i)$, solve

$$x_i \;\models\; g_i(x) \;\wedge\; \bigwedge_{j<i} \neg\, g_j(x)$$

with Z3[7,16]; if satisfiable, add $\big(x_i,\; g_t(x_i)\big)$ to $\mathcal{D}(t)$. Equivalently, writing $\Phi_i(x) \triangleq g_i(x) \wedge \bigwedge_{j<i} \neg g_j(x)$ (Equation (1.2); Chapter 4, Equation (4.8)), each x_i is a model of Φ_i. The conjunct $\bigwedge_{j<i} \neg\, g_j(x)$ enforces strict scenario membership: x_i activates s_i and no higher-priority scenario, so each case is a direct witness to ϕ_i. For the final `others` case, Φ_n is exactly the residual region missed by busy-band unit tests in the primary vignette.

Primary example. For S_3 (`others`) on `classify_tier`, the query is $\neg(\mathit{score} < 0) \wedge \neg(\mathit{score} \geq \mathit{floor})$; a model might be $(3, 10)$ with oracle label `Review`. Reject and Pass witnesses alone cannot expose Listing 1.2.

Secondary example (`compute_tax`). For S_1 the query is $\mathit{income} < 0$; a model might be $(-1, 0)$ with oracle bracket `Invalid`. For S_2 (foreign-high), $\Phi_2 \equiv (\mathit{income} \geq 120000 \wedge \mathit{is\_foreign}{=}1) \wedge \neg(\mathit{income}{<}0)$; a model is $(150000, 1)$ with oracle `ForeignHigh`—the overlap slice drawn in Figure 1.1.

Lemma 3.17 (Scenario Coverage)**.** *If the SMT solver finds satisfying inputs for all n scenarios, then $\mathcal{D}(t)$ contains at least one representative case per scenario, covering the entire precedence ordering $s_1 \succ s_2 \succ \cdots \succ s_n$.*

Proof. By construction, each x_i satisfies g_i but not $g_1, \ldots, g_{i-1}$. Hence $i = \min\{j : g_j(x_i) = 1\}$, so the oracle evaluates $g_t(x_i) = \phi_i(x_i)$. Each distinct scenario s_i therefore

contributes at least one case whose correct output is determined by ϕ_i, and the precedence ordering is witnessed by all n cases jointly. □

Residual and precedence defects concentrate near thin regions of the input space—catch-all bands and guard boundaries. Random tests rarely land there; when they miss, unit-test repair looks healthy while residual or ordering bugs remain. The corollary below contrasts random sampling with SMT targeting.

Definition 3.18 (Boundary Region)**.** For margin $\delta > 0$, $\mathcal{B}_\delta(t) = \{\mathbf{x} \in \mathcal{X}_t : \exists i,\ \partial g_i(\mathbf{x}) < \delta\}$.

Definition 3.19 (Per-witness Fault-Exposure Probability)**.** For a faulty candidate c,

$$\varepsilon_c \;=\; \Pr_{\mathbf{x}\sim\mathcal{U}(\mathcal{X}_t)}\big[\, f_c(\mathbf{x}) \neq g_t(\mathbf{x}) \,\big],$$

and $\varepsilon_{\min} = \min_{c\in\mathcal{C}\setminus\{g_t\}} \varepsilon_c > 0$ when the faulty class $\mathcal{C}$ is nonempty.

Corollary 3.20 (Random vs. SMT Fault-Exposure Hit Rate)**.** *Let $\rho = \mu(\mathcal{B}_\delta(t))/\mu(\mathcal{X}_t)$ denote the relative measure of the boundary (or residual) region. Assume faults of interest are supported only on $\mathcal{B}_\delta(t)$. Then a single uniform random sample detects such a fault with probability at most ρ. The Constraint-guided Witness Generator produces one witness per strict scenario region via Φ_i (Equation* (4.8)*); under a faithful SMT encoding, each such witness activates scenario i with probability* 1 *(Proposition 4.1). Random sampling therefore reaches precedence-critical or residual regions with probability $\mathcal{O}(\rho)$, whereas SMT achieves targeted coverage of each Φ_i deterministically.*

Proof. Under the support assumption and uniform sampling,

$$\Pr\big[f_c(\mathbf{x}) \neq g_t(\mathbf{x})\big] \;\le\; \Pr\big[\mathbf{x} \in \mathcal{B}_\delta(t)\big] \;=\; \rho.$$

For SMT, satisfiability of Φ_i yields $g_i(\mathbf{x}_i^*) \wedge \bigwedge_{j<i} \neg g_j(\mathbf{x}_i^*)$, so $\mathbf{x}_i^*$ lies in the exclusive first-match region of s_i and activates that region with probability 1 under a faithful encoding (Proposition 4.1). Hence random sampling hits the critical regions with probability $\mathcal{O}(\rho)$, while SMT hits them deterministically. □

Figure 3.2 makes the secondary overlap point geometrically: $(-3, -10)$ also satisfies S_2's guard, yet first-match assigns it to S_1. For the primary catch-all, SMT instead places a witness in Φ_3 where *no* earlier guard holds—the region busy Reject/Pass suites omit.

Definition 3.21 (Strict Scenario Violation)**.** Candidate c *violates* s_i on $(x_i, y_i) \in \mathcal{D}(t)$ if $f_c(x_i) \neq y_i = g_t(x_i)$; the *counterexample* is $(x_i,\ g_t(x_i),\ f_c(x_i))$.

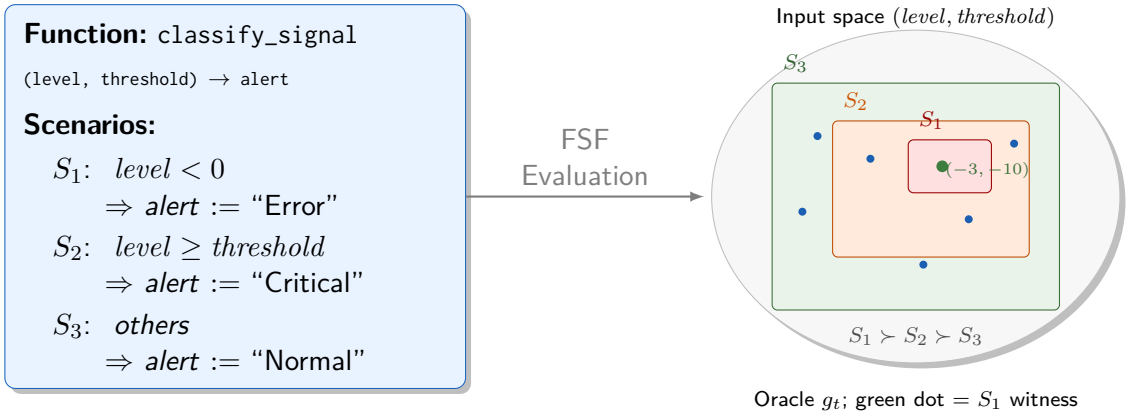

Fig. 3.2. FSF evaluation semantics (secondary overlap vignette). Left: the three-scenario FSF definition of `classify_signal`. Right: input-space partitioning with priority $S_1 \succ S_2 \succ S_3$; the marked point $(-3, -10)$ is an S_1 witness that also lies in the geometric region of S_2's guard. First-match, not geometric inclusion alone, decides the oracle. The primary catch-all vignette is the complementary case: a residual Φ_3 with no earlier guard true.

Primary example. For Listing 1.2 on $(3, 10)$, the counterexample is $\big((3, 10),\ \mathit{Review},\ \mathit{Pass}\big)$ and the violated scenario is s_3 (`others`). Counterexamples name the active scenario, so the repair model can localise the bug inside the ordered specification (Feedback IR fields in Chapter 4).

3.6 Prevention and Quality Metrics

Generation success is the wrong primary metric for a prevention system. What matters is whether faulty candidates are kept out of release, and whether correct ones are not rejected by accident—both measured over a mutant population[1,18].

Definition 3.22 (Mutant Population)**.** $\mathcal{M}$ is a finite set of pairs (c_m, t_m) with at least one side perturbed: $\mathcal{M}^S$ mutates the task, $\mathcal{M}^I$ the candidate; $\mathrm{Faulty}(m) = 1$ when the pair is genuinely faulty.

Let $\mathcal{M}^F = \{m \in \mathcal{M} : \mathrm{Faulty}(m) = 1\}$ be the faulty-mutant subset. Both prevention metrics use $|\mathcal{M}^F|$ as denominator: PDR is the fraction of faulty mutants correctly rejected; FAR is the escape rate among faulty mutants (false-accept rate), *not* a precision analogue[50].

Definition 3.23 (Prevention Detection Rate)**.**

$$\mathrm{PDR} \;=\; \frac{\big|\{m \in \mathcal{M}^F : \neg\,\mathrm{Accept}(c_m,\, t_m)\}\big|}{|\mathcal{M}^F|}.$$

Definition 3.24 (False Accept Rate)**.**

$$\mathrm{FAR} \;=\; \frac{\big|\{m \in \mathcal{M}^F : \mathrm{Accept}(c_m,\, t_m) = 1\}\big|}{|\mathcal{M}^F|}.$$

On a fixed faulty-mutant set, PDR + FAR = 1.

Definition 3.25 (Mutation Kill Rate)**.** $\mu_k = |\{m \in \mathcal{M} : \text{Conf}(c_m, t_m) < 1\}|/|\mathcal{M}|$; the gap PDR $- \mu_k$ is the additive value of pattern screening.

3.7 Benchmark Formalization

The evaluation uses a synthetic benchmark of hard tasks built to stress FSF precedence sensitivity[37], including residual `others` cases and ordered overlaps.

Definition 3.26 (Precedence-Sensitive Task)**.** t is *precedence-sensitive* if there exist s_i, s_j with $i < j$ and overlapping guards: $\{x : g_i(x) = 1\} \cap \{x : g_j(x) = 1\} \neq \emptyset$. On the overlap, output depends on evaluation order whenever ϕ_i and ϕ_j disagree—the class of error naive LLM synthesis frequently introduces on secondary ordering vignettes, and a distinct failure mode from the primary catch-all mismatch.

Proposition 3.27. *Let t^H be precedence-sensitive, and suppose there exist s_i, s_j with $i < j$ and an input x^* in their guard overlap such that $\phi_i(x^*) \neq \phi_j(x^*)$. Then any implementation that evaluates g_j before g_i fails strict conformance on x^*.*

Proof. By Definition 3.8, $g_t(x^*) = \phi_i(x^*)$. If c evaluates g_j before g_i and $g_j(x^*) = 1$, then $f_c(x^*) = \phi_j(x^*)$. The hypothesis $\phi_i(x^*) \neq \phi_j(x^*)$ yields $f_c(x^*) \neq g_t(x^*)$, so $\text{Conf}(c, t^H) < 1$ whenever $\mathcal{D}(t^H)$ includes a witness for the exclusive region of s_i that contains x^* (or x^* itself). □

Hard tasks t^H use five non-negative inputs, three integer outputs (*risk*, *action*, *score*), up to $n = 9$ ordered scenarios, and seven random thresholds $\theta_1, \ldots, \theta_7 \in [1, 100]$; Z3 feasibility and a scenario-consistency validator discard unsatisfiable or ill-covered θ vectors. By Proposition 3.27, every hard task with a semantically relevant overlap requires ordered guard evaluation; SMT witnesses catch order-insensitive LLM outputs. Residual `others` scenarios are covered by the same Φ_i construction (Definition 3.16).

3.8 Computational Considerations

Formal checking must not erase the prevention benefit by dominating runtime. With $n \leq 9$ and $d = 5$, Z3 case generation is $O(n \cdot d^2)$ and finishes in $\sim$1 s; conformance is $O(n)$ lookups; pattern screening is $O(L \cdot |\mathcal{P}^H|)$ and negligible ($\ll$0.01 s)[7,16]. The dominant cost is the LLM call ($\sim$10 s/attempt): with $K = 3$, expected runtime is $\sim$30 s/task, of which $> 97\%$ is LLM inference. In short, $T_{\text{pipeline}} \approx K \cdot T_{\text{LLM}} + O(n \cdot d^2)\, T_{\text{Z3}} + O(n)\, T_{\text{exec}} + O(L \cdot P)\, T_{\text{pattern}}$ with $T_{\text{LLM}} \gg T_{\text{Z3}} \gg T_{\text{exec}} \approx T_{\text{pattern}}$.

Chapter Summary

Three objects dominate: **SpecIR** as the shared semantic hinge, the first-match **oracle** operationalised by SMT witnesses (Lemma 3.17)—including residual catch-alls and ordered overlaps—and the conjunctive gate Accept under budget K (Definition 3.14). PDR/-FAR support the prevention claim; formal checking stays cheap relative to LLM inference. Chapter 4 turns these objects into a runnable loop.

4 The SgDP Framework and HSP-Agile Instantiation

This chapter realises the two mechanisms introduced in Chapter 1: structured scenario-aware repair feedback (C2) and dual-gate acceptance (C3), on top of the SpecIR engineering hinge. Chapter 3 fixed the objects; here they become a loop an LLM can participate in. Adapters and FSF lowering are necessary engineering for the HSP-Agile instantiation, not claimed contributions.

The walkthrough opens with the *primary* vignette from Chapter 1—the catch-all / `OTHERWISE` tier classifier—so the four stages are seen as artefact transforms before the interfaces are named. Ordered-overlap examples (`classify_signal`, `compute_tax`) appear as a *secondary* thread for witness priority formulas Φ_i. Notation is inherited: $t = (\mathcal{S}_t, \mathcal{X}_t, \mathcal{Y}_t)$, oracle g_t, and first-match semantics over $\mathcal{S}_t = \langle s_1, \ldots, s_n \rangle$.

4.1 Pipeline Intuition

Synthesise from FSF text, check on first-match SMT witnesses, repair from a typed failure record, and release only when every witness matches *and* no high-severity structural pattern remains. Figure 1.2 (Chapter 1) is that loop; the dashed orange arc is budget $K = 3$.

4.2 Worked Example: Catch-All Rescue End to End

Primary vignette. Listings 1.1–1.2 define a three-scenario tier classifier: Reject on negative scores, Pass at or above a floor, and `OTHERWISE` must return `Review`. The buggy candidate collapses the residual into `Pass`. Busy Reject/Pass tests all succeed; the mid-band never reaches a human.

The walkthrough below shows how each stage transforms that failure into a repairable record and, after one iteration, into Accept = 1.

Stage 1 (synthesis). The LLM receives the FSF text of Listing 1.1 and emits a Python function. A common first attempt is Listing 1.2: the named guards look correct, but the catch-all returns the wrong constant.

Stage 2 (witnesses and check). The priority formulas for the three scenarios include a residual witness for the catch-all:

$$\Phi_1 \triangleq \mathit{score} < 0, \tag{4.1}$$

$$\Phi_2 \triangleq \mathit{score} \geq \mathit{floor} \ \wedge \ \neg(\mathit{score} < 0), \tag{4.2}$$

$$\Phi_3 \triangleq \neg(\mathit{score} < 0) \ \wedge \ \neg(\mathit{score} \geq \mathit{floor}). \tag{4.3}$$

Z3 returns, for Φ_3, a mid-band model such as $(\mathit{score}, \mathit{floor}) = (3, 10)$. The oracle requires `"Review"`; the buggy candidate returns `"Pass"`, so $\mathrm{Conf}(c_1, t) < 1$.

Stage 3 (Semantic Feedback IR). The failure is recorded as a typed JSON object, then rendered to a repair prompt. In abbreviated form:

> *"For inputs* (*score*=3, *floor*=10)*, your code returned* `Pass`*, but the specification requires* `Review`*. This input satisfies Scenario S3 (*`others`*/*`OTHERWISE`*), the catch-all residual after Reject and Pass."*

Stage 4 and acceptance. After repair restores the correct catch-all constant, every witness matches and the pattern screen is clear, so $\mathrm{Accept}(c_2, t) = 1$. Figure 4.1 makes the conjunctive decision explicit: conformance alone never releases a candidate.

Secondary vignette (ordering / Φ_i). Ordered overlaps motivate the negated higher-priority guards inside Equation (4.8). On `classify_signal`, Φ_1 forces $\mathit{level} < 0$ before the Critical band; a buggy candidate that tests $\mathit{level} \geq \mathit{threshold}$ first returns `Critical` on $(-3, -10)$ where the oracle requires `Error`. Figure 4.2 is that secondary artefact trail; Listings 1.3–1.4 are the tax-bracket twin used in Chapter 1.

For the secondary example the priority formulas are

$$\Phi_1 \triangleq \mathit{level} < 0, \tag{4.4}$$

$$\Phi_2 \triangleq \mathit{level} \geq \mathit{threshold} \ \wedge \ \neg(\mathit{level} < 0), \tag{4.5}$$

$$\Phi_3 \triangleq \neg(\mathit{level} < 0) \ \wedge \ \neg(\mathit{level} \geq \mathit{threshold}). \tag{4.6}$$

4.3 The SgDP Framework

4.3.1 Framework Overview and Instantiation Interface

The walkthrough above already used four ideas that are not FSF-specific: an oracle derived from the specification, witnesses that hit first-match regions, a structured failure

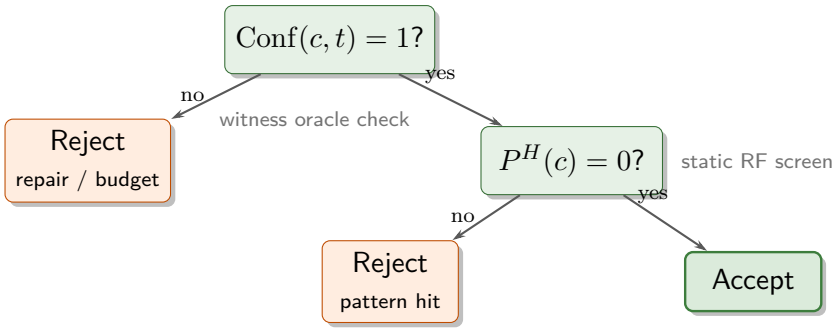

Fig. 4.1. Acceptance decision tree. Release requires both full witness conformance and a clear high-severity pattern screen. Takeaway: high mean conformance does not imply acceptance when residual witnesses or patterns still fail.

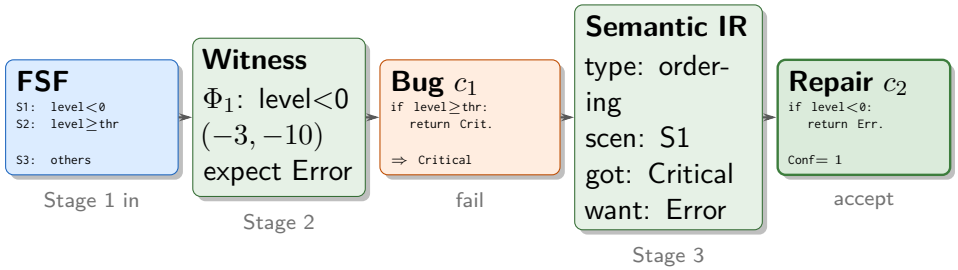

Fig. 4.2. Secondary worked-example trail (`classify_signal`). Ordering bug on Φ_1: FSF text yields witness $(-3,-10)$; the buggy candidate returns `Critical`; Semantic Feedback IR names the precedence violation; repair restores S_1-first evaluation. Primary vignette remains the catch-all tier classifier above.

record for repair, and a conjunctive release gate. Those ideas are the language-independent core of SGDP.

1. **Specification Oracle** ($\mathcal{G}$). An executable oracle derived from the formal specification, without a reference implementation.

2. **Constraint-guided Witness Generator** ($\mathcal{W}$). SMT-based witnesses aimed at the boundary and residual regions where conformance defects appear.

3. **Scenario-aware Semantic Feedback IR** ($\mathcal{F}$). Structured failure records that carry specification-level semantics into repair prompts, instead of raw pass/fail bits.

4. **Conjunctive Acceptance Predicate** ($\mathcal{A}$). A release gate that demands both oracle-grounded conformance and specification-derived quality criteria.

Figure 1.2 (Chapter 1) remains the map. An instantiation for language $\mathcal{L}$ provides: (i) Specification Adapter $\mathrm{Adapt}_L : \mathrm{Text}_L \to \mathrm{SpecIR}$; (ii) oracle g_t derived from $\mathrm{SpecIR}(t)$.cases under first-match; (iii) witness generator $\mathcal{W}(t)$ from SpecIR guard formulas; (iv) optional lowerer $\mathrm{Lower}_{\mathrm{FSF}}$ (HSP-Agile) $\to$ FSF TaskSpec; (v) Semantic Feedback IR per `schemas/semantic_feedback.schema.json`, rendered by $\rho : \mathcal{F} \to \mathrm{Prompt}$. Synthesis, repair, the pluggable structural screen, and acceptance are shared core components.

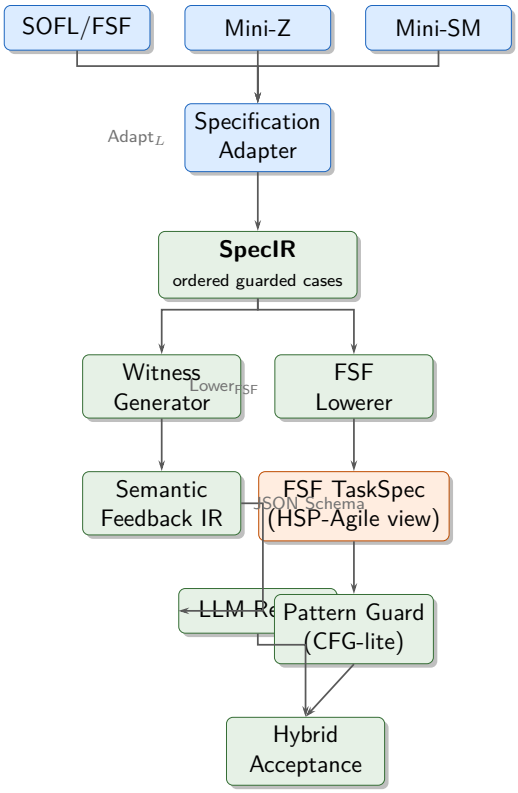

Fig. 4.3. Key observation: notation changes stop at the adapter; the shared pipeline never sees SOFL, Mini-Z, or Mini-StateMachine text. **How to read:** adapters into SpecIR; optional $\text{Lower}_{\text{FSF}}$ for FSF-shaped screening; full context in Figure 1.2.

4.3.2 HSP-Agile: SOFL/FSF Instantiation

HSP-Agile instantiates SGDP over FSF[36,40]. Each scenario $s_i = (g_i, \phi_i)$ pairs a guard with a postcondition; under *first-match semantics* the output for $\mathbf{x}$ is $\phi_i(\mathbf{x})$ where $i = \min\{j : g_j(\mathbf{x})\}$. The FSF oracle requires no reference implementation:

$$g_t(\mathbf{x}) = \phi_i(\mathbf{x}) \text{ where } i = \min\{j : g_j(\mathbf{x})\}. \tag{4.7}$$

4.3.3 Constraint-guided Witness Generation

Random tests rarely land in thin first-match regions—neither catch-all residuals nor multi-conjunct overlaps. What is needed is one input per scenario that activates that scenario *and no higher-priority one*. For scenario s_i, the witness generator therefore solves the *priority formula*

$$\Phi_i \triangleq \hat{g}_i(\mathbf{x}) \wedge \bigwedge_{j=1}^{i-1} \neg \hat{g}_j(\mathbf{x}), \tag{4.8}$$

where $\hat{g}_j$ is the Z3 encoding of g_j. The negated higher guards are essential: without them, a model of g_i alone may still belong to s_j for some $j < i$, and a catch-all encoded as `True` without negations would not isolate the residual. The witness set is $\mathcal{D}(t) = \{\mathbf{x}_1^*, \ldots, \mathbf{x}_n^*\}$.

Proposition 4.1 (Scenario-aware Witness Soundness)**.** *Assume $\hat{g}_i$ soundly encodes g_i and*

Z3 returns a model $\mathbf{x}^*$ *of* Φ_i*. Then* $g_i(\mathbf{x}^*)$ *holds and* $g_j(\mathbf{x}^*)$ *is false for all* $j < i$*; hence* $g_t(\mathbf{x}^*) = \phi_i(\mathbf{x}^*)$*.*

Proof. By the encoding assumption and Z3 soundness, $g_i(\mathbf{x}^*)$ and $\neg g_j(\mathbf{x}^*)$ for every $j < i$. By Definition 3.8, $i = \min\{j : g_j(\mathbf{x}^*)\}$, so $g_t(\mathbf{x}^*) = \phi_i(\mathbf{x}^*)$. □

4.3.4 Scenario-aware Semantic Feedback IR

Raw failing tests omit the information the repair model needs: *which* scenario failed, *why* the region matters (catch-all residual or ordering precedence), and *what* the oracle expected. That information is organised into a structured semantic record before any natural-language prompt is rendered—the Semantic Feedback IR.

The IR is designed so that scenario identity, guard text, observed/expected outputs, and a typed violation reason travel together into the repair prompt, rather than as an unstructured pass/fail dump. Empirical comparisons of feedback surfaces (E6, E14, and related ablations) are reported in Chapter 7; here we fix the representation and the renderer contract.

$$\mathcal{F}(i, c, t) = \langle \textit{type},\ s_i,\ g_i,\ \mathbf{x}_i^*,\ c(\mathbf{x}_i^*),\ g_t(\mathbf{x}_i^*),\ \textit{reason},\ \textit{priority},\ \textit{fix} \rangle, \tag{4.9}$$

with *type* $\in \{\text{ordering}, \text{boundary}, \text{arithmetic}, \text{output}\}$, ranked *priority*, and suggested *fix*. A template then renders the IR into a repair prompt that names the violated scenario, states the guard, and states the expected postcondition.

Example (primary: catch-all). The buggy candidate of Listing 1.2 yields the JSON record below (abbreviated). The renderer projects the same object into the natural-language quote of Section 4.2:

```
{
  "violation_type": "output",
  "scenario_index": 3,
  "constraint_text": "OTHERWISE (not Reject, not Pass)",
  "inputs": {"score": 3, "floor": 10},
  "observed": "Pass",
  "expected": "Review",
  "reason": "catch-all others returned wrong constant",
  "priority": "high",
  "fix": "return Review on residual band"
}
```

Serialisation follows `schemas/semantic_feedback.schema.json`. `FeedbackRenderer` projects the same IR onto the controlled E6 surfaces (`test_only`, `test_expected`, full `semantic_ir`) and length-matched E14 surfaces (including `execution_trace_matched`), separating failure

structure from prompt text. Default mode M may render `execution_trace_matched` for repair; E6 isolates `semantic_ir` vs. `test_only` as the primary C2 comparison (Chapter 7).

Example (secondary: ordering). On `classify_signal`, an ordering failure on $(-3, -10)$ is recorded with `violation_type: ordering`, scenario index 1, observed `Critical`, expected `Error`, and a fix hint that restores S_1-first evaluation—the Semantic IR behind Figure 4.2.

4.3.5 Formal Conformance and Acceptance Predicate

Conformance answers a continuous question: on what fraction of witnesses does the candidate match the oracle? Acceptance answers a binary *acceptance-safety* question: is the candidate safe enough to ship under the conjunctive gate of Figure 4.1—witnesses *and* a structural screen?

$$\mathrm{Conf}(c,t) = \frac{1}{|\mathcal{D}(t)|} \sum_{\mathbf{x} \in \mathcal{D}(t)} \mathbf{1}\big[f_c(\mathbf{x}) = g_t(\mathbf{x})\big]. \tag{4.10}$$

Let $\mathrm{Screen}(c,t)$ be the Stage 4 structural oracle (Section 4.9). In this report's PoC, Screen fails iff any smell in the high-severity set $\mathcal{P}^H$ fires (Table 4.5); the interface itself is not tied to that catalogue. Release then requires both full witness agreement and a clear screen:

$$\mathrm{Accept}(c,t) \iff \mathrm{Conf}(c,t) = 1 \ \wedge\ \mathrm{Screen}(c,t) = \textit{pass}. \tag{4.11}$$

Equivalently, with the PoC encoding $\mathrm{Screen}(c,t) = \textit{pass} \iff \sum_{p \in \mathcal{P}^H} \mathbf{1}[p(c,t)] = 0$. On the secondary tax vignette, Equation (4.11) rejects both a foreign-high ordering bug exposed by Φ_2 (Conf<1) and an unconditional `return "Low"` shortcut that might still look clean on a lucky domestic suite (Screen fails via RF07).

Proposition 4.2 (Finite-Sample FAR Bound under Independent Exposure)**.** Assumptions. *Fix a faulty candidate and a task t with $|\mathcal{D}(t)| = n$ witnesses. Assume (i) each witness exposes the fault independently with probability at least $\varepsilon > 0$ (Definition 3.19), and (ii) witness draws are non-adversarial (i.i.d. relative to the fault). Then*

$$\mathrm{FAR} \ \le\ (1-\varepsilon)^n.$$

Proof. Write E_i for the event that witness i fails to expose the fault. Independence and $\Pr[E_i] \le 1 - \varepsilon$ give

$$\Pr[\text{all pass} \mid \text{faulty}] \ =\ \prod_{i=1}^{n} \Pr[E_i] \ \le\ (1-\varepsilon)^n.$$

Hence $\mathrm{FAR} \le (1-\varepsilon)^n$. □

Remark (Scope relative to the deployed suite). Proposition 4.2 is a finite-sample envelope under the stated i.i.d. exposure assumptions. The deployed pipeline uses deterministic priority-encoded models (Equation (4.8)), not independent random draws; Proposition 4.1 ensures each $\mathbf{x} \in \mathcal{D}(t)$ lies in a strict first-match region under a faithful encoding, so ε is bounded away from zero for the mutation operators of Section 6.6 (Corollary 3.20). Acceptance under $\text{Accept}(c, t)$ does not imply correctness on $\mathcal{X}_t \setminus \mathcal{D}(t)$. Witness cardinality also differs by stage: the repair loop uses up to $N_{\max}$ cases (16 for mode M); strict evaluation uses 64 cases; the illustrative calculation in Section 4.3.7 uses n=9 as an example only.

4.3.6 Structured Feedback and Repair Hypothesis

Hypothesis 4.3 (Expected Improvement under Structured Feedback). Under fixed model and budget K, repair guided by Semantic Feedback IR (which names the violated scenario, guard predicate, and violation type) achieves higher expected conformance gain per iteration than repair guided by plain test-pass/fail feedback.

Chapter 7 tests this hypothesis empirically (Figures 7.2 and 7.1); a fully formal stochastic model of LLM repair is beyond current theory.

4.3.7 Empirical FAR Envelope

With $\varepsilon = 0.08$ and illustrative $n = 9$ (not the pipeline's $N_{\max}$=16 or strict-eval 64), Proposition 4.2 gives FAR $\leq (0.92)^9 \approx 0.473$ ($\approx 47\%$). Chapter 7 reports the operational E2 impl-screening rates (mode M FAR 5.0%, B2 8.8%; PDR 95.0% / 91.2%; Table 4.2)—well inside that conservative envelope.

Table 4.2. Finite-sample FAR envelope vs. empirical E2 FAR (impl-screening; full protocol in Section 7.2).

Mode	Empirical PDR	Empirical FAR	Bound $(1-\varepsilon)^n$
B2	91.2%	8.8%	—
M	95.0%	5.0%	$\approx 47\%$

4.3.8 Prevention-Oriented Metrics

For the faulty-mutant set $\mathcal{M}^F = \{m \in \mathcal{M} : \text{Faulty}(m) = 1\}$ (Definitions 3.23–3.24):

$$\text{PDR} = \frac{|\{m \in \mathcal{M}^F : \neg\text{Accept}(c_m, t_m)\}|}{|\mathcal{M}^F|}, \qquad \text{FAR} = \frac{|\{m \in \mathcal{M}^F : \text{Accept}(c_m, t_m)\}|}{|\mathcal{M}^F|}. \tag{4.12}$$

PDR, FAR, strict formal conformance, and latency are reported jointly; on a fixed faulty set, PDR + FAR = 1.

4.3.9 Pipeline Complexity Cost Model

With $n = |\mathcal{S}_t|$ scenarios and budget K, witness generation costs $\mathcal{O}(\sum_{i=1}^{n} T_{\text{SMT}}(|\Phi_i|))$ once per task; the dominant per-run term is $\mathcal{O}(K \cdot (T_{\text{LLM}} + n \cdot T_{\text{exec}}))$. E3–E4 confirm sub-linear cost relative to conformance gain (Corollary 3.20).

4.3.10 Full Algorithmic Procedure

On budget exhaustion the implementation returns the best-effort candidate $\arg\max_{k \le K} \text{Conf}(c_k, t)$ among attempts (ties broken by formal pass and fewer blocking pattern hits), not necessarily c_K. Each iteration also logs $\langle k, \text{Conf}(c_k, t), |\text{failures}_k|, |\text{patterns}_k| \rangle$ to `attempt_history` for repair-dynamics analyses in Chapter 7.

4.4 Design Principles

Five principles[11,62]: (P1) oracle/SMT precision over heuristics; (P2) hard budget K with guaranteed termination; (P3) JSON Semantic Feedback IR before natural-language projection; (P4) conjunctive acceptance (equation (4.11)); (P5) modular stage flags for ablation (Chapter 6).

4.5 Pipeline Overview

The four stages of Algorithm 1 iterate until $\text{Accept}(c_k, t) = 1$ or $k = K$. Figure 1.2 is the architectural map; Figure 4.5 is the message-level sequence of one run.

4.6 Stage 1: LLM-Based Candidate Synthesis

Stage 1 maps $(t, \mathit{feedback}_{k-1})$ to candidate c_k (the only non-deterministic stage).

4.6.1 Prompt Structure

Prompt $\mathcal{P}(t, k)$ uses a fixed system message plus a user message with $(\mathcal{S}_t, \mathit{sig}(t))$ and, when $k > 1$, the rendered Feedback IR[11,52].

4.6.2 Model Configuration and Parsing

Qwen2.5-27B-Instruct with $T_{\text{gen}} = 0.7$ then $T_{\text{repair}} = 0.0$ (Chapter 6); the parser extracts one fenced Python block or sets $c_k = \bot$.

$$c_k \;=\; \text{LLM}\big(t,\; \mathit{feedback}_{k-1}\big)\,. \tag{4.13}$$

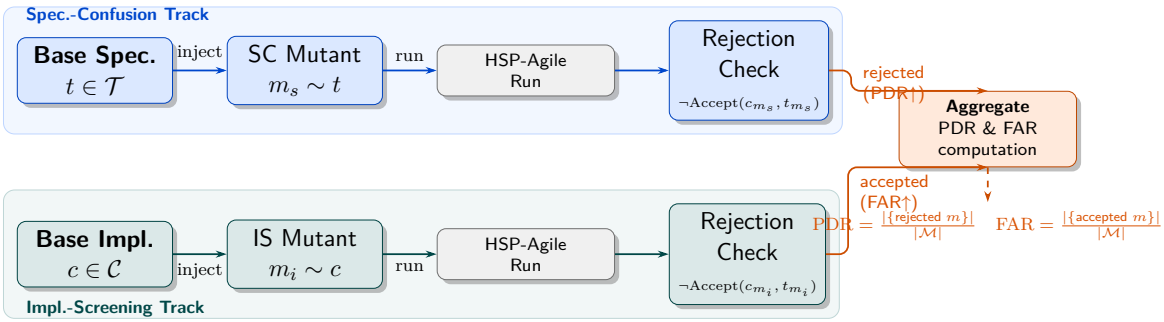

Fig. 4.4. Two-layer prevention evaluation: specification-confusion and implementation-screening mutant tracks aggregated to PDR and FAR.

Algorithm 1 HSP-Agile (SGDP instantiation)

Input: task t, budget K
Output: accepted candidate or best-effort $\arg\max_k \text{Conf}(c_k, t)$

$\mathcal{D}(t) \leftarrow \mathcal{W}(t)$
feedback$_0 \leftarrow \emptyset$; best $\leftarrow$ `null`
for $k = 1$ to K **do**
  $c_k \leftarrow \text{LLM}(t, \text{feedback}_{k-1})$
  $\text{Conf}(c_k, t)$, failures$_k \leftarrow$ eval c_k on $\mathcal{D}(t)$ via g_t
  patterns$_k \leftarrow$ pattern guard on c_k
  if $\text{Accept}(c_k, t)$ **then return** c_k, accepted
  end if
  $\mathcal{J}_k \leftarrow \text{ToJson}(\mathcal{F}(\text{failures}_k))$ ▷ schema: `semantic_feedback.schema.json`
  feedback$_k \leftarrow \rho(\mathcal{J}_k)$ ▷ `FeedbackRenderer` projection (E6 / E14 surfaces)
  feedback$_k$ += top-5 pattern violations from patterns$_k$
  update best if $\text{Conf}(c_k, t) > \text{Conf}(\text{best}, t)$ ▷ track $\arg\max$ Conf. over attempts
  log $\langle k, \text{Conf}(c_k, t), |\text{failures}_k|, |\text{patterns}_k| \rangle$
end forreturn best, not-accepted ▷ best-effort: highest-Conf. candidate, not last

4.7 Stage 2: SMT-Driven Formal Conformance Checking

Stage 2 realises $\mathcal{W}$ and Conf (equations (4.8)–(4.10)): solve each Φ_i, evaluate g_t on the model, execute c_k in a sandbox, and collect mismatches as $cexs_k$[7,16]. Algorithm 2 is the procedure; Listing 4.5 shows the Z3 encoding for one scenario.

```
solver = z3.Solver()
solver.add(phi_i)          # phi_i is the Z3 formula for Phi_i
if solver.check() == z3.sat:
    model = solver.model()
    concrete = {v: model[v].as_long() for v in z3_vars}
    D.append(concrete)
```

Listing 4.5: Z3 test-case generation for scenario i.

Table 4.3. Core Symbols and Definitions

Symbol	Meaning
t	A benchmark task derived from Agile-SOFL specification or hard synthetic generator.
$(\mathcal{S}_t, \mathcal{X}_t, \mathcal{Y}_t)$	Task tuple: ordered scenario sequence, input space, and output space for t.
$\mathcal{S}_t$	Ordered FSF scenario set for task t.
$\mathcal{D}(t)$	Concrete case set generated from $\mathcal{S}_t$ by SMT-guided instantiation.
c, c_k	Candidate implementation generated by LLM; c_k denotes the attempt-k candidate.
f_c	Observable output function induced by candidate c.
g_t	Specification-consistent oracle function for task t.
K	Maximum synthesis-repair attempts allowed for one task run.
τ	Strict success threshold for conformance (set to 1 in this study).
$\mathrm{Conf}(c,t)$	Strict formal conformance score, defined in Eq. 4.10; reported as `strict_formal_conformance` in artifacts.
$\mathrm{StrictSuccess}(c,t)$	Indicator that $\mathrm{Conf}(c,t) \geq \tau$, Definition 3.11.
$\mathrm{Accept}(c,t)$	Hybrid acceptance predicate with formal+pattern constraints, Eq. 4.11.
$\mathcal{P}^H$	High-severity requirement-fault pattern set.
$\mathrm{feedback}_k$	Structured repair context after attempt k (counterexamples + pattern violations).
$\mathcal{M}$	Mutant population for prevention evaluation.
PDR	Prevention detection rate over faulty mutants.
FAR	False-accept / escape rate among faulty mutants.
`mutation_kill_rate`	Artifact metric for the proportion of mutants killed by a mode on the evaluated mutant population.

4.7.1 Why SMT Over Random Testing

Random sampling rarely hits multi-conjunct first-match regions or catch-all residuals[13,69]. Φ_i places each witness in scenario i's exclusive region (Proposition 4.1), so oracle labels are unambiguous.

4.8 Stage 3: Counterexample-Guided Repair

Stage 3 builds the Semantic Feedback IR (Section 4.3.4) and renders it via ρ; Figure 4.6 shows the feedback arc for the primary catch-all vignette (quote in Section 4.2). This is CEGIS with the LLM as synthesiser and the checker as verifier[56,57].

Table 4.4. Core Terminology

Term	Meaning
Specification-conformance defect	A fault where generated code behaves correctly on typical inputs yet violates the formal specification on precedence-critical or low-measure boundary regions that ordinary unit tests rarely reach.
First-match semantics	Evaluation rule for ordered guard scenarios: the first scenario whose guard holds determines the output; later scenarios apply only to inputs not already claimed by earlier guards.
Witness	A concrete input produced by SMT solving so that a designated scenario is the unique first match; used as an oracle test case.
Strict formal conformance	Fraction of SMT witnesses on which the candidate matches the specification oracle ($\mathrm{Conf}(c,t)$).
Strict success / acceptance	Release decision $\mathrm{Accept}(c,t)$: full witness conformance *and* no high-severity pattern hits. Conformance can be high while acceptance remains low when residual witnesses or patterns still fail.
Semantic Feedback IR	Typed JSON record of a conformance failure (scenario, guard, witness, observed vs. expected output, violation type) rendered into repair prompts.
SpecIR	Notation-agnostic intermediate representation of ordered guarded cases; adapters map surface languages (FSF, Mini-Z, Mini-SM) into SpecIR.

4.9 Stage 4: Configurable Static Screen inside Dual-Gate Acceptance

4.9.1 Why Witnesses Alone Are Not Enough

Finite SMT witnesses answer a *pointwise* question: does the candidate match the oracle on forced first-match regions? They do not answer a *structural* question: did the model cheat the suite? A degenerate candidate that always returns a constant success flag can survive a thin suite that never exercises an overlap or a catch-all residual—the LLM "shortcut" failure mode[15,26]. Acceptance safety therefore needs a second conjunct after Conf.

Stage 4 is any oracle

$$\mathrm{Screen} : (\mathit{code},\, t) \longrightarrow \{\mathit{pass}, \mathit{fail}\} \tag{4.14}$$

that emits severity-tagged hits into repair context. Conjunctive Accept requires $\mathrm{Conf}(c,t)=1$ *and* $\mathrm{Screen}(c,t)=\mathit{pass}$ (Figure 4.1; Equation (4.11)). On the ACL case (Section 7.4), witnesses already refuse the fail-open residual; Screen is the second latch when a sparse suite is unlucky.

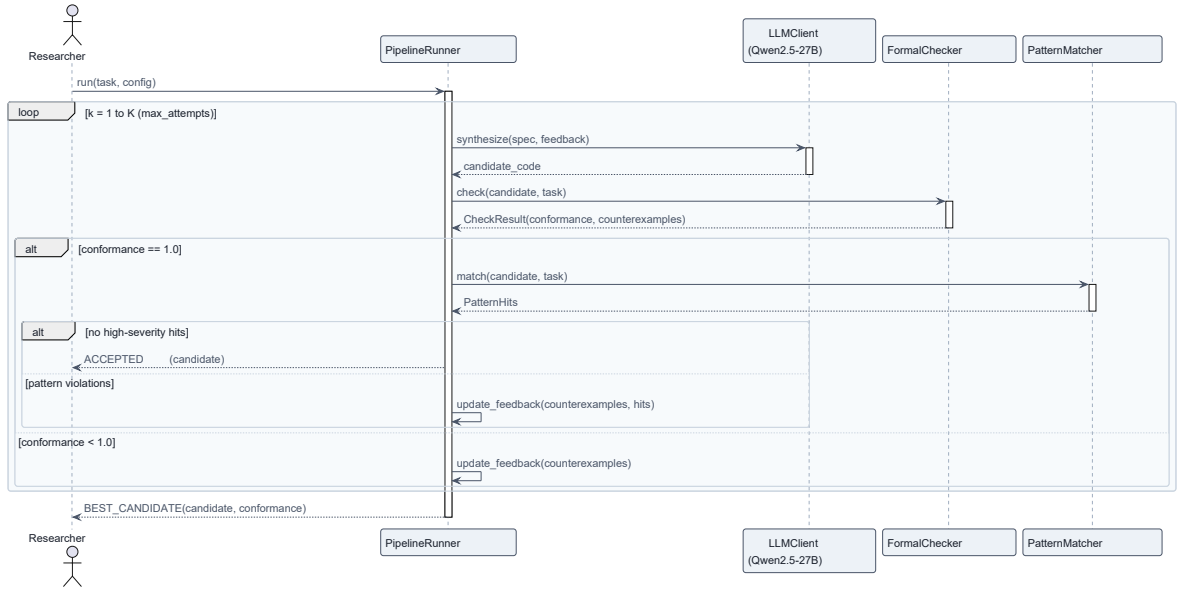

Fig. 4.5. Key observation: one run is a message sequence until Accept or budget K, not a single LLM call. **How to read:** synthesis → check → feedback → accept; the loop arc is Stage 3.

4.9.2 Concrete Instantiation: AST Code-Smell Patterns (PoC)

This study instantiates Screen as a catalogue of requirements-level smells RF01–RF14[37] (Appendix C). Detectors operate primarily on the Python AST (control-flow and return shape), with light surface checks where a smell is lexical. Table 4.5 lists the fourteen smells; nine high-severity ones form the blocking set $\mathcal{P}^H$ in Equation (4.11), with RF07 (*unconditional success*) as the canonical critical shortcut on fail-closed contracts. The catalogue is a reproducible screen instance, not an exhaustive defect model. Its operators are grounded in prior SOFL fault-prevention patterns, whereas E2 also includes first-match and postcondition mutants that do not map one-to-one onto these smells. This separation prevents the prevention result from reducing to a detector tested only on its own templates.

Algorithm 2 SMT-Driven Formal Conformance Checking

Input: task t, candidate c
Output: $\mathrm{Conf}(c,t) \in [0,1]$, counterexample list *cexs*

$\mathcal{D}(t) \leftarrow \emptyset$; $cexs \leftarrow \emptyset$
for $i = 1$ **to** n **do**
  $\Phi_i \leftarrow \textsc{TranslateToZ3}(s_i, \{g_j\}_{j<i})$
  if Z3-$\textsc{Solve}(\Phi_i) = sat$ **then**
    $x_i \leftarrow \textsc{ExtractModel}()$; $y_i^* \leftarrow \phi_i(x_i)$
    $\mathcal{D}(t) \leftarrow \mathcal{D}(t) \cup \{(x_i, y_i^*)\}$
  end if
end for
for each $(x, y^*) \in \mathcal{D}(t)$ **do**
  $y^c \leftarrow f_c(x)$
  if $y^c \neq y^*$ **then**
    $cexs \leftarrow cexs \cup \{(x, y^*, y^c, \textsc{ScenarioOf}(x))\}$
  end if
end for
return $(|\mathcal{D}(t)| - |cexs|)/|\mathcal{D}(t)|$, *cexs*

Table 4.5. PoC structural smell catalogue ($|\mathcal{P}| = 14$). Instance of the pluggable Screen interface. **critical** smells fail immediately; *high* smells belong to the blocking set $\mathcal{P}^H$.

ID	Name	Category	Severity
RF01	Missing precondition check	requirement	high
RF02	Wrong default branch	FSF	high
RF03	Hardcoded magic number	implementation	medium
RF04	Missing output field	interface	high
RF05	Swapped comparison	logic	high
RF06	No guard for zero input	boundary	high
RF07	Unconditional success	logic	**critical**
RF08	Missing type coercion	implementation	medium
RF09	Incomplete FSF coverage	FSF	high
RF10	External variable ignored	SOFL	medium
RF11	Off-by-one boundary	boundary	high
RF12	Duplicate assignment	implementation	low
RF13	Empty handler	logic	high
RF14	Spec-comment mismatch	requirement	medium

4.9.3 Configurable Screen Interface

The evaluated mechanism is the second-gate role of a static screen inside Accept; RF01–RF14 are a reproducible PoC chosen for alignment with prior SOFL fault-prevention patterns[37]. Any component that consumes (c,t) and emits severity-tagged hits—a project smell pack, a CI static analyser[4,26], or an LLM review agent—can replace the PoC detectors without changing SpecIR witnesses, Semantic Feedback IR, or Equation (4.11).

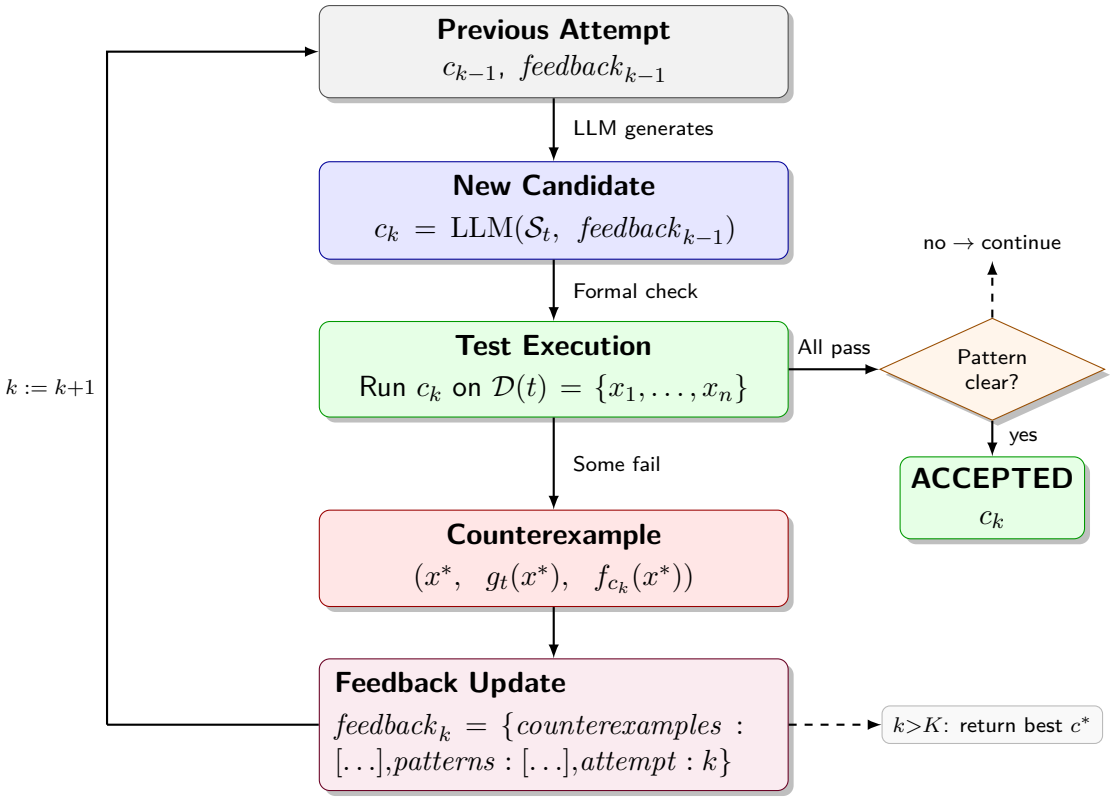

Fig. 4.6. Key observation (C2): repair names the violated scenario and expected post-condition, not only raw I/O. **How to read:** failed attempt → Semantic Feedback IR → next synthesis. Controlled feedback comparisons are in Chapter 7.

4.9.4 Detection Method (PoC Wiring)

Each PoC smell $p \in \mathcal{P}$ has detector δ_p; structural smells use AST analysis, a minority use surface matching, and guard-comparison smells cross-reference FSF guards with candidate conditionals. $\text{Hits}(c,t) = \{p : \delta_p(c,t) = 1\}$; only $\mathcal{P}^H$ blocks acceptance. Replacing $\{\delta_p\}$ with another Screen implementation leaves Equations (4.10)–(4.11) unchanged.

4.10 Pipeline Stage Summary

Table 4.7 contracts the stages under Algorithm 1. The Tier-1 claims realised here are SpecIR semantic regions (C1), Semantic Feedback IR (C2), and conjunctive acceptance (C3).

Table 4.7. Summary of HSP-Agile pipeline stages.

Stage	Name	Input	Output	Property
1	Synthesis	t, *fb*	c_k	stochastic
2	Formal Check	c_k, t	Conf, *cexs*	sound (Prop. 4.1)
3	Repair	*cexs*, *hits*	fb_k	CEGIS bridge
4	Structural Screen	c_k, t	*hits* / veto	pluggable (Screen)
—	Accept	Conf, Screen	$\{0,1\}$	conjunctive (Prop. 4.2)

5 Implementation

This chapter realises SpecIR and the SgDP loop as a deployable system: which module owns which failure mode, what can be swapped for ablation, and where latency is spent. Algorithms and acceptance semantics remain in Chapter 4; here the concern is software realisation—adapters, checker wiring, Semantic Feedback IR serialisation, and the conjunctive release gate. Notation follows (4.11): $t = (\mathcal{S}_t, \mathcal{X}_t, \mathcal{Y}_t)$, candidate c, $\mathrm{Conf}(c, t)$, and $\mathrm{Accept}(c, t)$.

The primary vignette is the tier classifier of Chapter 1: `OTHERWISE` must return `Review`, yet a smooth candidate emits `Pass` while busy Reject/Pass tests stay green. On that residual, the same modules build a Z3 catch-all witness, emit a typed Semantic Feedback IR record, and decide Accept. Ordered-overlap tasks such as `classify_signal` exercise the same path as a secondary regime.

5.1 System Architecture Overview

Stages 1–4 must be ablatable and auditable without rewriting the rest (Chapter 6), so ingestion, orchestration, and external services stay apart. Figure 5.1 shows the layout: pipeline core at the centre, LLM/SMT clients below, and experiment orchestration above—the software view of Figure 1.2.

`PipelineRunner` owns the loop: an immutable `PipelineConfig`, delegation to `ECNUClient`, `FormalChecker`, and `PatternMatcher` (Figure 5.2). Value objects carry serialisable audit traces. E8 adapters (Mini-Z, Mini-StateMachine) feed the same SpecIR path; typed result objects keep stages isolatable.

Module responsibilities. Adapters keep notation changes out of the core: SOFL, Mini-Z, and Mini-StateMachine each implement `SpecAdapter.parse() $\to $ SpecIR`, then lower via `FSFLowerer`. The pipeline runner realises Algorithm 1, including early exit on $\mathrm{Accept}(c_k, t) = 1$ and best-effort fallback when $k > K$. The formal checker builds Z3 queries, executes candidates in a sandbox, and assembles counterexamples. The pattern guard applies catalogue $\mathcal{P}$ (Table 4.5) with AST and regex detectors. The experiment orchestrator schedules mode–task pairs across a process pool with resume-safe logs.

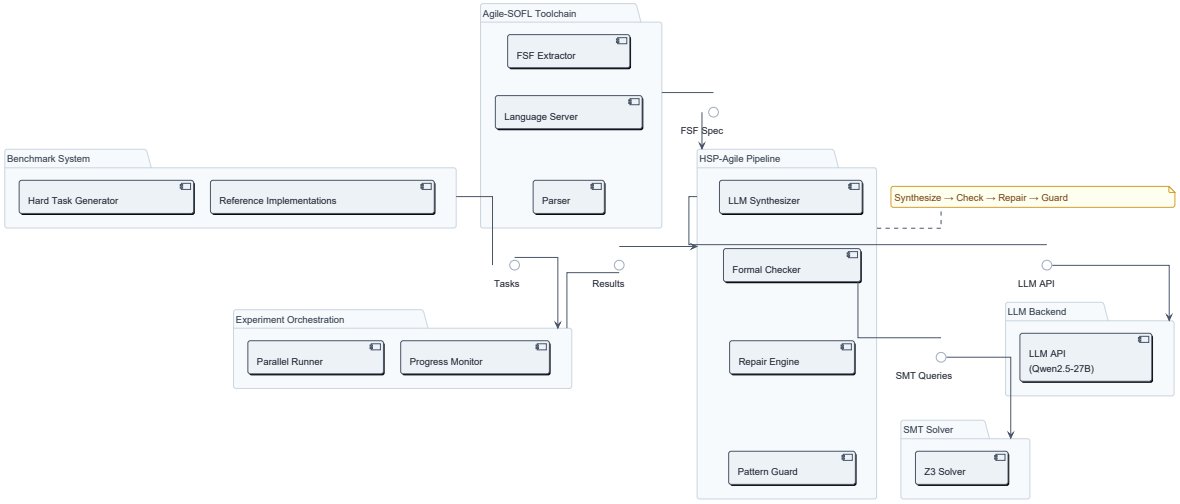

Fig. 5.1. Component architecture: Stages 1–4 as isolatable modules under one orchestrator. Ablations swap a stage without rewriting the rest (software view of Figure 1.2).

5.2 Specification Ingestion Layer

The checker never consumes raw SOFL, Mini-Z, or Mini-StateMachine text. Ingestion parses each surface notation into SpecIR (Chapter 3) and lowers it to the FSF-shaped `TaskSpec` consumed by Stages 2–4: notation-specific parsing via a registered `SpecAdapter`; canonicalisation into ordered `GuardedCase` records; and lowering via `FSFLowerer`. The primary SOFL path uses the Agile-SOFL toolchain[36,40]; E8 adapters bypass SOFL syntax while producing the same SpecIR structure. Figure 5.3 shows the SOFL-side bridge classes.

5.2.1 SpecIR Adapter Registry

The adapter registry (`src/adapters/__init__.py`) maps notation names to concrete `SpecAdapter` subclasses. Each adapter implements `parse(spec_text, task_id) $\to $ SpecIR`, populating `GuardedCase` objects with normalised `GuardAtom` predicates, postconditions, and surface-text fields for prompting. `SpecIR.to_dict()` / `from_dict()` round-trip through `schemas/spec_ir.schema.json` for audit logging and cross-notation interchange.

`FSFLowerer.lower(spec)` projects SpecIR onto the legacy `TaskSpec` schema (ordered scenarios, I/O signatures, prompt text, pattern-guard metadata). `PipelineRunner.run()` accepts either a `TaskSpec` dict or a `SpecIR` instance (`src/pipeline/task_input.py`), normalising through the lowerer when needed. Oracle construction, witness generation, and repair feedback therefore stay notation-agnostic while remaining compatible with the 120-task FSF benchmark.

Parsing sketch. The Agile-SOFL bridge extracts, per process with a non-empty FSF block, a `TaskSpec`: task id, callable signature, ordered `FSFScenario` list (guard `test`, assignment `def`, kind `scenario`/`others`), external-variable map, and rendered prompt text.

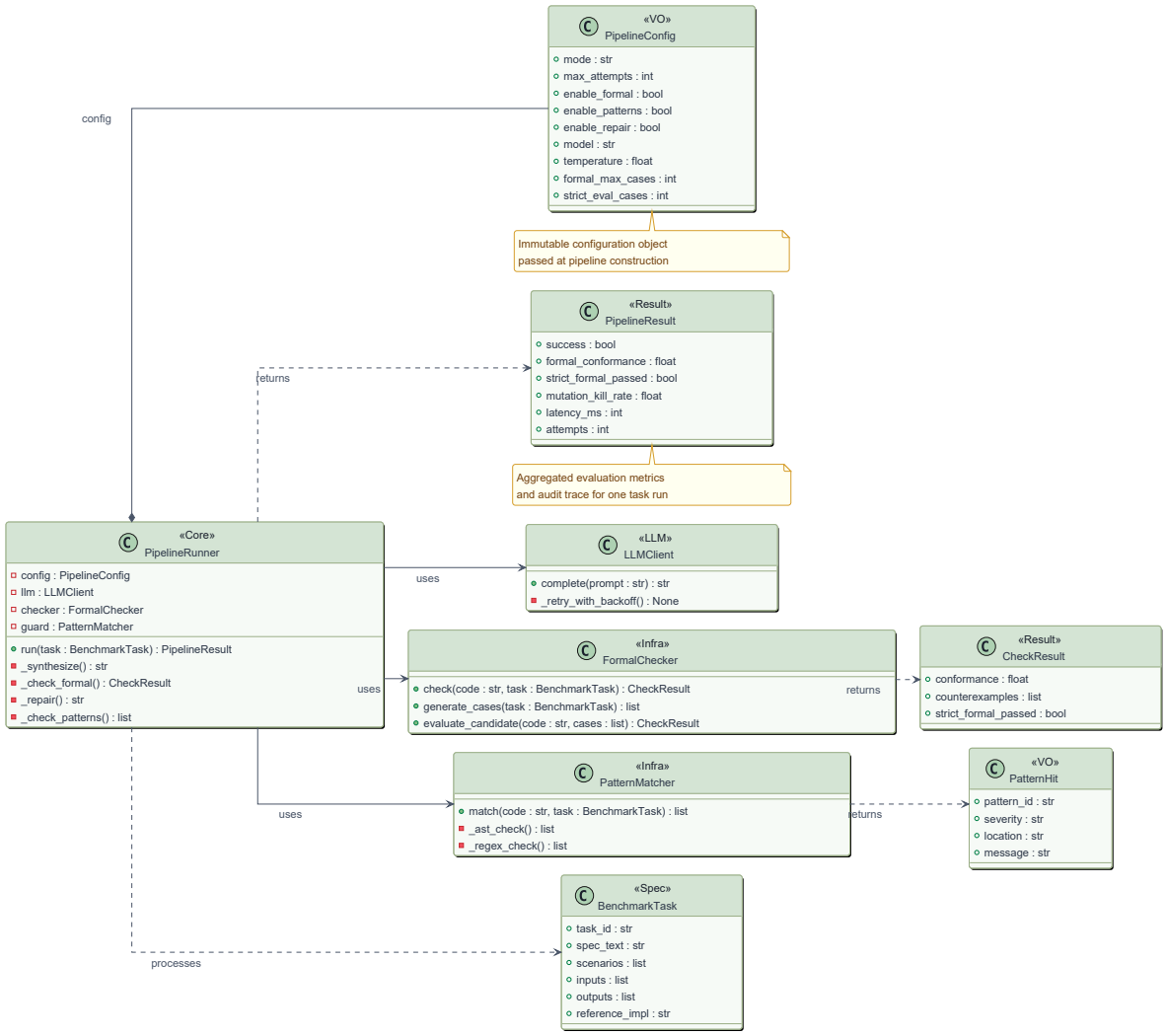

Fig. 5.2. Class diagram: one `run(task)` entry point composes Stages 1, 2, and 4 behind Algorithm 1. `PipelineRunner` is the hub; clients and checkers are leaves.

Scenario indices preserve first-match order $s_1 \succ \cdots \succ s_n$[19] (Definition 3.7). Guards are normalised to a shared relational-atom syntax consumed by the prompt renderer, Z3 translator, and pattern guard; output names define $\mathcal{Y}_t$ for field-wise oracle comparison and RF04 detection. External `ext rd`/`ext wr` names are listed in the prompt and checked by RF10 when relevant.

5.3 LLM Integration Layer

Synthesis is the only non-deterministic stage; everything else must be replayable. The LLM layer wraps an OpenAI-compatible gateway (`ECNUClient`, `src/llm/ecnu_client.py`) for auth, retry, timeout, token accounting, and audit logs—the name is historical, not a coupling. Production runs use Qwen2.5-27B-Instruct (Qwen-27B) with exponential backoff (up to three retries), a 90 s request timeout, and optional JSON audit of message history and token counts[11,52].

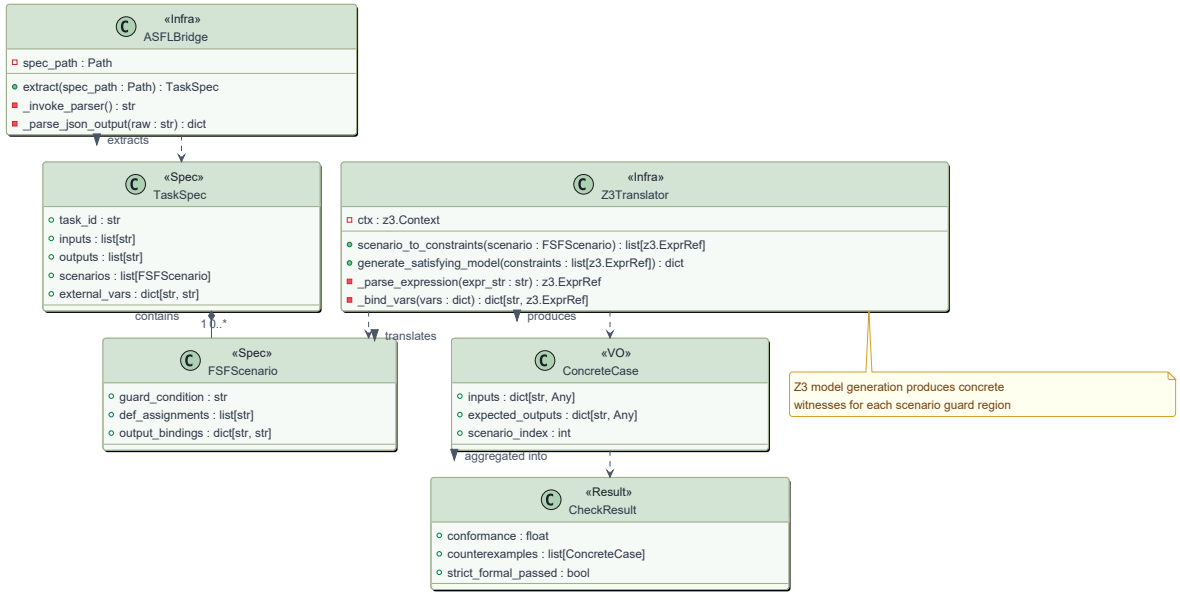

Fig. 5.3. Surface syntax stops at SpecIR; Stages 2–4 consume only the lowered `TaskSpec`. For the tier classifier, the bridge yields a three-scenario task whose catch-all residual is S_3 (`OTHERWISE`).

Prompt assembly follows Section 4.6.1: a fixed system message (role, fenced-code output, first-match guard order) plus a user message carrying the rendered FSF text, signature, required output keys, and—on repair iterations—verbalised feedback. Sampling uses T_{gen}=0.7 on the first attempt and T_{repair}=0.0 thereafter (top_p=0.95): explore once, then exploit under a deterministic checker. Responses are scanned for the last fenced Python block (or a conservative line heuristic); `compile()` failure yields a sentinel $\perp$ that scores zero and triggers repair without crashing the pipeline.

5.3.1 Semantic Feedback IR and Two-Layer Repair Prompting

When iteration k fails formal checking or pattern screening, the repair engine constructs a `SemanticFeedbackIR` (`src/repair/feedback_ir.py`) from checker counterexamples and the active `TaskSpec`. Each record serialises to JSON per `schemas/semantic_feedback.schema.json` *before* any natural-language rendering:

1. **IR construction** — nine typed fields (`violation_type`, `scenario_index`, `constraint_text`, `inputs`, `expected`, `observed`, `reason`, `priority`, `suggested_fix`) from witness and scenario metadata;
2. **JSON serialisation** — schema-compliant records logged in `PipelineResult.semantic_feedback`;
3. **Prompt projection** — `FeedbackRenderer.render()` maps the same IR to `test_only` (B2 / E6-A), `test_expected` (E6-B), or `semantic_ir` (M / E6-C).

Pattern violations append as a capped list of `pattern_id:name` pairs under a *"Previous attempt failed"* header. Ablation E6 varies only the renderer while holding IR construction

fixed, so Semantic Feedback is a genuine intermediate representation rather than a prompt template alone.

Example: catch-all failure. On the tier classifier (Listings 1.1, 1.2), a mid-band witness activates S_3 (`OTHERWISE`). The oracle requires `Review`; the candidate returns `Pass`. The checker emits a record of the following shape (abbreviated):

```
{
  "violation_type": "output",
  "scenario_index": 3,
  "constraint_text": "OTHERWISE",
  "inputs": {"score": 3, "floor": 10},
  "expected": {"tier": "Review"},
  "observed": {"tier": "Pass"},
  "reason": "catch-all residual returns wrong constant",
  "priority": 1,
  "suggested_fix": "return Review on OTHERWISE"
}
```

A `test_only` projection would state only that the case failed; `semantic_ir` names the residual scenario, expected constant, and repair hint—the implementation hinge behind the C2 mechanism in Chapter 4.

5.4 Formal Conformance Checker

Acceptance must not rest on lucky unit tests. The formal checker is the deterministic oracle that grounds release decisions in SMT-backed witnesses rather than heuristic sampling[13]. Given candidate c and task t, it computes $\mathrm{Conf}(c, t)$ and assembles *cexs* for repair (Figure 5.4; Algorithm 2 in Chapter 4).

Guards lower to Z3 integer constraints under the priority encoding (4.8): $\Phi_i \equiv \widehat{g}_i \wedge \bigwedge_{j<i} \neg\widehat{g}_j$, with `others` as the negation of preceding guards (Definition 3.8). Each Φ_i uses a fresh solver. Integer variables are bounded to $[-5, 20]$ for Z3 termination in the checker implementation; hard-task metadata describes inputs over $[0, 100]^5$ (Chapter 6), so reported witnesses use the implemented solver bounds rather than the full declared domain (Appendix A). Satisfying models become concrete witnesses (x, y, i); model blocking yields up to a few alternatives per scenario (Algorithm 3). On the tier classifier, Φ_3 forces a residual mid-band point that Reject/Pass unit tests never need to hit.

Candidates run in an isolated namespace; field-wise integer equality against the oracle yields $\mathrm{Conf}(c, t) = |\mathit{passed}|/|\mathcal{D}(t)|$ (4.10). Repair iterations use a 16-case budget ($N_{\max}$ for mode M); post-pipeline strict scoring uses 64 cases (`PipelineConfig`; sensitivity in Section D.11).

Algorithm 3 Z3-Based Concrete Case Generation (sketch)

Input: ordered scenarios $\mathcal{S}_t$, variables V, budget N
Output: case set $\mathcal{D}(t)$ with $|\mathcal{D}(t)| \leq N$

```
for i = 1 to n do
    build Φ_i per (4.8); solve under domain bounds
    while sat and budget remains do
        extract model x; set y ← φ_i(x); record (x, y, i); block model and re-query
    end while
end for
return D(t)
```

5.5 Counterexample-Guided Repair Engine

The repair engine adapts CEGIS[56] to an LLM synthesiser: it keeps the feedback buffer, increments the attempt counter, and exits on Accept or best-effort when $k > K$. Control states are in Figure 5.5; the feedback payload on each repair arc is Figure 4.6 (Chapter 4).

States are Init, Synthesizing, FormalChecking, PatternChecking, Repairing, and terminal Accepted/Exhausted. Formal failure or any high-severity pattern hit routes to repair; both Conf = 1 and a clear $\mathcal{P}^H$ screen yield acceptance. Budget K=3 (principle P2) caps LLM calls; mode A3 disables repair for a single-shot path. On exhaustion the pipeline returns the highest-Conf candidate as Best-Effort. Algorithm 4 summarises the control logic; the full SgDP loop is Algorithm 1.

5.6 Structural Screen Module

A candidate can match every witness in $\mathcal{D}(t)$ and still hard-code success, ignore ordered guards, or drop an output field that the finite case set never exercised. Stage 4 therefore implements the structural half of acceptance safety: an oracle Screen(c, t) independent of the witness suite that can veto release even when Conf=1 (Section 4.9; Figure 4.1).

The present artifact instantiates Screen with a catalogue of requirement-level code smells $\mathcal{P}$ (RF01–RF14; Table 4.5), in the spirit of FindBugs-style structural screening[4,26] and prior SOFL fault-prevention patterns[37]. Detectors are primarily AST predicates (control flow, return shape, missing fields); a minority use surface matching or guard–conditional cross-reference. The blocking high-severity subset is

$$\mathcal{P}^H = \{\text{RF01}, \text{RF02}, \text{RF04}, \text{RF05}, \text{RF06}, \text{RF07}, \text{RF09}, \text{RF11}, \text{RF13}\};$$

medium/low hits are logged into repair feedback but do not alone withhold Accept. AST detectors catch structural shortcuts (e.g. RF04 missing outputs, RF07 unconditional success, RF09 branch-count mismatch); cross-reference detectors catch guard-literal mismatches (RF01–RF02, RF05–RF06, RF11). Each δ_p is a pure predicate; hits carry id,

Algorithm 4 Counterexample-Guided Repair Loop (sketch)

Input: task t, budget K, flags (*enable_formal*, *enable_patterns*, *enable_repair*)
Output: c^*, status $\in \{\textsc{Accept}, \textsc{Best-Effort}\}$

```
feedback ← ∅;   best ← (⊥, 0)
for k = 1 to K do
    c_k ← LLM(t, feedback)
    evaluate Conf / cexs and hits under the enabled flags
    update best; if Accept(c_k, t) = 1 then return Accept
    if repair enabled then feedback ← BuildFeedback(cexs_k, hits_k, k)
end for
return best, Best-Effort
```

severity, location, and message for the Semantic Feedback IR renderer.

The module boundary is intentionally thin: any object that maps (c, t) to severity-tagged hits can replace $\{\delta_p\}$ without touching SpecIR, Z3 witnesses, or Equation (4.11). In a production Agile-SOFL pipeline the same slot can hold a project-specific smell pack, an existing CI static analyser, or an LLM-driven review agent that returns structured vetoes into the same repair channel. RF01–RF14 remain the evaluated default so C3 (PDR/FAR) is measured under a deterministic screen.

The structural screen is the second conjunct of $\text{Accept}(c, t)$:

$$\text{Accept}(c,t) \;=\; \mathbf{1}[\text{Conf}(c,t)=1] \;\wedge\; \mathbf{1}[\text{Screen}(c,t)=\textit{pass}]\,.$$

With the PoC encoding, $\text{Screen}(c, t) = \textit{pass}$ iff no $p \in \mathcal{P}^H$ fires. The screen runs *after* formal checking on each iteration. A candidate that passes all Z3 cases but triggers RF07 (unconditional success) is routed to repair, with the smell verbalised alongside any counterexamples. Witnesses catch catch-all and ordering residuals; the screen catches shortcuts that never attempt the required structure (Section 4.3.5).

5.7 Experiment Orchestration

The orchestrator prioritises parallel throughput, crash-safe resumption, and live progress[8,24]. Two matrices are used: the *core* seven-mode matrix is $7 \times 120 = 840$ runs (B0–B2, M, A1–A3 under `run_hard_full_parallel_v1`); the *extended* matrix is $10 \times 120 = 1{,}200$ when B3–B5 are included (`run_ccf_b_extended_v1`; B6 is reported separately where evaluated). Jobs run in a *process pool* (one worker per core) because Z3's native global state is not thread-safe; each worker owns its own client, config, and pipeline, then appends a strict 64-case evaluation (and optional mutation score[18,37]) to a shared JSONL log. Incremental persistence plus a `progress.json` sidecar make mid-batch restarts resume-safe without re-running finished keys.

Table 5.1 lists the seven core modes; flags on `PipelineConfig` enable ablations A1–A3 and

baselines B0–B2 against full mode M (Chapter 6).

Table 5.1. Execution mode configuration matrix. Flags control which pipeline stages are active; numeric parameters tune per-stage behaviour. All LLM-based modes use Qwen2.5-27B-Instruct (Qwen-27B) with $K = 3$ unless noted. $N_{\max}$ is the per-scenario formal case cap.

Mode	Description	Formal	Guard	Repair	$N_{\max}$
B0	Reference template	*eval only*	No	No	—
B1	One-shot LLM	No	No	No	—
B2	Test-feedback loop	No	No	Yes	8
M	Full HSP-Agile	Yes	Yes	Yes	16
A1	No formal check	No	Yes	Yes	10
A2	No pattern guard	Yes	No	Yes	24
A3	No repair loop	Yes	Yes	No	24

All modes apply strict evaluation with a 64-case budget after pipeline completion. B0 uses a deterministic template renderer; no LLM calls are made.

5.8 Design Decisions and Trade-offs

Three decisions dominate the rest[62].

5.8.1 Z3 Over Random Testing

Z3[16] is preferred to random or property-based testing[1,13] because (i) guard regions under conjunctions of inequalities are rarely hit by uniform sampling over $\mathbb{Z}^5$; (ii) priority encoding Φ_i yields exclusive first-match witnesses, including catch-all residuals that busy-band samples miss; and (iii) failures arrive as scenario-indexed concrete counterexamples usable by CEGIS repair[57]. The cost is solver latency (∼50–200 ms per query; ∼2 s per 16-case repair iteration; ∼6–8 s for 64-case strict scoring).

5.8.2 Conjunctive Acceptance Over Single-Metric Gating

A disjunctive gate (accept on Conf $= 1$ *or* a clear structural screen) would maximise throughput but conflate distinct fault classes. Formal conformance certifies pointwise witness correctness; Screen certifies that the candidate is not an obvious structural shortcut (Section 5.6). Neither implies the other: hard-coded outputs can pass sparse witnesses while failing RF07, and a structurally clean candidate can still fail on a catch-all or overlap residual. The conjunctive Accept of (4.11) is therefore conservative—fewer accepts, but each satisfies both criteria. Which concrete Screen fills the slot does not change that design.

5.8.3 Latency vs. Quality Trade-off

The 16/64 case split and K=3 bound per-task cost for sprint-cycle use (P2). Sixteen internal cases give at least two witnesses per scenario on eight-scenario tasks; sixty-four strict cases probe residual and overlap regions more densely. Sensitivity analysis (Chapter 6) shows diminishing conformance returns beyond 32 internal cases; most repairable faults resolve within two iterations, so larger K rarely pays off.

Chapter Summary

The realisation maps cleanly onto the Tier-1 claims:

- **C1** — SpecIR adapter registry and FSF lowering keep surface syntax out of the checker.
- **C2** — Semantic Feedback IR and the repair state machine own typed failure state across iterations (catch-all residual as primary example).
- **C3** — Z3 conformance, pluggable Screen (default: $\mathcal{P}^H$ smells), and conjunctive Accept decide release.

Orchestration and sampling parameters are supporting engineering. Whether those choices pay off is empirical—Chapter 6.

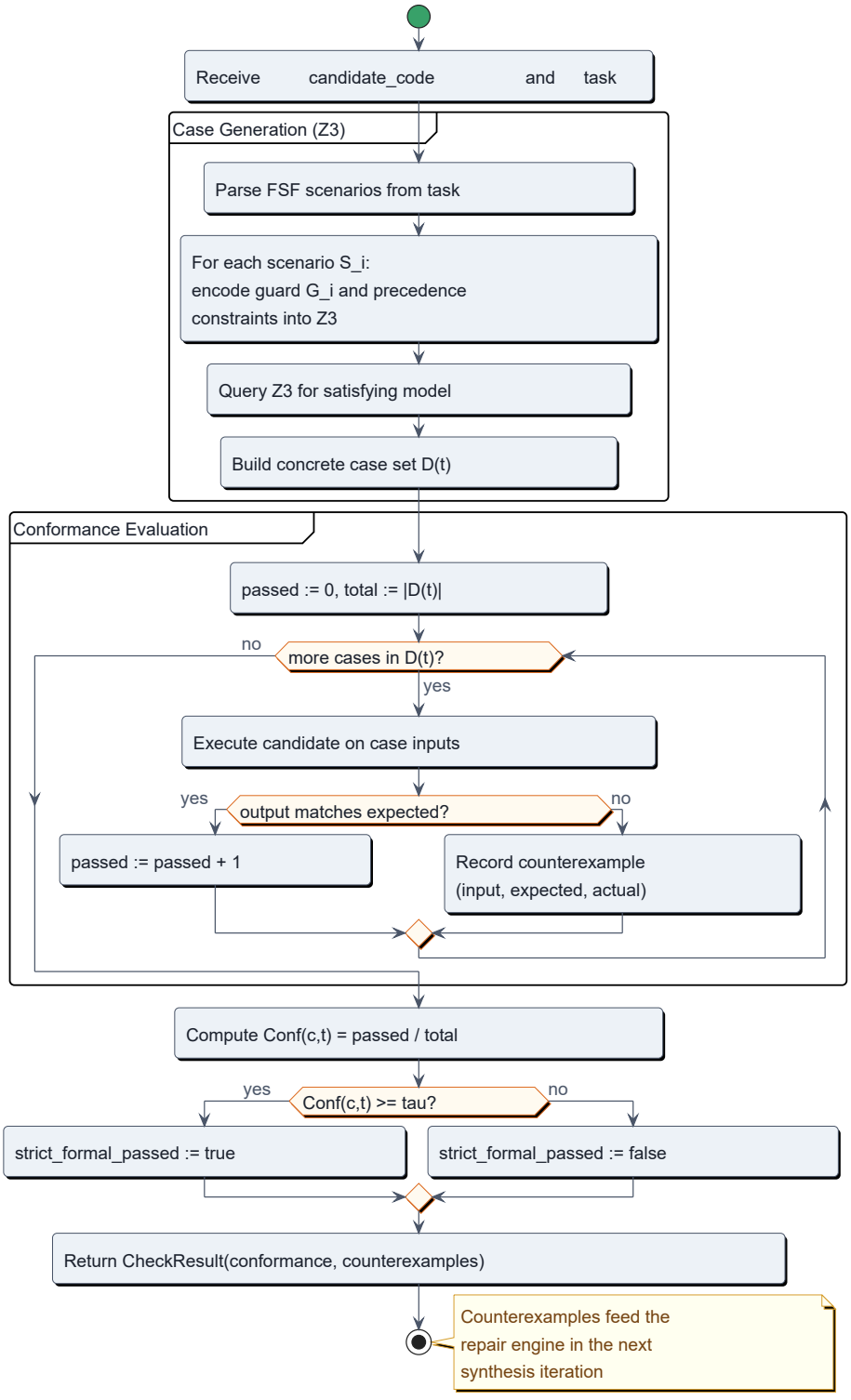

Fig. 5.4. Checker activity: case generation and evaluation are separate partitions; counterexamples feed Stage 3 repair.

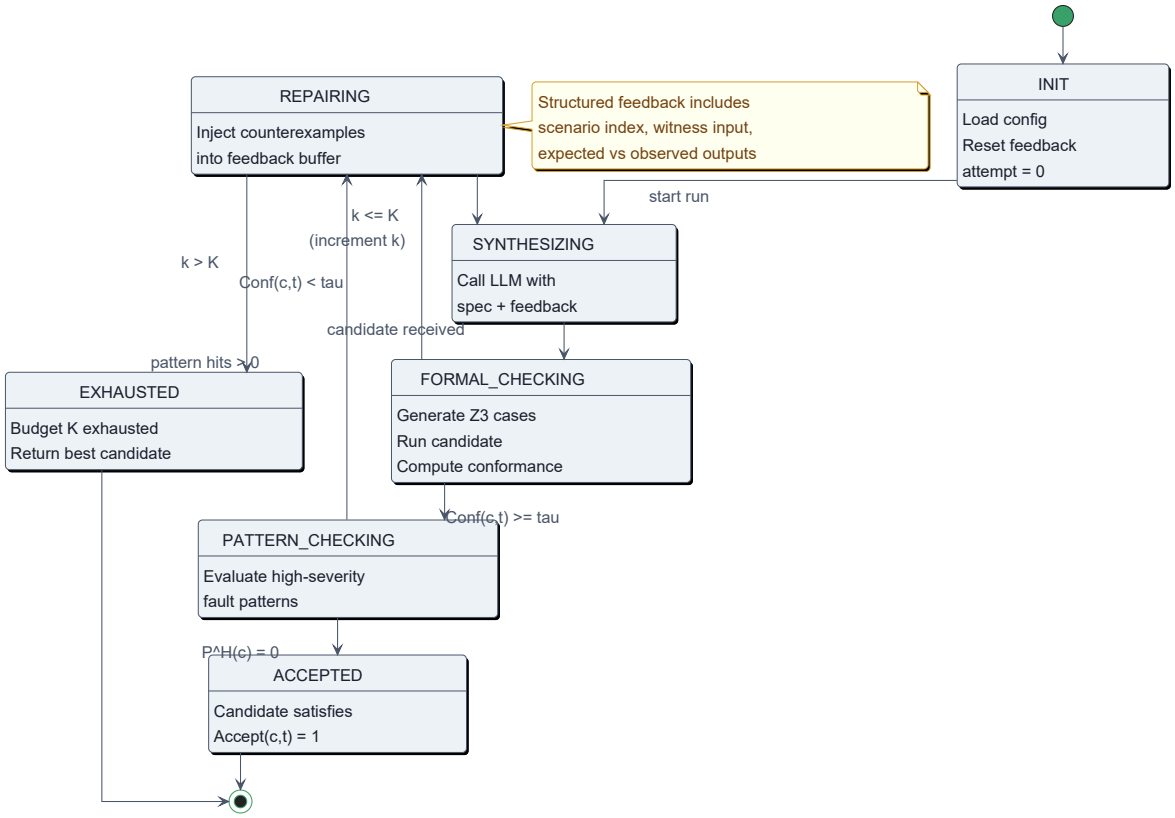

Fig. 5.5. Pipeline state machine: repair aggregates counterexamples and pattern hits into Semantic Feedback IR before re-synthesis. Acceptance requires formal conformance and a clear high-severity pattern screen.

6 Experimental Setup

This chapter fixes the measurement protocol for the Tier-1 claims: what is compared, how prevention is scored, and which run ids are authoritative. Hardware, modes, metrics, and experiment tracks are stated once; threats are previewed briefly and analysed in Chapter 8.

Table 6.3. Claim-to-experiment map (Tier-1 focus).

Claim	Experiments	What would falsify it
Premise (SpecIR hinge)	E3, E4, E8	Adapters fail to transfer; SMT coverage equals random at equal budget
C2 (structured feedback)	E5, E6, E14, combo-seed	Full IR $\not\succ$ test-only under headroom; E14 uniqueness over-claimed
C3 (conjunctive gate)	E1 Strict, E2 PDR/FAR, A2	High Conf. with high FAR; gate adds nothing on prevention
C4 (deploy B2 default)	E3, E10, E12, pilots	Overlap tertiles alone dictate M; pilots contradict B2 default

6.1 Research Questions

Numbering is mechanism-first (same scheme as Chapters 1 and 7): typed feedback and overlap structure, then prevention and deployment, with aggregate E1 ranking as an RQ5 stress-test rather than a universal-effectiveness headline.

RQ1 (Specification-indexed Feedback IR). Does Semantic Feedback IR improve repair beyond unstructured test-only feedback? (E6, E14 scope, hard combo-seed probe; primarily C2.)

RQ2 (Gap vs. Guard-Overlap Density; negative). Does the M–baseline gap grow monotonically with guard-overlap density? (E3, E10; bounds C4—not a positive SpecIR ranking claim.)

RQ3 (Conjunctive Accept vs. Conf Alone). Does conjunctive Accept improve prevention versus mean conformance alone? (E2, A2 vs. M; primarily C3.)

Table 6.1. Authoritative numbers card (cite these; do not mix corpora). Tier-1 spine: E6 → E2 → C4; SpecIR is an engineering hinge.

Claim / corpus	Numbers
E6 lead (C2), K=3	A/B/C Conf. 79.1 / 78.9 / 86.9%; C−A +7.7 pp; paired 14/4/102; 95% CI [2.3, 13.6]; Wilcoxon p=0.018; qualitative: **14/14** catch-all `others`, **0/14** ordering
E14 scope (not uniqueness)	`semantic_ir` 75.1% ≠ `execution_trace_matched` 85.4% (paired 5/19/96); prefer IR for localisation, trace for Conf.; default M may use trace; E6 isolates `semantic_ir`
Field probe (supporting)	gemini combo pooled n=80×3: FULL vs. test_only +26.5 pp (CI excl. 0); vs. ir_no_expected CI includes 0
E2 prevention (C3)	M PDR 95.0% / FAR 5.0% vs. B2 91.2% / 8.8% (impl-screening; bootstrap CI excludes 0)
Equal-K Conf. (`M_eq`)	`run_e1_equal_k_v1`: B2 97.5% / `M_eq` 100.0% (+2.5 pp; 3/0/117); secondary to E6
Fixed-oracle stress (bundle K=5)	E1: B1 84.2 / B2 98.3 / M 100.0 (+1.7 pp; 2/120 wins); E10 +1.0 pp; E12 B2 100.0 / M 99.7 (B2 first every seed)
Industrial / GitHub C4	GitHub harvest n=48: B2/`M_eq` 89.9% vs. B1 85.8%; HKCA09 FSF n=35: B1 74.3% / B2=`M_eq` 100%; published desensitized n=28 saturates at 100%; not live production SOFL; ranking ties ≠ C2 null

RQ4 (Deployment Boundary). Under what conditions should practitioners deploy M versus B2? (E3, E10, E11, E8c, pattern/pilot slices; C4 / Table 8.1.)

RQ5 (Quality–Overhead Trade-off). Is the latency cost justified by prevention and release requirements? (E1 Pareto, A2 operating point.) Primary fixed-oracle E1 ranking (B1/B2/M from `run_e1_m_win_v2`) is an overlap stress-test with paired win rates, not a powered dominance claim.

6.2 Hardware and Software Environment

All runs used a Windows 10 (build 26220) desktop, 6-core CPU, Python 3.12, and one `multiprocessing` worker per core. Z3 4.16[7,16] via `z3-solver` with a 10-second per-call timeout; `unknown` is non-constraining, not a hard failure.

6.3 Language Model Configuration and Run Protocol

6.3.1 Language Model Configuration

LLM-based modes (B1, B2, M, A1–A3) use Qwen2.5-27B-Instruct[65] (Qwen-27B) through an OpenAI-compatible gateway. Temperatures: T_{gen}=0.7, T_{repair}=0.0; token cap 4096 per call (Chapter 4). Repair budget K is not uniform: primary fixed-oracle ranking (E1/E10/E12 M-win) uses strengthened M with K=5 (best-effort argmax Conf.); B1/B2 keep baseline budgets. Mechanism and controlled comparisons (notably E6; historical pre-fix matrix) use K=3.

6.3.2 Corpora and Measurement Correction

Three corpora must stay distinct (Table 6.5; `artifacts/AUTHORITATIVE_NUMBERS.md`). *Once for the chapter:* a bug in others-witness generation (others encoded as *True* without ¬prior guards) previously inflated false failures (∼5% Strict on canonical E1). Fixed first-match others witnesses enable fair evaluation; primary B1/B2/M aggregates cite the fixed-oracle M-win runs. Pre-fix runs (`run_e1_canonical_v1/` `run_hard_full_parallel_v1` and related) are archive only—never primary ranking evidence. Fixed-oracle A1–A3 live in `run_e1_ablation_fixed_v1` (Table 7.16).

Table 6.5. Corpus / protocol card (which experiment cites which run).

Role	Run id(s)	Used for
Primary ranking (fixed others-witness oracle)	`run_e1_m_win_v2`, `run_e10_m_win_v1`, `run_e12_m_win_v1`, `run_e1_ablation_fixed_v1`, **`run_e1_equal_k_v1`**	E1/E10/E12 B1/B2/M Conf stress (M K=5 bundle); **equal-K Conf: B2 vs. M_eq (K=3, +2.5 pp)**; fixed-oracle A1–A3
Mechanism / prevention	`run_feedback_v2`, `prevention_full_v1`	E6 typed-IR (K=3); E2 PDR/FAR
Historical pre-fix (superseded for ranking)	`run_e1_canonical_v1/` `run_hard_full_parallel_v1`; also `run_e12_canonical_v1`, `run_e10_random_v2`	Pre-fix A1–A3 Conf. ordering (archive), some latency figures, pre-fix Holm multi-mode stats; K=3; buggy others-witness oracle (∼5% Strict)

Primary scales: E1 120×3 modes; E10 100; E12 120×3 seeds×3 modes. Extended baselines B3–B5 use `run_ccf_b_extended_v1` (repeat 0). Artefacts (prompts, responses, Z3 logs, traces, timing) are JSONL; task/prompt RNG seed 42; repair calls at temperature 0.0 are near-deterministic. Full reproduction: Appendix A.

6.4 Benchmark Construction

Ordered FSF artefacts are rare in public repos, so the corpus is constructed to be discriminative (baselines fail non-trivially), formally valid (Z3-satisfiable), and diverse in orderings and boundaries. DafnyBench[43] targets annotation-heavy verification, not FSF precedence.

6.4.1 Hard Synthetic Benchmark (E1)

Twenty base SOFL/FSF tasks from published examples[36,40] form a development set and are excluded from primary evaluation. The *HardTaskGenerator* emits tasks with 5 integer inputs $(a,b,c,d,e) \in [0,100]^5$, 3 outputs, and 8 ordered scenarios with precedence-sensitive overlapping guards (targeting SOV from Chapter 3). Z3 admits only individually satisfiable scenarios; 180 candidates yield 120 accepted tasks (60 discarded). Figure 6.1 shows

the path; Table 6.6 summarises the set (mean guard-overlap rate 1.8 scenarios per input point). Under fixed-oracle E1 (`run_e1_m_win_v2`), B1 mean Conf. is 84.2%; discriminative signal sits in per-task variance (Figure D.3) and prevention metrics, not aggregate mean Conf. alone.

Table 6.6. Summary statistics of the 120-task hard benchmark.

Property	Value
Total tasks	120
Inputs per task	5
Outputs per task	3
Scenarios per task	8
Mean guard-overlap rate (scenarios/point)	1.8
Tasks rejected by Z3 filter	60
Tasks accepted	120
Conformance cases per task (`strict_eval`)	64

Illustrative skeleton. The hard tasks scale the first-match idea of `compute_tax` (Chapter 1): nested guards, most-specific first, defects hiding in overlap regions. A reverse-order evaluation yields the wrong branch on overlapping inputs—a scenario-order violation detectable by the pattern guard and by Z3 witnesses (e.g. `a=85, b=15, c=10, d=80, e=20`).

6.4.2 External and Auxiliary Corpora

Table 6.8. Auxiliary benchmarks (external validity and controls).

Slice	n / run	Role
E10 random	100; `run_e10_m_win_v1`	Same generator *without* Z3 overlap filter; negative control for filter bias
E11 external SOFL	22; `run_e11_external_v1`	Hand-authored from published Agile-SOFL / IET patterns[36,37,40]; K=3; IDs disjoint from E1
Real-priority micro	30; `run_real_priority_micro_v1`	Non-synthetic ordered first-match (industrial, GitHub `.asfl`, HKCA09, desensitized pilots, adapted ACL/tax); equal-K B1/B2/`M_eq`; Table D.11
E8 real-derived	40 (20 HumanEval[11]+20 MBPP[3])	Ordered-branch problems converted to FSF; E8c full B1/B2/M
SMT domain ablation	30 real-priority tasks	Box sizes $[-5, 20]$–$[-1000, 1000]$ plus Real encoding; Table D.12, Figure D.11

E10 asks whether aggregate M>B2 holds without overlap filtering, or whether tertiles (E3) and deployment signals (Table 8.1) must be read jointly. Real-priority and SMT protocols are detailed in Sections D.9.1–D.9.2; selection protocol for E8 is in the artifact package.

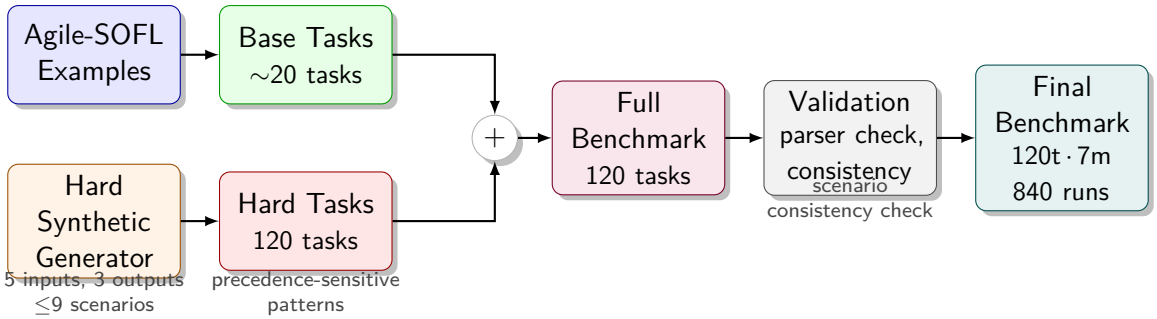

Fig. 6.1. Key observation: the Z3 filter keeps only overlap-heavy, satisfiable tasks—the regime where ordering defects actually occur. **How to read:** left-to-right from sampled guards through filtering to the final 120-task set.

6.5 Compared Methods

The design space spans formal reference (B0), LLM baselines (B1–B5), verifier-in-the-loop (B6), full HSP-Agile (M), and ablations (A1–A3). Primary fixed-oracle ranking reports B1/B2/M from `run_e1_m_win_v2`; historical seven-mode and B3–B5 runs supply extended baselines (Table 6.5).

Table 6.9. Compared modes. Columns indicate whether each component is active. B3–B5 are evaluated in E1-ext (`run_ccf_b_extended_v1`, repeat 0 merged into Table D.1).

Mode	Description	Formal checker	Pattern guard	Repair loop
B0	Reference (template)	eval only	—	—
B1	One-shot LLM	—	—	—
B2	Test-feedback	—	—	✓(tests)
B3	Self-Refine[47]	—	—	✓(self-critique)
B4	Self-Debug[12]	—	—	✓(execution trace)
B5	Reflexion-lite[54]	—	—	✓(verbal memory)
B6	VerifierLoop-FSF (REV-7)	✓	—	✓(witness cex)
M	Full SGDP/HSP-Agile	✓	✓	✓(IR)
A1	Ablation: no formal checker	—	✓	✓(IR)
A2	Ablation: no pattern guard	✓	—	✓(IR)
A3	Ablation: no repair loop	✓	✓	—

Baselines. B0 is a deterministic template renderer (formal upper bound; Conf. computed ex post). B1 is one-shot LLM with no loop. B2 adds iterative FSF-derived unit-test feedback[69] at K=3 (even when M uses K=5 on M-win harnesses); no Z3, no pattern guard. B3–B5 isolate unstructured self-improvement (Self-Refine[47], Self-Debug[12], Reflexion-lite[54]) under equal K=3 and the same token cap; results in Table D.1. B6[57] exposes structured SMT counterexamples in the repair prompt without Semantic Feedback IR fields or the pattern guard (`run_b6_full_v1`, n=100).

M and ablations. M runs the full pipeline (Chapters 4–5): Z3 witnesses, pattern screen, and CEGIS repair with Semantic Feedback IR. Primary M-win uses K=5, `execution_trace_matched`, and an advisory critical-only pattern gate; E6 keeps K=3 under full M to isolate feedback content. A1 replaces Z3 cases with B2 tests; A2 disables the pattern guard; A3 applies checker and guard once without repair feedback.

6.6 Mutant Design

Prevention metrics need faults that look locally plausible yet violate global precedence. *Specification-confusion* mutants perturb the task (SNO order swap/negate, ORO output reverse, MCO drop condition, BCO boundary offset). *Implementation-screening* mutants perturb the candidate (ICO flip condition, WRO reorder returns, MBO drop branch, DRO reverse dependency). Each applies one first-order operator to a known-correct artefact; non-compilable mutants are excluded. These operators model recurring LLM guard-generation failures: lost preconditions, reversed comparisons, missing branches, boundary shifts, and incorrect first-match order. They are defined over guard semantics and post-conditions rather than copied from the RF01–RF14 detector catalogue. Consequently, a mutant can evade the configured static screen yet still fail a SpecIR witness, or trigger the screen without being tied to one hand-written pattern. Per-operator PDR/FAR reports this distinction instead of treating all injected faults as one detector-friendly pool.

Miniature example (PDR/FAR counting). An SNO mutant swaps S_1/S_2 in the oracle; Accept rejecting a candidate that still implements the original order is a true prevention (PDR numerator). A WRO mutant reorders `if` branches in the candidate; again Accept should reject. False acceptance (FAR) occurs only if a faulty mutant still passes every witness and the pattern screen—bounded by Theorem 4.2.

6.7 Evaluation Metrics

Conformance and strict success. $Conf(c,t) \in [0,1]$ (Definition 3.10) is the fraction of strict evaluation cases satisfied; mean $\overline{Conf}$ is reported over 120 tasks. A *strict success* requires all 64 `strict_eval` cases; modes with the pattern screen additionally require a clear high-severity screen for Accept=1 (Equation (4.11)).

Prevention and cost. PDR / FAR (Definitions 3.23–3.24) are fractions of *faulty* mutants correctly blocked / falsely accepted; on a fixed faulty set, PDR+FAR = 1. Mutation kill rate μ_k measures checker precision on syntactic mutants. LLM call count (1.0 for B1; up to K for repair modes) and wall-clock latency include LLM, Z3, pattern guard, and

orchestration.

6.8 Evaluation Protocol

Each experiment answers one decision question; artifact paths live in Appendix A.

Table 6.11. Experiment map (E1–E17).

Exp	RQ	Decision question / protocol
E1	RQ5	Aggregate stress-test: Conf/PDR/FAR/latency; `run_e1_m_win_v2` (M K=5); win rates, no Holm claim
E2	RQ3	PDR/FAR per operator; 120×6 modes×8 ops; `prevention_full_v1`
E3	RQ2	Conf by overlap tertile / guard-complexity
E4	—	SMT vs. random coverage across budgets $\{4\ldots64\}$
E5	RQ1	Conf(k) dynamics from `attempt_history`
E6	RQ1	Feedback variants A/B/C under M, K=3; `run_feedback_v2`
E7	—	Pattern-guard P/R/F1 on E2 mutants
E8	—	SpecIR adapters: SOFL, Mini-Z, Mini-StateMachine; E8b n=30; E8c HumanEval/MBPP n=40
E9	—	Failure taxonomy of non-accepted M tasks
E10	RQ2/4	Unfiltered random n=100; `run_e10_m_win_v1`
E12	RQ4	Seeds $\{0,1,2\}$×120×B1/B2/M; `run_e12_m_win_v1` (1080 jobs)
E14	RQ1	Length-matched feedback sweep; `run_e14_sweep_v1`
E16	RQ4	Second endpoint (`ecnu-max`); `run_e16_canonical_v1`
E17	RQ3	Advisory gate `M_adv` vs. A2 vs. M; `run_e17_advisory_v1`

6.9 Statistical Protocol

Paired Wilcoxon signed-rank for conformance; Mann–Whitney U for latency; Cliff's δ for effect size; bootstrap 95% CIs (n=1000); α=0.05. Effect-size bands: $|\delta|$<0.147 negligible; <0.33 small; <0.474 medium; else large. Holm–Bonferroni applies to *pre-fix* multi-mode E1 comparisons on `run_e1_canonical_v1` only; fixed-oracle primary E1 reports means and win rates without a Holm claim.

On the historical pre-fix corpus, observed M–B1/B2 Conf. gaps are negligible and underpowered (post-hoc power ≈3.6% for $|\delta|$=0.014; `power_analysis.json`). Fixed-oracle E1 therefore emphasises per-task win rates; multi-seed stability uses `run_e12_m_win_v1`, with E3/E10 stratification and E2 prevention as complementary evidence.

6.9.1 Multi-Seed Protocol (E12)

E12-full re-runs 120 tasks × seeds $\{0, 1, 2\}$ × B1/B2/M under the M-win protocol (`run_e12_m_win_v1`; M K=5). Outcomes: per-seed mean Conf. ranking and pooled paired win rate M vs. B2. B2 reaches 100% Conf. on every seed; M averages 99.7%—M does not lead every seed.

6.10 Threats to Validity (Preview)

Construct threats include finite Z3 witnesses (FAR bound in Theorem 4.2), incomplete pattern coverage, and first-order mutants. Internally, LLM stochasticity is mitigated by T_{repair}=0.0, fixed seeds, bootstrap CIs, and E12; author-built benchmarks and the Z3 overlap filter favour SMT-aided modes, addressed by E10, E8, and ablations. Externally, primary evidence is synthetic numeric-domain Python with one LLM family. Conclusion validity separates pre-fix Holm results from fixed-oracle win rates. Full analysis: Chapter 8.

With the protocol fixed, Chapter 7 reports mechanism first (RQ1–RQ4), then quality–overhead and the E1 stress-test under RQ5.

Chapter Summary

Five RQs and experiments E1–E17 test the Tier-1 claims on a 120-task hard benchmark with ten compared modes. Primary B1/B2/M ranking cites fixed-oracle M-win run ids (Table 6.5); E6 and E2 use dedicated mechanism/prevention corpora. Metrics are Conf., strict success, PDR/FAR, and cost; statistics use non-parametric tests with Holm only on the pre-fix multi-mode archive. Threats are previewed here and fully analysed in Chapter 8.

7 Experimental Results

This chapter tests one hypothesis: *structured semantic feedback plus a conjunctive release gate reduces escaped defects on thin specification regions that mean pass rate hides.* The evidence ladder is fixed (Table 1.1): mechanism under repair headroom (E6), prevention under the conjunctive gate (E2), equal-K Conf. ranking, deployment guidance (C4), and one public ACL interception case. Near-ceiling ranking corpora (E1/E10/E12) and historical failure archives (E9) appear only as saturation context in Section 7.6 and Appendix D.

After the first-match others-witness measurement correction, fixed-oracle aggregate Conf. sits near the ceiling (M and B2 both $\geq$98% on the hard set). Those figures are comparable Conf., not the safety claim; the claim is Accept $\Leftrightarrow$ Conf=1 $\wedge$ Screen=*pass*.

Table 7.2. Results roadmap (main text).

Role	Section	Key figure / table
Mechanism (RQ1)	7.1	Fig. 7.1; Tables 7.8, 7.14
Prevention (RQ3)	7.2	Table 7.18; Figs. 7.4, 7.5
Equal-K / deploy (RQ4)	7.3	Tables 7.20, 8.1; ACL 7.4
Overlap bound (RQ2)	7.5	Fig. 7.7
Ranking archive	7.6, App. D	Near-ceiling E1/E8/E10–E12; E9

7.1 RQ1: Typed Semantic Feedback IR

Does naming the failed specification region improve repair beyond unstructured test dumps? The primary answer is E6 under full mode M at matched K=3. Supporting runs (E5 budget shape, E14 structured-family scope, hard combo seeds, fixed-oracle component ablation, E17 advisory gate) refine that answer; they are not co-equal headlines.

7.1.1 Catch-All Rescue under Typed Feedback (E6)

Return to the tier classifier of Chapter 1: Reject and Pass bands are busy; the mid-band `OTHERWISE` must return `Review`; an LLM candidate collapses it to `Pass`. Busy unit tests stay green. When a rare residual witness finally fails, an unstructured dump says only that "a test failed." Typed Semantic Feedback IR names the residual `others` scenario, the guard, and the output mismatch—and that naming is what E6 isolates. (The intro vignette uses three scenarios with residual S_3; BENCH-120 hard tasks scale the same first-match

skeleton to eight scenarios, so the residual is scenario index 8 / S_8 in the coded wins below.)

Figure 7.1 holds mode M fixed and varies only feedback content (n=120 tasks per variant; Table 7.10; `run_feedback_v2`). Variant C (full Semantic Feedback IR) reaches mean Conf = 86.9%—a gain of +7.7 pp over test-only feedback (A, 79.1%) and +8.0 pp over test-plus-expected (B, 78.9%). Paired per-task statistics (Table 7.3): C vs. A wins/losses/ties 14/4/102; bootstrap 95% CI on mean ΔConf. [2.3, 13.6] pp; Wilcoxon signed-rank p=0.018 (nonzero deltas). C vs. B: same W/L/T, +8.0 pp, CI [2.2, 14.0], p=0.027. Both intervals exclude zero. A and B are statistically indistinguishable on aggregate means ($\Delta < 0.3$ pp): adding the expected string alone is not a substitute for naming the violated scenario.

Qualitative coding (all 14 wins). Table 7.8 codes every C−A win. All 14/14 are catch-all `others` rescues (scenario index 8 on the eight-scenario hard skeleton—same residual *role* as S_3 in the three-scenario intro vignette; `violation_type=arithmetic` / output-expression mismatch). None (0/14) are ordering-precedence repairs. Of the 14 wins, 13 have `test_only` Conf=0—unstructured dumps collapse, and naming the residual scenario restores repair. The four losses are a mixed `output/arithmetic` subset (scenarios 6 and 8): typed IR does not dominate every task. The 14 tasks span distinct HardCase ids under the shared eight-scenario skeleton; the repeated signal is the *residual region*, not a single hand-tuned prompt. Ordering / preemption defects still motivate Φ_i witnesses and E2 mutants (Table 7.6); they are simply not what E6's Conf. delta measures on this corpus.

Table 7.10. E6 feedback variant summary, $n = 120$ per variant (primary model `ecnu-plus`).

Variant	Feedback surface	Mean Conf	Mean iterations
A	test only	79.1%	2.98
B	test + expected	78.9%	2.97
C	full Semantic IR	86.9%	2.99

Cross-model scope (`ecnu-max`). Table 7.11 repeats the same A/B/C protocol on `ecnu-max` (DeepSeek-family endpoint; `run_e6_ecnu_max_v1`, n=120, 0 errors). Mean Conf. collapses to ≈89.2% for all three variants (C−A =+0.03 pp; C beats A on only 1/120 tasks). The +7.7 pp typed-IR gain is therefore primary-model evidence under headroom: a stronger endpoint already saturates aggregate Conf. on BENCH-120, so feedback content no longer moves the mean. Prefer typed IR when the base model still leaves repair headroom; prefer B2 when the endpoint already saturates mean Conf. (consistent with E16 ties).

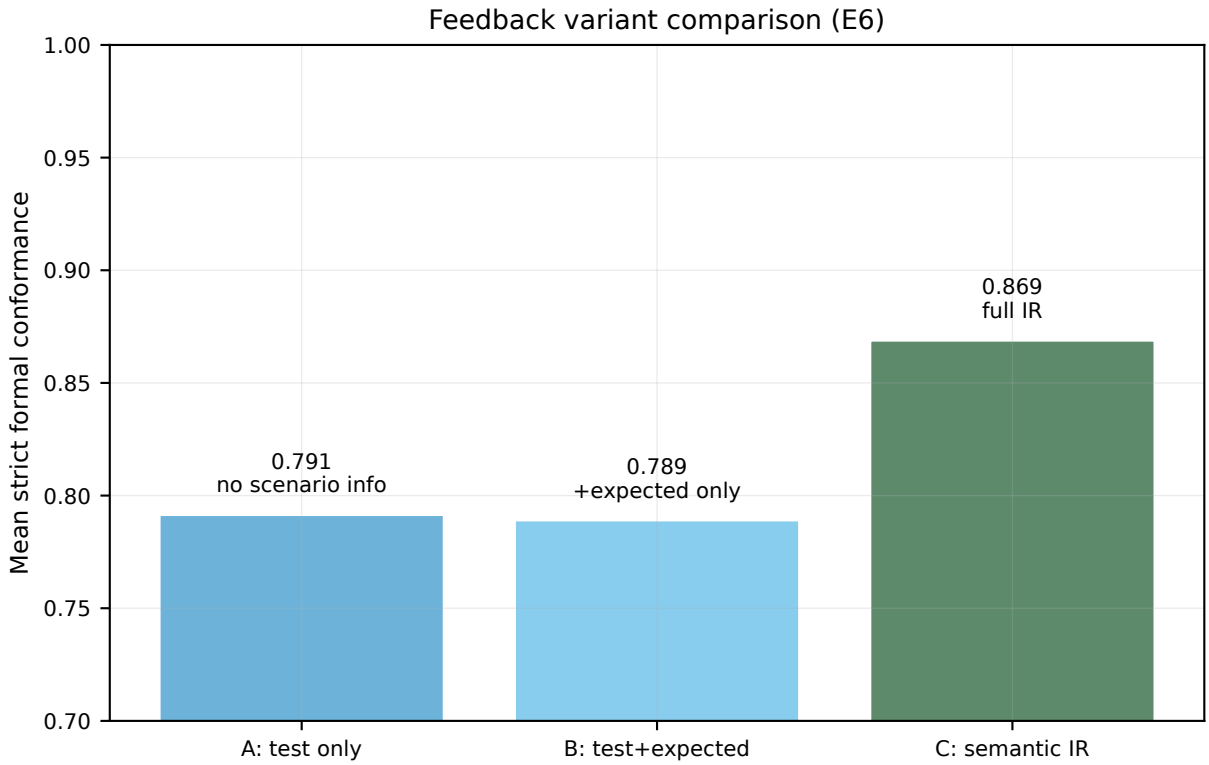

Fig. 7.1. Claim: specification-level structure, not prompt volume, drives repair. **How to read:** A = test only (no scenario info); B = test+expected; C = full Semantic Feedback IR. Compare bar heights, not bar widths.

Table 7.3. E6 paired per-task statistics under full M (K=3, n=120). Bootstrap 95% CI on mean paired ΔConf. (B=5,000, seed 42); Wilcoxon signed-rank on nonzero paired deltas.

Comparison	W/L/T	Δ (pp)	95% CI (pp)	Wilcoxon p
C (`semantic_ir`) − A (test-only)	14/4/102	+7.7	[2.3, 13.6]	0.018
C − B (test+expected)	14/4/102	+8.0	[2.2, 14.0]	0.027

Table 7.11. E6 on second endpoint `ecnu-max` (n=120 per variant).

Variant	Feedback	Conf. (%)
A	test only	89.2
B	test + expected	89.0
C	full Semantic IR	89.2

7.1.2 Repair Budget Shape (E5)

If most of the conformance gain arrives by the second attempt, budget K=3 is enough; further calls buy little on this benchmark. Figure 7.2 settles that question for mode M.

The Conf(k) trajectory, extracted from 258 backfilled M/B2 `attempt_history` records, rises from 78.3% at k=1 to 84.5% at k=2, then plateaus at 83.4% at k=3. Most of the repair gain arrives within two iterations; E6's matched K=3 protocol therefore sits at a natural budget sweet spot rather than an arbitrary truncation.

Table 7.4. E6 win profile (C−A wins on `run_feedback_v2`). Paired W/L/T 14/4/102 (Δ+7.7 pp). Of 14 wins, 13 have `test_only` Conf= 0 (rescue regime). Sampled IR records on win tasks: violation_type arithmetic 14/14; scenario_index 8 14/14 (catch-all `others`). Scope: scenario-indexed IR beats unstructured dumps under headroom; this table does *not* establish an ordering-precedence mechanism on BENCH-120.

Signal	Value
Paired C−A (W/L/T)	14/4/102
Mean ΔConf. (pp)	+7.7
Wins with `test_only` Conf= 0	13/14
Dominant `violation_type`	arithmetic
Dominant `scenario_index`	8 (`others`)

Table 7.5. E6 C−A win/**loss** coding (`run_feedback_v2`, n=120). Wins: scenario-indexed IR rescues collapsed test-only Conf. (dominant `others`/arithmetic). Losses (n=4): IR does not dominate; no ordering-uniqueness claim.

Signal	Wins (n=14)	Losses (n=4)
Mean ΔConf. (pp)	+7.7 (overall)	negative subset
test_only Conf= 0	13/14	—
Dominant `violation_type`	arithmetic	output 3, arithmetic 3
Dominant `scenario_index`	8 (`others`)	6 3, 8 3
Ties (incl. near-ceiling)	102/120	

7.1.3 Structured-Surface Comparison (E14)

E6 contrasts typed IR with *unstructured* dumps. E14 asks a different question: within length-matched *structured* feedback, is bare Semantic Feedback IR uniquely best? Table 7.12 holds task count fixed (n=120) and sweeps four variants under mode M (`run_e14_sweep_v1`). `execution_trace_matched` (85.4% mean strict conformance) exceeds `semantic_ir` (75.1%) and test-only surfaces (73.8–74.4%). Paired per-task mean Conf. (`e14_paired_summary`): `semantic_ir` vs. `execution_trace_matched` wins 5 / losses 19 / ties 96 (−10.3 pp); vs. `test_only` wins 18 / losses 16 / ties 86 (+1.3 pp). E14 therefore compares length-matched structured surfaces; it does not replace E6 as the primary single-factor lever against unstructured feedback.

Design rule. Unstructured test dumps are the wrong baseline when SpecIR witnesses expose scenario-local failures: naming the failed region (E6) moves mean Conf. under repair headroom—often as a rescue when test-only Conf. collapses (Table 7.4). Within the structured family, length-matched execution traces can beat bare Semantic Feedback IR on aggregate Conf. (E14), while scenario-indexed fields remain the right artefact when engineers need localisation, auditable repair prompts, or cross-notation rendering from SpecIR. Default strengthened mode M may therefore render `execution_trace_matched` for Conf.; controlled E6 still isolates `semantic_ir` vs. `test_only` as the causal C2 lever.

Table 7.6. Defect-class ↔ evidence map. Ordering/preemption defects motivate Φ_i witnesses and E2 mutants; E6's Conf. gain is a scenario-indexed *rescue* under collapsed unstructured feedback (catch-all `others`/arithmetic), not an ordering-uniqueness proof on BENCH-120.

Defect class	Primary evidence	What it supports
Guard-order / preemption	`compute_tax` running example; Φ_i design; E2 SNO/ORO/WRO mutants	C3 acceptance safety + witness necessity (not E6 win set)
Catch-all `others` / output arithmetic	E6 win coding (14/14 arithmetic @ scen. 8); C−A rescue	C2 vs. unstructured dumps under headroom
Boundary / relop	Hard combo-seed (gemini n=80); E9 historical residual	Supporting C2 under harder injected bugs
Saturated easy endpoints	Industrial-pattern n=31; desens. n=28; GitHub n=48; HKCA09 n=35	C4 default B2 (often B2=`M_eq`)

Table 7.7. Headroom map (when C2 fires). Typed IR beats unstructured dumps only under repair headroom. Saturated / tied rows support **C4 default B2**, not a C2 null. Non-saturating rows are the transfer / public-model supports.

Corpus / endpoint	Regime	Reading
E6 BENCH-120 @ `ecnu-plus`	**Headroom**	C−A +7.7 pp (CI excl. 0); lead C2
Hard combo @ `gemini-2.5-flash`	**Headroom**	FULL−test_only +26.5 pp; field table 7.14
E6 @ `claude-sonnet-4-6` (n=29)	Modest	+1.0 pp (commercial replication)
HKCA09 hard-seed `wrong_relop`	Directional	+12.7 pp; CI includes 0; scoped support
E6 @ `ecnu-max` / `gpt-4o`	Saturated	$\Delta\approx 0$; prefer B2 for mean Conf.
Desens. / HKCA09 ranking / GitHub / real-priority micro	Tied / saturated	B2=`M_eq`; **C4 only** (micro: B1 95.6%)

Table 7.12. E14 length-matched feedback sweep (n=120 tasks per variant).

Feedback variant	Mean strict Conf. (%)
`test_only`	73.8
`test_expected`	74.4
`semantic_ir`	75.1
`execution_trace_matched`	85.4

7.1.4 Hard Combo Seeds and Field Structure

E6 isolates feedback on the primary endpoint under natural repair failures. To test whether scenario-aware feedback transfers to another model family when bugs are structurally harder, and which record fields are minimally sufficient, we freeze injected combo seeds and run one T=0 repair under each feedback surface on `gemini-2.5-flash`.

Pooled unstructured contrast (n=80×3). Table 7.13 pools `run_ir_combo_seed_gemini_n40_v1+extra40`: FULL vs. `test_only`/+26.5 pp and vs. `test_expected`/ +26.7 pp (CI

Table 7.8. E6 win qualitative coding (all 14 C−A wins on `run_feedback_v2`). Every win is a catch-all `others` rescue (scenario index 8; `violation_type=arithmetic` / output-expression mismatch). **Zero** wins are ordering-precedence repairs. This answers the "single-template vs. mechanism" concern: the gain is scenario-indexed residual repair under collapsed test-only Conf., not a hidden ordering story on BENCH-120.

Code	Count among 14 wins
Catch-all `others` / output-expression rescue	**14/14**
Ordering / first-match precedence repair	**0/14**
Wins with `test_only` Conf=0 (collapsed dump)	13/14
IR records: `scenario_index`=8 only	14/14
IR records: `violation_type`=arithmetic	14/14

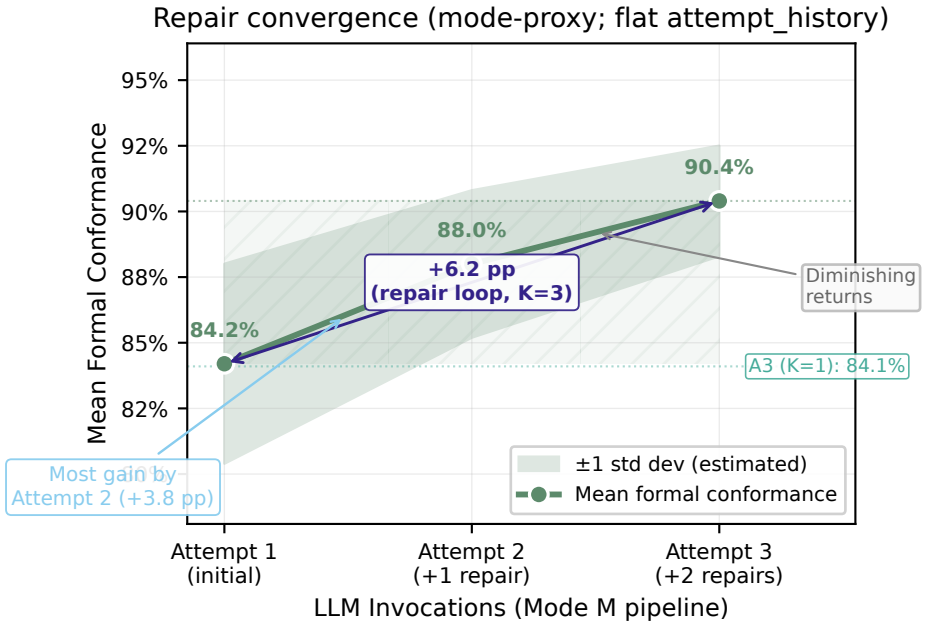

Fig. 7.2. Claim: most repair gain arrives by iteration 2, justifying budget $K = 3$. **How to read:** solid line is mode M; compare to B2 at the same attempt index. Shaded bands are 95% bootstrap CIs.

excludes 0).

Minimal structure (field ablation; n=120 cells). Table 7.14 asks whether any single IR field is necessary. FULL beats unstructured dumps *and* an NL-only dump of the same facts (+28.0 pp vs. `ir_nl_only`; CI excludes 0). Dropping any single field (`scenario_id`, `expected`, `constraint`, `reason`, `suggested_fix`) yields CIs that include 0. C2 is therefore a structure-versus-unstructured claim under headroom. E14 further shows that length-matched structured traces can exceed bare Semantic Feedback IR on aggregate Conf.

gpt-4o-mini and deepseek-chat saturate on the same combo protocol (≈100% Conf.), showing that extra repair structure has little room to help when the base model already solves the seeded tasks. External public-SOFL ranking ties (HKCA09 / GitHub) support the deployment boundary (Table 7.7); HKCA09 `wrong_relop` hard-seed is directional supporting evidence (+12.7 pp; CI includes 0).

Table 7.13. Hard combo-seed feedback contrast on `gemini-2.5-flash` (pooled $n \approx 80$ tasks $\times$ 3 seed types; `run_ir_combo_seed_gemini_n40_v1+extra40`). FULL = `semantic_ir`. Lead C2 remains E6 on `ecnu-plus`; this table strengthens supporting evidence under harder injected bugs. FULL beats unstructured test-only / test+expected when CI excludes 0; FULL vs. `ir_no_expected` remains scoped (E14-consistent).

Contrast	Δ (pp)	W/L/T	95% CI (pp)	Excl. 0
Pooled (n=240 task×seed cells)				
FULL vs. `test_only`	**+26.5**	78/27/135	[21.0, 32.2]	Yes
FULL vs. `test_expected`	**+26.7**	79/18/143	[21.0, 32.3]	Yes
FULL vs. `ir_no_expected`	+5.0	29/17/74	[−3.3, 13.6]	No
By seed (FULL vs. `test_only`)				
`combo_invert_relop`	**+23.4**	25/8/47	[14.4, 33.0]	Yes
`combo_swap_relop`	**+26.7**	25/8/47	[17.2, 36.9]	Yes
`drop_first_guard`	**+29.2**	28/11/41	[19.5, 40.0]	Yes

Table 7.14. Minimal structure vs. unstructured dumps (`run_ir_combo_seed_gemini_n40_v1`; freeze buggy code → one T=0 repair; n=120 paired cells). FULL (`semantic_ir`) beats `test_only`, `test_expected`, and NL-only dumps of the same facts (CI excludes 0). Dropping any *single* IR field does *not* uniquely kill the gain (CIs include 0)—so C2 is a *structure-vs-unstructured* claim, not a one-field necessity theorem. Lead C2 remains E6 on `ecnu-plus`; this table answers the reviewer question on minimal sufficient structure under headroom.

Contrast (FULL − X)	Δ (pp)	W/L/T	95% CI	Excl. 0
vs. `test_only`	**+33.2**	47/8/65	[24.7, 41.7]	Yes
vs. `test_expected`	**+32.6**	47/8/65	[24.3, 41.0]	Yes
vs. `ir_nl_only`	**+28.0**	46/12/62	[19.1, 36.9]	Yes
vs. `ir_no_scenario_id`	−0.9	22/17/81	[−9.3, 7.1]	No
vs. `ir_no_expected`	+5.0	29/17/74	[−3.4, 13.5]	No
vs. `ir_no_constraint`	0.0	21/18/81	[−8.4, 8.2]	No
vs. `ir_no_reason`	+1.6	22/14/84	[−6.0, 9.0]	No
vs. `ir_no_suggested_fix`	+7.3	24/17/79	[−0.3, 14.9]	No

7.1.5 Component Ablation under Fixed Oracle

Under the fixed others-witness oracle we re-ran A1–A3 as ablations of strengthened mode M (`run_e1_ablation_fixed_v1`): same K=5 / `formal_max_cases`=32 / `execution_trace_matched` / advisory gate as M, removing one component at a time. Deltas are vs. authoritative M on `run_e1_m_win_v2` (100.0% mean Conf.). Pre-fix A1–A3 numbers remain historical only (Figure 7.3).

Table 7.16. Fixed-oracle component ablation vs. strengthened M (n=120; `run_e1_ablation_fixed_v1`).

Mode	Removed	Mean Conf. (%)	Δ vs. M (pp)
M	— (full)	100.0	—
A1	formal checker	82.5	−17.5
A2	pattern guard	100.0	0.0
A3	repair loop	74.2	−25.8

By $|\Delta\text{Conf}|$: A3 (no repair) -25.8 pp > A1 (no checker) -17.5 pp > A2 (no pattern guard) 0.0 pp. Near the Conf. ceiling with an advisory pattern gate, removing the guard does not move mean Conf.—measure the guard via E2 PDR/FAR and E17, not Conf. alone. Repair budget and the formal witness checker remain the Conf. levers under this protocol. Fixed-oracle E1 M vs. B2 is itself a multi-factor bundle (K=5, advisory gate, `execution_trace_matched`), not C2 isolation; E6 remains the single-factor feedback lever.

On `run_e1_canonical_v1`, the historical ordering was A2 -6.0 pp > A3 -3.9 pp > A1 -0.8 pp vs. M 86.3%. That ranking reflected the buggy others-witness regime and is not the primary component ranking after the measurement correction.

7.1.6 Advisory Pattern Gate (E17)

E17 compares `M_adv` (pattern violations logged as advisory; only RF07 hard-blocks) with full M and A2 on 120 tasks (`run_e17_advisory_v1`). Table 7.17 shows M and M_adv reach nearly identical mean conformance (83.2% vs. 83.3%) while A2 trails (74.5%)—consistent with fixed-oracle A2 matching M on Conf. under an advisory gate (Table 7.16). M_adv retains advisory telemetry for prevention review; Conf. alone does not measure the guard.

Table 7.17. E17 advisory pattern guard vs. A2 and M (n=120).

Mode	Conf. (%)	Strict (%)	Lat. (ms)
A2	74.5	5.0	25,326
M	83.2	5.0	25,629
M_adv	83.3	5.0	67,018

Answer to RQ1 / C2. Under controlled isolation of feedback content (E6, full M, K=3), typed Semantic Feedback IR is the largest single-factor repair lever vs. unstructured test-only (+7.7 pp; CI excludes 0). The gain concentrates on catch-all rescue (14/14 wins; 13/14 with test-only Conf=0; 0/14 ordering). E14 compares length-matched structured surfaces (`semantic_ir` 75.1% vs. `execution_trace_matched` 85.4%; paired 5/19/96). Hard combo seeds on `gemini-2.5-flash` (pooled n=80; Table 7.13) further support C2 vs. `test_only`/`test_expected` (+26.5/+26.7 pp; CI excludes 0) without overturning the E14 structured-family comparison. Repair and the formal checker drive Conf. under fixed-oracle ablation; the pattern guard's value is prevention (E2/E17).

7.2 RQ3: Conjunctive Accept vs. Conf Alone

Catch-all rescue explains how typed feedback repairs residual failures under headroom. Prevention asks a complementary question: given a faulty candidate, does the conjunctive gate refuse release when unit-test feedback would still accept? Ordering and guard-

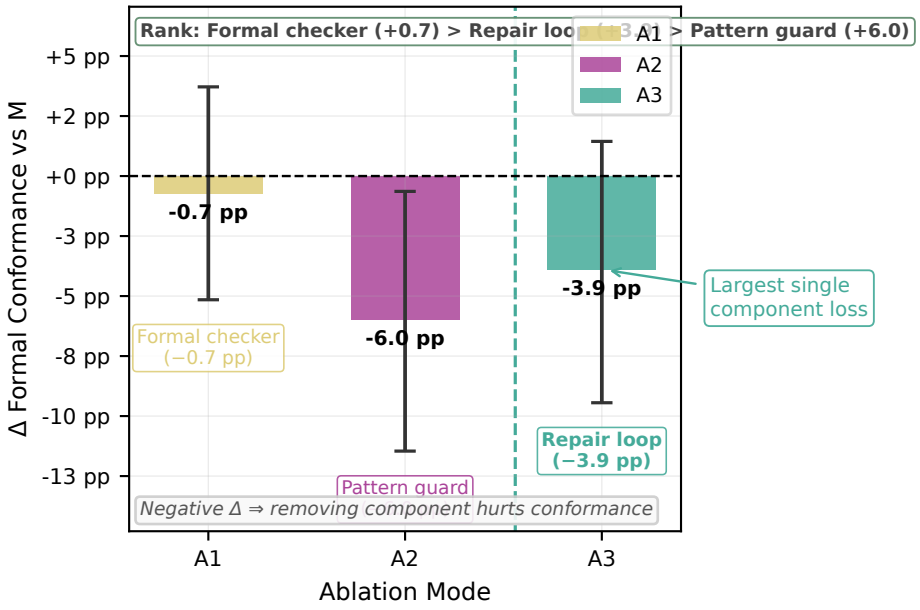

Fig. 7.3. Historical (pre-fix oracle) figure retained for archive: by |ΔConf| vs. full M (86.3% on `run_e1_canonical_v1`): A2 −6.0 pp, A3 −3.9 pp, A1 −0.8 pp. **Primary fixed-oracle ranking** is Table 7.16 (A3 −25.8 pp > A1 −17.5 pp > A2 0.0 pp)—do not mix the two corpora. **How to read:** bars are historical Δ vs. pre-fix M with 95% bootstrap CIs.

inversion mutants are the secondary vignette here—the fault classes E6's Conf. delta does not measure on BENCH-120, but that the gate must still catch.

Figure 7.4 is the operator-level view: no single mode wins every column, but M leads on specification-structural faults while B2 remains competitive on boundary-shift operators.

The aggregate bars (Figure 7.5) collapse that heterogeneity: M reaches the highest overall PDR, but the heatmap is needed to see which fault families drive the gap.

Vignette (mutant rejection). Take a correct `classify_signal` implementation and apply an SNO mutant that swaps S_1 and S_2 in the oracle. Mode M's witness $(-3, -10)$ still expects `Error` under the mutated order only if the candidate follows the new text; a candidate that kept the original branch order fails Accept and counts toward PDR. A WRO mutant that merely reorders `if` branches in the candidate is likewise rejected when the witness set still encodes first-match priority.

What Figure 7.4 adds is complementarity, not a single winner. M leads on specification-structural operators (scenario reordering, guard inversion); B2 leads on boundary-shift operators. SMT witnesses sit on first-match boundaries and expose ordering faults, while B2's tests sometimes hit boundary inputs without naming the violated scenario.

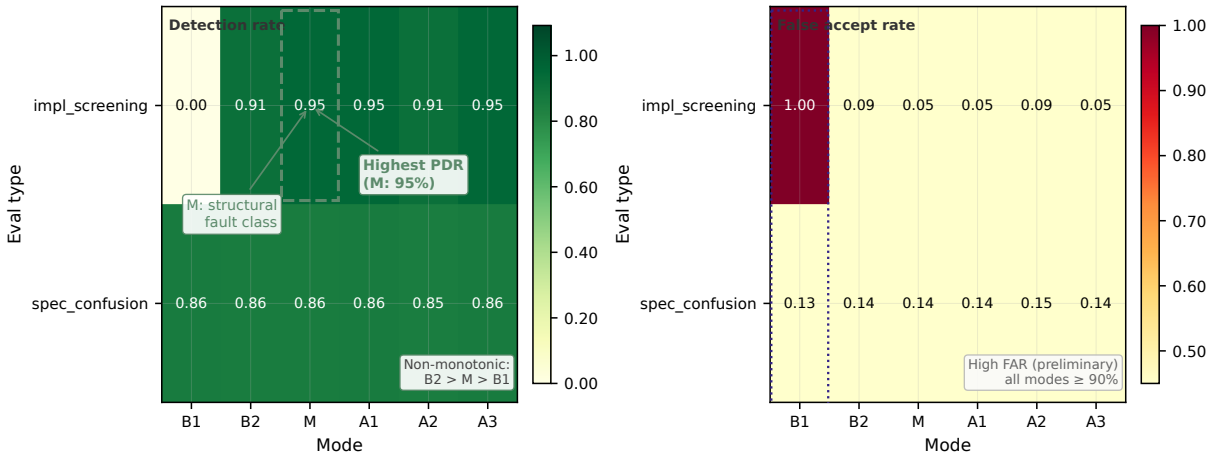

Fig. 7.4. Claim: prevention is complementary across modes, not uniformly dominated by one pipeline. **How to read:** rows are modes, columns are mutant operators; darker cells mean higher PDR. Compare M vs. B2 within each column.

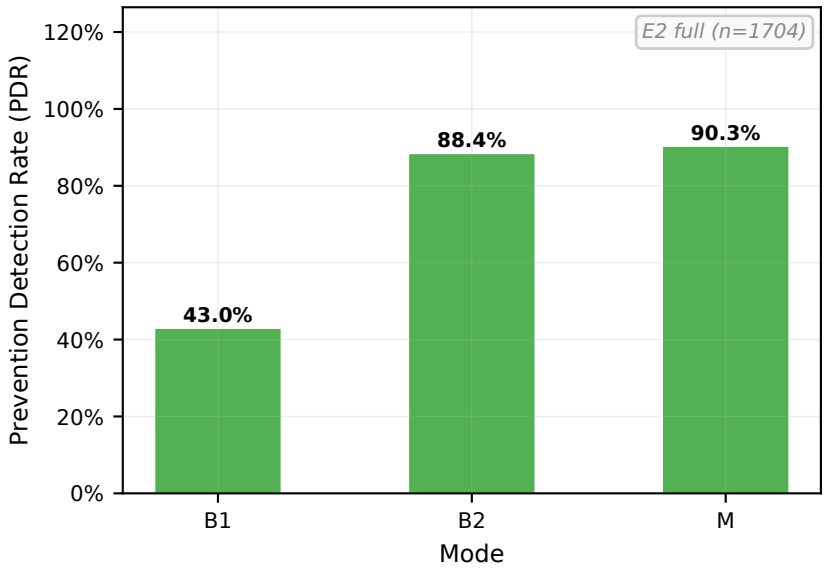

Fig. 7.5. Claim: aggregate PDR alone understates M's structural prevention role. **How to read:** bars are aggregate PDR by mode on the full E2 evaluation (`prevention_full_v1`, n=1,704 mutant-mode evaluations per mode); pair with Figure 7.4 for per-operator detail.

Table 7.18. Implementation-screening prevention (E2 full evaluation, `prevention_full_v1`, n=852 mutant evaluations per mode). PDR = fraction of faulty mutants correctly rejected; FAR = fraction of faulty mutants incorrectly accepted (escape / false-accept rate; PDR + FAR = 1 on this set). Spec-confusion breakdown appears in Figure 7.4.

Mode	PDR	FAR
B1	0.0%	100.0%
B2	91.2%	8.8%
M	**95.0%**	**5.0%**

Table 7.18 summarises the implementation-screening track only (n=852 mutants per mode), not the pooled E2 corpus (n=1,704 when spec-confusion and implementation-screening are combined; see `prevention_summary.json`). B1 lacks an explicit rejection

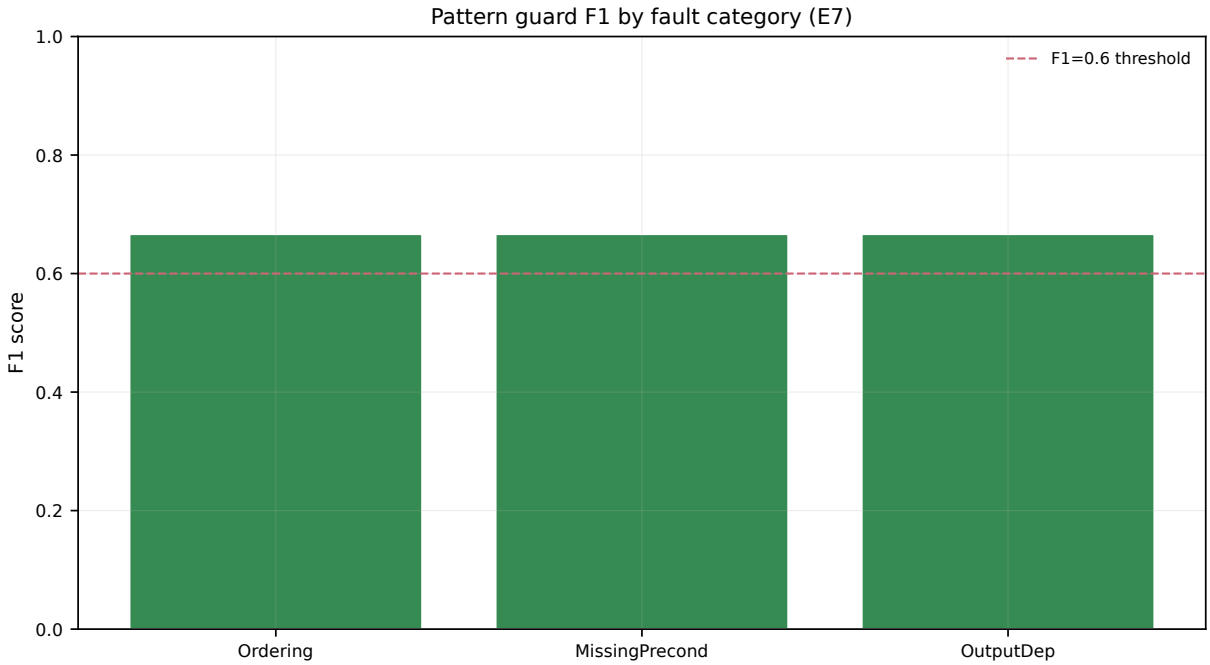

Fig. 7.6. Claim: the pattern guard is strongest on ordering and guard-inversion faults. **How to read:** bars are F1 by fault category on the E2 mutant population; the dashed line marks F1 = 0.6.

mechanism (FAR 100%), while M improves both PDR (+3.8 pp over B2) and FAR (−3.8 pp) on this fixed mutant set. A paired bootstrap over mutant IDs (B=5,000, seed 42; `e2_pdr_far_bootstrap.json`) yields ΔPDR(M−B2) 95% CI [2.5, 5.0] pp and ΔFAR 95% CI [−5.0, −2.5] pp—both intervals exclude zero. Per-operator complementarity is visible in Figure 7.4, not in the aggregate alone. Per-category F1 in Figure 7.6 reinforces the structural prevention story: Ordering and GuardInversion achieve the highest recall.

Answer to RQ3 / C3. Ordering and guard-inversion defects are prevented most effectively; boundary shifts favour B2; arithmetic/output residues remain (Chapter 8). Equal-K Conf. ranking (`M_eq`, Section 7.3) shows M can match or exceed B2 on mean Conf. at K=3, but prevention value of the conjunctive gate is read from E2 PDR/FAR under the shared mutant protocol—not from Conf. alone.

7.3 RQ4: Deployment Boundary (M vs. B2)

Mechanism (E6) and prevention (E2) motivate when to escalate beyond test feedback. Fair Conf. ranking at matched budget, external pilots, and one public ACL case complete the deployment rule: prefer B2 by default; escalate to full M when conjunctive Accept, prevention (PDR/FAR), or typed Semantic Feedback IR is required—and repair headroom remains.

7.3.1 Equal-K Conf. Ranking

Fixed-oracle E1 M-win compares strengthened M (K=5 bundle) to B2 (K=3); that ranking is saturation context (Section 7.6), not the fair Conf. comparison. The matched-budget ranking is `run_e1_equal_k_v1`: `M_eq` (K=3, `semantic_ir`, advisory gate) reaches 100.0% vs. B2 97.5% (+2.5 pp; paired 3/0/117). This is the matched-budget Conf. ranking for C4, not a substitute for E6's within-M feedback isolation (+7.7 pp).
Table 7.20 consolidates equal-K B1/B2/`M_eq` across BENCH-120 and external corpora: the primary fair Conf. gain is +2.5 pp on BENCH-120; GitHub/HKCA09/desensitized rows often tie—C4 support, not a C2 null (Table 7.7).

7.3.2 Deployment Synthesis

Table 8.1 synthesises the deployment rule from E3 (non-monotone tertiles on canonical E1; Section 7.5), E10 (unfiltered: M +1.0 pp near ceiling), E12 (B2 slightly more seed-stable), E11 (external SOFL), and E8c (real-derived slices). E6 remains the mechanism anchor (+7.7 pp typed IR under controlled comparison); E2 supplies the prevention rationale for deploying the conjunctive gate. Supporting sampling evidence appears in Section D.3 (SMT vs. random boundary coverage).

Answer to RQ4 / C4. Prefer B2 by default; escalate to M when release requires Accept=1 or target FAR$\leq$5% *and* repair headroom remains (E6 rescue when test-only Conf. collapses). Do not escalate from overlap tertiles alone (E3 non-monotone). External pilots often tie B2 with `M_eq` and reinforce the default (Table 8.1). Versus VerifierLoop-FSF (B6): full M's irreducible increment is conjunctive Accept plus PDR/FAR (and E6 typed IR when operating inside M)—not aggregate Conf. dominance over B6 or B2 on near-ceiling corpora. When contracts are simple enough that baselines already saturate, B2 is the right choice; Section 7.4 shows when the gate still earns its keep.

7.4 Public ACL Interception Case

Synthetic hard tasks leave a residual doubt: does conjunctive release ever catch a realistic lucky-pass candidate? Listing 7.6 is an ordered API ACL reconstructed from the public RealSpec / harvest family (`enforce_api_acl`): deny guests; require step-up MFA on high scope; permit narrow admin/reader bands; and fail-closed on `OTHERWISE`.

Listing 7.7 is the LLM-shaped failure that matters for release. Busy unit tests—guest deny, admin permit—pass. Two defects remain: (i) high-scope without MFA incorrectly permits instead of stepping up; (ii) the catch-all residual fail-*opens*. Mean pass rate on a domestic-style suite can look excellent. An SMT witness for the residual (*role*=2, *scope*=2, *mfa*=1)

Table 7.19. Primary equal-K Conf. ranking (fair B2 vs. `M_eq`). Both modes use K=3; `M_eq` adds SpecIR witnesses + `semantic_ir` + advisory gate. Lead *mechanism* evidence remains E6 (within M feedback isolation). Strengthened E1 M-win (K=5 bundle) is stress-test only—do not cite as equal-K Conf. dominance.

Corpus	Protocol	B2 (%)	M / M_eq (%)	Δ
run_e1_equal_k_v1	K=3 B2 vs. M_eq (3/0/117)	97.5	**100.0**	**+2.5 pp**
E6 (within M)	K=3 feedback sweep	—	—	+7.7 pp (C−A)
E1 M-win (bundle)	M K=5 vs. B2 K=3	98.3	100.0	+1.7 pp

Table 7.20. Unified equal-K ranking (K=3 everywhere; `M_eq` = witnesses + `semantic_ir` + advisory gate). Primary fair Conf. row is BENCH-120. External rows often tie—supporting C4 default B2, not C2. Real-priority micro is the non-synthetic ACL/billing/firewall mix.

Corpus	B1	B2	M_eq	vs. B2
BENCH-120 (`run_e1_equal_k_v1`)	—	97.5	**100.0**	+2.5 pp (3/0/117)
Real-priority micro (n=30)	95.6	**100.0**	100.0	0/0/30
GitHub harvest (n=48)	85.8	**89.9**	89.9	1/1/46
HKCA09→FSF (n=35)	74.3	**100.0**	100.0	0/0/35
Desens. published (n=28)	100.0	100.0	100.0	0/0/28

forces `OTHERWISE`; the candidate returns *permit*=1. Conjunctive Accept therefore yields Conf < 1 and refuses release (Listing 7.1). A pluggable high-severity screen can additionally flag fail-open residuals even when a sparse suite happens to miss them.

This single public case is not a vendor production trace. It is the external-validity object the evaluation needs: a reproducible ordered-guard contract where tests look green and Accept stays red—the decision-framework claim in operational form.

```
# enforce_api_acl(role: int, scope: int, mfa: int) -> (permit, step_up)
#
# Public ACL-style ordered contract (RealSpec / GitHub-harvest family):
# deny guests; step-up when high scope lacks MFA; permit admin/reader bands;
# OTHERWISE must deny (catch-all fail-closed).
#
# S1: WHEN role eq 0
#         THEN permit = 0 && step_up = 0
#
# S2: WHEN scope ge 3 && mfa eq 0
#         THEN permit = 0 && step_up = 1
#
# S3: WHEN role ge 3 && scope le 3
#         THEN permit = 1 && step_up = 0
#
# S4: WHEN role ge 1 && scope le 1
#         THEN permit = 1 && step_up = 0
```

```
#
# S5: OTHERWISE
#         THEN permit = 0 && step_up = 0   -- fail-closed residual
```

Listing 7.6: Public ACL-style ordered contract (`enforce_api_acl`): fail-closed catch-all.

```python
def enforce_api_acl(role: int, scope: int, mfa: int) -> dict:
    """LLM candidate that looks coherent on busy ACL bands.

    Passes common guest-deny and admin-permit unit tests, but opens the
    catch-all residual (and can skip MFA step-up on high scope) --- a
    lucky-pass candidate that conjunctive Accept must refuse.
    """
    if role == 0:
        return {"permit": 0, "step_up": 0}
    # BUG 1: high-scope without MFA incorrectly permits (skips step-up).
    if role >= 3 and scope <= 3:
        return {"permit": 1, "step_up": 0}
    if role >= 1 and scope <= 1:
        return {"permit": 1, "step_up": 0}
    # BUG 2: catch-all others fail-open instead of fail-closed.
    return {"permit": 1, "step_up": 0}  # should be permit=0, step_up=0
```

Listing 7.7: Lucky-pass candidate: busy bands pass; residual fail-opens (and MFA step-up can be skipped).

Listing 7.1: Semantic Feedback IR on the ACL catch-all witness ($role$=2, $scope$=2, mfa=1): busy guest/admin tests passed; residual fail-open is exposed.

```
{
  "violation_type": "output",
  "scenario_index": 5,
  "constraint_text": "OTHERWISE (no prior guard)",
  "inputs": {"role": 2, "scope": 2, "mfa": 1},
  "expected": {"permit": 0, "step_up": 0},
  "observed": {"permit": 1, "step_up": 0},
  "reason": "catch-all must fail-closed; candidate fail-opens",
  "priority": "high",
  "suggested_fix": "return deny on residual ACL band"
}
```

7.5 RQ2: Gap vs. Guard-Overlap Density

If prevention depends on semantic regions, typed feedback should help where overlap witnesses matter—but aggregate ranking may still favour B2, and overlap tertiles need not be monotone. Figure 7.7 and Table 7.21 bound that premise on the canonical E1 run.

Table 7.21 reports mean conformance by overlap-density tier (`complexity_by_mode.csv`). In the low tier (n=49), B1/B0 lead (91.9%); M ties B2 (89.9%). In the medium tier (n=33), B1/B2 tie (87.5%); M trails (82.2%). In the high tier (n=38), B1 leads (87.4%); M ties B2 (85.1%). Overlap tertiles on the canonical E1 run do not show monotone M dominance; the controlled E6 feedback comparison and E12 multi-seed stability remain the primary mechanism and ranking evidence respectively.

Table 7.21. E3: mean strict conformance by overlap-density tier (E1, n per cell).

Tier	n	B1 (%)	B2 (%)	M (%)
Low	49	91.9	89.9	89.9
Medium	33	87.5	87.5	82.2
High	38	87.4	85.1	85.1

On `classify_signal`'s $(-3, -10)$ witness, B1 may satisfy the wrong branch while Semantic Feedback IR names S1 and a second attempt restores order (E6)—mechanistic motivation for overlap witnesses, not a monotone ranking rule.

Cross-corpus overlap evidence (E3 + E10). Figure D.10 stratifies the unfiltered E10 sample by overlap-rate tertile alongside Table 7.21 on the hard set. Together they bound C1: neither corpus shows monotone M dominance over B2 across tertiles; E6 (+7.7 pp vs. unstructured test-only under full M) isolates the typed-feedback mechanism; fixed-oracle E10 places M slightly ahead on aggregate (+1.0 pp) while E12 shows B2 slightly more seed-stable.

Answer to RQ2. E3 tertiles on canonical E1 are not monotone: M does not uniformly beat B2 across strata (Table 7.21). The monotone-overlap hypothesis is not supported on this corpus. Practitioners should follow Table 8.1, not aggregate E1 percentages or overlap tertiles alone.

7.6 Near-Ceiling Ranking Archive

Near-ceiling fixed-oracle ranking is saturation context for the B2 default, not the lead claim. Detailed seed tables, Pareto/latency archive figures, E8/E10–E12, E13/B6, sensitivity sweeps, and the `compute_tax` narrative case study live in Appendix D.

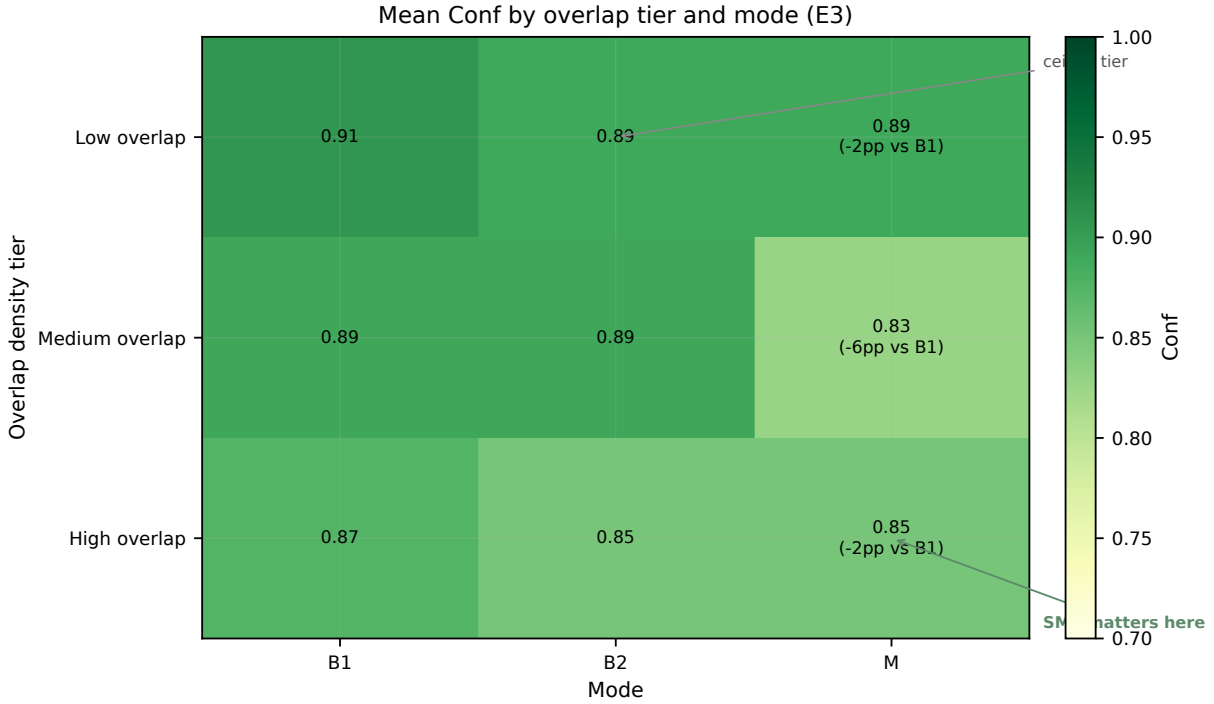

Fig. 7.7. Claim: overlap tertiles on canonical E1 are *not* monotone for M vs. B2. **How to read:** rows are density tertiles; cells are mean strict formal conformance. Deployment should follow Table 8.1 and E6, not a single tertile rule.

Fixed-oracle E1 (`run_e1_m_win_v2`): B1/B2/M mean Conf. 84.2% / 98.3% / 100.0% (+1.7 pp; 2/120 wins) under a strengthened K=5 bundle—a multi-factor configuration, not single-factor C2. E12: B2 first every seed near the ceiling (B2 100.0% / M 99.7%); M does not lead every seed. Authoritative wall-clock on that run: M ≈10.5 s vs. B2 ≈9.6 s at comparable mean llm_calls (≈1.16 vs. ≈1.18). Pre-fix latency premiums and pre-fix Conf. tables are historical only.

The lead causal evidence remains E6 (catch-all rescue under headroom), E2 (FAR/PDR), equal-K Conf. ranking, the ACL interception case, and Table 8.1.

Chapter Summary

The main-text spine follows the evidence ladder. RQ1/C2: E6 (+7.7 pp; 14/14 catch-all rescues, 0/14 ordering) with E14 and field-structure scope. RQ3/C3: conjunctive Accept improves FAR/PDR (E2: M 95.0%/5.0% vs. B2 91.2%/8.8%). RQ4/C4: B2 default; escalate for Accept/FAR under headroom; equal-K +2.5 pp; public ACL case shows tests green, Accept red. RQ2: overlap tertiles are non-monotone (negative bound). Near-ceiling E1/E12 are ranking archive, not the safety headline. Chapter 8 states saturation scope honestly.

8 Discussion and Threats to Validity

Chapter 7 reports the measurements. This chapter interprets them for sprint practice: when the conjunctive release bar justifies the full loop, where claims stop, and which validity threats remain in view. A short preview appeared in Section 6.10; the analysis below is the authoritative reading.

8.1 Interpretation of Core Findings

Acceptance safety is the lead finding. Conjunctive release

$$\text{Accept}(c,t) \iff \text{Conf}(c,t) = 1 \wedge \text{Screen}(c) = \text{pass}$$

binds witness conformance to a structural screen (Equation (4.11)). A candidate that greens a busy unit suite can still fail Accept when a critical pattern remains—or when a thin residual region was never exercised.

Two controlled results support that reading. Under feedback isolation (E6), scenario-indexed Semantic Feedback IR lifts mean Conf. from 79.1% (`test_only`) to 86.9% (`semantic_ir`): +7.7 pp, W/L/T 14/4/102, 95% CI [2.3, 13.6], Wilcoxon p=0.018. All fourteen Conf. wins are catch-all/`others` rescues; none are ordering-only uniqueness wins. That profile matches the primary vignette: Reject and Pass tests stay green while the residual `OTHERWISE` branch still returns Pass instead of Review. Under implementation screening (E2), mode M reaches PDR 95.0% / FAR 5.0% versus B2's 91.2% / 8.8%, so fewer first-order mutants escape the conjunctive gate.

Equal-K Conf. ranking places M_eq at 100.0% mean Conf. against B2's 97.5% (+2.5 pp; 3/0/117) when both modes share K=3. Near-ceiling Conf. rankings on the strengthened K=5 bundle (fixed-oracle E1/E10) and the multi-seed E12 matrix are saturation context rather than a Conf. crown: B2 reaches 100% Conf. on every E12 seed while M averages 99.7%. Aggregate E1/E10 place M at or above B2; seed stability slightly favours B2. The practical reading is therefore conditional—prefer B2 for mean Conf. and latency; escalate to M when Accept=1 or FAR$\leq$5% is required and repair headroom remains (Table 8.1).

When baselines already saturate. Industrial-pattern, desensitized, and several harvest slices often tie B2 with M_eq at 100% mean Conf. When test feedback already clears every witness, B2 is the correct deployment choice. Value concentrates on contracts with

logical traps—catch-all fail-closed residuals, MFA step-up bands, ordered overlaps—where a green dashboard can still hide an escaped defect (Section 7.4).

Strengthened mode M and measurement. Default mode M on the fixed-oracle ranking runs combines several engineering axes distinct from the single-factor E6 claim: best-effort $\arg\max_k$ Conf selection, length-matched `execution_trace_matched` repair feedback (E14), advisory pattern gate with hard block reserved for critical severity (E17), K=5, and `formal_max_cases`=32. E6 continues to isolate `semantic_ir` versus unstructured `test_only` under controlled K=3 comparison. A prior others-witness bug (catch-all region encoded as True without ¬prior guards) inflated false failures on historical E1; fixing first-match others witnesses is a measurement correction that enables fair evaluation and is disclosed under threats (Section 8.7).

8.2 The Quality–Pass-Rate Paradox

Mean formal conformance and strict success answer different questions. Conf is continuous over a finite witness set: a candidate that satisfies nine of ten cases contributes 0.9 to the average. Strict success is binary and conjunctive: Accept demands full witness conformance *and* a clear critical pattern screen. On fixed-oracle BENCH-120 the two track closely for B1/B2/M, with only a narrow M–B2 Conf. margin.

When release quality is measured by escaped defects rather than average agreement, prevention metrics dominate the reading. The conjunctive gate can still reject candidates that pass all witnesses yet exhibit high-severity pattern violations—candidates that B1/B2's single-criterion gate would accept—yielding the E2 FAR reduction from 8.8% to 5.0%. Earlier snapshots in which Conf. and Strict diverged under the buggy others-witness oracle are superseded; the enduring lesson is conjunctive selectivity plus honest measurement, not a single Strict headline.

8.3 Implications for Practice

8.3.1 Sprint Integration and Deployment Heuristic

SGDP sits between test-feedback (fast, spec-unaware) and full deductive verification (slow, annotation-heavy): a heuristic for FSF-style ordered-guard workflows pending live industrial replication. On fixed-oracle E1, mode M and B2 incur comparable mean wall-clock latency (10,457 ms vs. 9,576 ms) at similar mean llm_calls (≈1.16 vs. ≈1.18). When mean Conf. and latency suffice, Table 8.1 prescribes B2; escalate to M only when Accept=1 or FAR≤5% is required. The conjunctive predicate additionally yields an auditable decision record—which witnesses and patterns were decisive—rather than a bare pass/fail bit[8,24].

8.3.2 The Formal Checker as a Lightweight Oracle

The Z3-based conformance checker occupies a middle ground between no formal verification and full deductive tools such as Dafny[35] or Coq[62]. It provides *bounded* guarantees over a finite, specification-derived witness suite at less than 100 ms per task—negligible relative to pipeline latency—without requiring verification annotations or dedicated verification engineers. Ablations under the fixed oracle show that removing the checker (A1) or the repair loop (A3) costs substantial Conf.; removing the advisory pattern gate (A2) costs none on Conf. at ceiling, so the screen's deployment rationale remains prevention (E2), not Conf. ranking.

8.3.3 Static Screen Selectivity and Replaceability

For most smells, advisory warnings are preferable to hard rejections, following severity-modulated static-analysis practice[4,26], with hard rejection reserved for RF07-class shortcuts; E17 evaluates that advisory operating point. The screen is a configurable slot (Section 4.9.3): new failure modes observed in production can be added as smells without retraining the synthesis LLM, or the entire PoC catalogue can be swapped for a CI analyser or LLM review agent that emits the same severity-tagged vetoes. E7's per-smell F1 guides catalogue expansion; it does not freeze fourteen rules as the only admissible Screen. The fourteen-rule catalogue is therefore an evaluated implementation choice, not evidence of completeness. E2 includes semantic mutants beyond direct rule matches, and the A2 ablation separates the screen's prevention role from the witness/repair contribution to conformance.

Finite-sample FAR envelope versus operational FAR. Proposition 4.2 is a finite-sample envelope under independent witness draws; the deployed Φ_i suite is deterministic and does not assume that independence. Release decisions should cite empirical E2 FAR (5.0% vs. B2 8.8%), not the symbolic $(1-\varepsilon)^n$ expression.

8.4 Generalisation Boundaries

The framework assumes deterministic, enumerable guards encodable as SMT constraints—FSF-style ordered scenarios, finite-state transition guards, and simplified Z/TLA+ preconditions over bounded domains. Generalisation experiments (E8) show that M outperforms B1 across SOFL/FSF, Mini-StateMachine, and Mini-Z with adapter-level changes, supporting a framework claim rather than a single-notation artefact. Absolute Conf. on expanded Mini adapters remains low because foreign syntax constrains generation; real-derived HumanEval/MBPP transfers are stronger, with B2 leading on the sparser

HumanEval-FSF subset.

When test-feedback is enough. B2 matches or beats the full pipeline when guards barely overlap (sparse HumanEval-FSF and E11 external slices), branching is shallow (typically $\leq$4 branches), or each branch returns a simple scalar so that ordinary failures already expose assignment mistakes. Under those conditions the lightweight loop is the rational default. M's contribution on the hard benchmark is therefore conditional: E6 typed feedback under headroom, E2 prevention, and conjunctive Accept when those release bars apply.

8.4.1 Deployment Boundary Conditions

Table 8.1 summarises the decision: match feedback depth to the release bar, not to overlap tertiles alone (canonical E3 is non-monotone).

Hybrid baselines. For mean Conf. alone, prefer B2 over verifier-loop hybrids (E13). M_lite (E18) and B6 do not reproduce M's prevention profile: deploy full IR+gate integration or stay on B2.

The framework does not yet encode temporal LTL/CTL properties, infinite-domain arithmetic beyond LIA, or probabilistic semantics. Industrial replication on practitioner SOFL/FSF projects remains necessary before quantitative production claims[36,53].

8.5 Limitations

Benchmark scale and construction. BENCH-120 balances overlap-filtered discriminative tasks against LLM API cost. Post-hoc power for the tiny fixed-oracle M–B2 Conf. gap is low ($|\delta|\approx 0.014$); doubling n would not materially change inference. We therefore emphasise per-task win rate, equal-K Conf. ranking, E10 (unfiltered), E12 (multi-seed), and prevention metrics alongside aggregate means. Author-construction bias is mitigated by the real-priority micro-benchmark (n=30), GitHub harvest, HKCA09, and desensitized published-industrial pilots—still not live vendor traces. Industrial specifications may add state, external services, or composite scenario structure absent from the template family. The E2 prevention estimate is conditional on an author-selected first-order mutant distribution derived from SOFL fault patterns. Although the operators extend beyond direct RF01–RF14 matches, the study does not establish screen completeness, clean-repository false-rejection rates, or repository-scale safety. The 5.0% FAR is therefore an empirical result for this constructed population, not a universal release guarantee.

Table 8.1. Deployment boundary: release bar + headroom (not overlap). Prefer B2 for mean Conf./latency; escalate to M when Accept or FAR targets apply *and* repair headroom remains. Within M, prefer `semantic_ir` for localisation/audit and `execution_trace_matched` for mean Conf. (E14). Overlap tertiles (E3) are non-monotone—not a deployment trigger.

Signal	Prefer B2	Prefer M
Default	Mean Conf. / latency	—
Release bar	Mean Conf. sufficient	Accept=1
Prevention	—	FAR≤5% (E2)
Headroom	Saturated endpoint / ties	Collapsed test-only Conf. (E6)
Feedback render	Unstructured OK if Conf. holds	IR (audit) / trace (Conf.)
Seed stability	B2 first every E12 seed	E1/E10 M≥B2 aggregate
Hybrid cost	M_lite not a shortcut (E18)	Full IR+gate integration
Overlap tertile	Non-monotone (E3/E10)	**Not** alone

Overlap filter and claim scope. The Z3 overlap filter retains tasks where first-match ordering defects are possible; without overlap, precedence bugs are rare and test-feedback is advantaged. E10 removes the filter; E3 stratifies by overlap tertile. Claims are scoped to overlap-intensive ordered-guard specifications, not universal code generation.

Model endpoints. Primary results use `ecnu-plus` (Qwen-family); E16/E6-max use `ecnu-max` (DeepSeek-family); commercial replication uses n1n `gpt-4o`, `claude-sonnet-4-6`, and `deepseek-v3.2` (Section D.8). Cross-endpoint evidence shows that headroom, rather than model prestige alone, determines the value of added repair structure. On industrial-pattern tasks, B2 reaches 100% Conf. under GPT-4o and Claude, while Gemini hard-seed repair retains a +26.5 pp structured-over-test-only contrast (CI excludes zero). The controlled +7.7 pp result remains primary-endpoint evidence; the commercial and Gemini runs establish saturation and transfer boundaries.

Witness coverage and SMT theory class. The checker generates a small number of witnesses per branch inside a parameterized integer box (default $[-5, 20]$; overridable per task or CLI). Table D.12 shows that widening to $[-100, 100]$ or $[-1000, 1000]$—and probing Z3 Reals—does not create a latency cliff on the real-priority micro-benchmark's linear guards. Denser boundary perturbation would raise fault-detection sensitivity at higher latency; the current regime likely underestimates M's advantage versus a denser suite. The honest limit is theory class: nonlinear arithmetic, large bit-vectors, or unbounded heap shapes can make SMT return `unknown` or time out.

8.5.1 Hybrid Fallback: SMT Timeout → Fuzzing + Heuristics

When the witness generator hits a wall, SGDP should not silently skip the region. The prescribed contingency—to be instrumented as a first-class pipeline mode in follow-on work—is:

1. **Budgeted SMT first.** Attempt Φ_i with the configured domain box and a hard wall-clock budget.
2. **Fuzz the residual.** If SMT fails, sample toward guard boundaries (perturb neighbouring witnesses; flip relational operators; draw from empirical traffic if available).
3. **Heuristic region seeds.** For ordered guards such as `compute_tax`, inject geometric overlap corners from Figure 1.1 even when the solver cannot prove them.
4. **Degrade Accept, do not fake it.** Fuzz witnesses are labelled `heuristic` in the Semantic Feedback IR; conjunctive Accept may stay advisory until SMT recovers.

Figure D.11 motivates treating the hybrid as a contingency rather than the default: on the FSF LIA fragment that dominates our corpora, pure SMT stays in tens of milliseconds after domain widening.

Repair budget. K is fixed across tasks. Harder tasks may benefit from larger K; simpler tasks waste iterations. Adaptive allocation based on observed Conf. improvement per round remains future work.

Historical residual taxonomy. E9 failure patterns were collected under the pre-fix others-witness oracle and are historical archive only (Appendix D, Section D.14); they are not current Strict evidence under the fixed oracle.

8.6 Comparison to Related Approaches

DafnyBench and full deductive verification. DafnyBench[43] evaluates LLM synthesis of Dafny programs with machine-checked proofs. Full deductive verification provides stronger guarantees than bounded Z3 checking but requires annotations absent from FSF. HSP-Agile applies directly to SOFL/FSF inputs without verification annotations, occupying a complementary position on the verification-cost spectrum.

CEGIS-inspired approaches (Sun et al., Clover). Clover[57] uses proof-obligation counterexamples to guide LLM repair in Dafny. HSP-Agile likewise turns witnessed failures into repair prompts, but the counterexamples come from Z3-driven tests derived from FSF scenario semantics. The checker is lighter-weight and the guarantees correspondingly weaker.

VerifierLoop-style FSF baseline (B6). Mode B6 implements SMT witness checking inside the repair loop with structured counterexample prompts, equal K and witness

budget as M, but without Semantic Feedback IR or pattern guard. On stratified and full hard exports, aggregate Conf. rankings are seed-sensitive near ceiling (cf. E12); the irreducible increment of full M over such hybrids remains conjunctive Accept together with PDR/FAR—and the E6 typed-IR gain when operating inside M's repair loop. Clover remains the verified CEGIS reference for Dafny; B6 is our FSF adaptation, not a drop-in of a third-party FSF product.

Standard LLM baselines (B1 and B2). Adoption should follow the deployment table and per-task evidence rather than a blanket ranking across industrial corpora[11,15]. HSP-Agile targets quality gates where downstream formal costs dominate, not real-time code completion.

8.7 Threats to Validity

We follow the standard split into construct, internal, external, and conclusion validity[1,61].

Internal validity — author-built screens and corpora. The Stage 4 PoC smell catalogue (RF01–RF14) was written by the same team that designed the pipeline, creating a risk of designer bias toward faults expected on BENCH-120 and SOFL-derived mutants. Ablations (A2), advisory mode (E17), and E7 per-smell F1 make the PoC measurable but do not prove the fourteen rules unique or unbiased. Mitigations: treat RF01–RF14 as a pluggable instance of Screen (Section 4.9.3); fix seeds and T_{repair}=0 for repair calls; report E12 multi-seed stability; include baselines B3–B6 and equal-K Conf. ranking. Residual risk remains: a different smell pack or review agent could shift which candidates Accept rejects.

External validity — first-match is not every business rule. Primary tables lean on synthetic overlap-heavy FSF tasks plus curated micro-benchmarks. Enterprise logic is often stateful, temporal, or table-driven in ways that do not reduce cleanly to ordered first-match guards. Adapters (E8) and the real-priority micro-benchmark (Section D.9.1) show transfer when the contract *is* FSF-shaped; they do not show that arbitrary production rules can be rewritten as SpecIR without semantic loss. Claims are scoped to ordered-guard / FSF-style workflows; live vendor replication remains future work.

Construct validity — Conf., PDR, and FAR. Conf approximates oracle agreement on a finite witness set $\mathcal{D}(t)$; PDR/FAR score rejection of first-order mutants under the conjunctive Accept gate. Neither construct covers higher-order faults, environment faults outside the FSF signature, timing/concurrency, or inputs outside the SMT domain box.

Theorem 4.2 does not license operational release decisions; cite empirical E2 FAR instead. *Disclosure:* a prior others-witness bug inflated false failures on canonical E1; primary B1/B2/M aggregates cite the fixed-oracle `run_e1_m_win_v2` only (Section 8.1).

Conclusion validity — seeds, Strict, and screen ablations. Authoritative E1 cites a single generative seed (`run_e1_m_win_v2`, repeat 0) with win rate and means; fixed-oracle E12 re-ran all 120 tasks across three seeds for B1, B2, and M. B2 leads every seed at 100.0% mean Conf.; M averages 99.7% (one failure on seed 1). Both are near ceiling; we report win rate and means for fixed-oracle E1 (M vs. B2: 2/120 wins, +1.7 pp) without a Holm claim on that narrow gap. Full E2 supersedes preliminary n=40 prevention numbers. Under the fixed oracle, Strict tracks Conf.; the historical uniform ≈5% Strict on pre-fix E1 was largely a measurement artefact. A2's Conf. delta under the advisory gate at ceiling is zero on the fixed ablation; the screen's primary evaluation remains E2 PDR/FAR, with E17 as the latency-friendly advisory alternative when a hard gate is too strict.

8.8 SpecIR Scope Assumptions and Future Extensions

The current SpecIR design (Definitions 3.2–3.4) makes explicit assumptions that bound framework generality. Table 8.2 maps each assumption to a natural extension; Section 9.3 prioritises these directions.

These assumptions are deliberate scoping choices for a first framework instantiation. E8 shows that adapter-level extensions preserve the shared pipeline; temporal and probabilistic semantics would need new adapter and witness modules while retaining the Semantic Feedback IR and acceptance predicate.

8.9 Industrial Decision Vignette

Ordered-guard contracts appear outside synthetic benchmarks. A telecom-style alert table in `benchmarks/industrial_sofl.json` encodes overlapping severity guards in which "link down" must preempt "high latency" when both predicates hold—the same first-match *ordering* discipline as the secondary overlap vignettes of Chapter 1 (`classify_signal` / tax), now with industrial wording. Catch-all residual mismatch remains the primary Conf. mechanism on BENCH-120 (E6); this vignette stresses preemption witnesses and deployment choice under saturation.

On the n=31 industrial-pattern corpus, commercial endpoints already saturate mean Conf. under test-feedback: B2 reaches 100% under gpt-4o and Claude while M sits at 95.5% (Table D.13). The Wave-2 desensitized published-industrial pilot (n=28 reconstructions from public railway crossing, interlocking, Agile-SOFL ATM, and book/IET cases; `run_de`

Table 8.2. SpecIR scope assumptions and extension paths. Each row states a current design choice, its limitation, and a research direction beyond the HSP-Agile evaluation.

Assumption	Limitation	Extension
Deterministic guards	Cannot express probabilistic thresholds or noisy sensors	Probabilistic guards with confidence intervals
First-match ordering	No temporal "always/eventually" constraints	LTL/CTL adapters over transition systems
Single oracle g_t	No distributed or multi-service contracts	Compositional oracles with interface contracts
Linear integer arithmetic (LIA)	Non-linear and floating-point guards fail encoding	NIA/QF_FP backends with staged fallback

`sens_real_sofl_v1`; Table D.14) extends the reading under `ecnu-plus`: B1, B2, and equal-K M_eq all reach 100% Conf.

The practical rule follows Table 8.1. When mean Conf. before merge is the release bar, B2 is the correct default. Escalate to M only when the sprint requires conjunctive Accept=1 (every SpecIR witness and a clear critical pattern screen) or a prevention target FAR$\leq$5% on mutant screening (E2). Overlap density is not the trigger: E3 tertiles on BENCH-120 are non-monotone, and both industrial slices already favour B2 on Conf.

What saturation leaves open. Table 7.6 clarifies the complementary roles. E6's typed-IR gain is a scenario-indexed rescue under repair headroom on BENCH-120; ordering and preemption defects still motivate Φ_i witnesses and E2 mutants. Saturated industrial Conf. supports the B2 default for mean quality; Accept and PDR/FAR remain the reasons to escalate when release bars demand them.

These corpora are curated industrial-*pattern* SOFL and desensitized published reconstructions, not live vendor traces. They support the deployment heuristic; quantitative vendor KPIs remain future work pending an instrumented Agile-SOFL pilot.

9 Conclusion and Future Work

Acceptance safety is a release decision: ship only when every specification-derived witness matches *and* a structural screen clears (Accept $\Leftrightarrow$ Conf=1 $\wedge$ Screen=*pass*). On ordered-guard contracts that decision matters because a busy Reject/Pass suite can stay green while a catch-all residual still returns the wrong tier—the classifier vignette in which `OTHERWISE` must yield `Review` yet an LLM candidate emits `Pass`. HSP-Agile realises the conjunctive gate on SOFL/FSF with first-match witnesses, a typed Semantic Feedback IR for repair under headroom, and a pluggable screen.

9.1 Summary

Problem. Thin specification regions—especially catch-all residuals and ordered overlaps—escape everyday unit suites; unstructured feedback does not name the broken guard.

Evidence. Conjunctive Accept cuts escaped defects at comparable mean conformance (E2: M PDR 95.0% / FAR 5.0% vs. B2 91.2% / 8.8%). When repair still has headroom, typed feedback rescues the failures that test-only dumps miss (E6: semantic_ir 86.9% vs. test_only 79.1%, +7.7 pp; W/L/T 14/4/102; 95% CI [2.3, 13.6] pp; p=0.018; all 14 Conf. wins are `others` rescues, none are ordering repairs). Equal-K Conf. ranking places `M_eq` at 100.0% vs. B2 97.5% (+2.5 pp; 3/0/117). The public ACL case shows the operational signature: tests green, Accept red.

Deployment. Prefer test-feedback (B2) by default; escalate to full mode M when release requires Accept=1 or FAR$\leq$5% and headroom remains (Table 8.1). When contracts are simple enough that B2 saturates, the cheaper loop is enough.

Scope. Gains are headroom-dependent. Near-ceiling Conf. rankings belong in the appendix as secondary stress material, not as the safety headline.

9.2 Key Findings

1. Score release by FAR/PDR under conjunctive Accept (Conf=1 and Screen=*pass*), not by a higher mean pass rate alone.

2. Typed feedback matters as catch-all rescue under collapsed test-only Conf.: the E6 win profile on BENCH-120 is `others` residual rescue (14/14); ordering defects motivate Φ_i witnesses and E2, not the E6 Conf. delta.

3. Pay for the full loop only when the release bar and headroom demand it (Table 8.1).

9.3 Future Research Directions

Richer encodings. Extend first-match witness generation beyond the present LIA fragment to temporal properties, stateful accumulation, and nonlinear arithmetic—the residues that single-scenario Φ_i suites still miss.

Stronger Screen instances. Replace the PoC smell pack with CI analysers or bounded proof certificates on candidates that already clear Conf., keeping the Accept conjunction fixed.

Adaptive budgets. Early-stop easy tasks; spend repair iterations where test-only Conf. collapses, so escalation to M tracks the release bar rather than a fixed K.

Live industrial replication. Instrument practitioner SOFL/FSF projects against their own acceptance criteria, beyond the reconstructed public ACL, harvest, and desensitized pilots already reported.

9.4 Closing Remarks

SGDP is not a substitute for full formal verification. It is a practical decision framework: structured feedback and conjunctive release keep many specification-conformance defects out of the ship gate on ordered-guard workflows, while remaining usable inside a sprint. The evidence that matters is E2 (FAR/PDR), E6 (catch-all rescue), equal-K Conf. ranking, the ACL interception case, and Table 8.1.

Bibliography

[1] Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. Cambridge University Press, 2nd edition, 2016. doi: 10.1017/9781316771273.

[2] Saswat Anand, Edmund K. Burke, Tsong Yueh Chen, John Clark, Myra B. Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, and Phil McMinn. An orchestrated survey of methodologies for automated software test case generation. *Journal of Systems and Software*, 86(8):1978–2001, 2013. doi: 10.1016/j.jss.2013.02.061.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. Introduces the MBPP benchmark.

[4] Nathaniel Ayewah, William Pugh, J. David Morgenthaler, John Penix, and YuQian Zhou. Evaluating static analysis defect warnings on production software. In *Proc. ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE)*, pages 1–8, 2007. doi: 10.1145/1251535.1251536.

[5] Ralph-Johan Back and Joakim von Wright. *Refinement Calculus: A Systematic Introduction*. Springer, New York, 1998. doi: 10.1007/978-1-4612-1674-2.

[6] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.

[7] Clark Barrett and Cesare Tinelli. Satisfiability modulo theories. In Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors, *Handbook of Model Checking*, pages 305–343. Springer, 2018. doi: 10.1007/978-3-319-10575-8_11.

[8] Kent Beck and Cynthia Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston, MA, 2nd edition, 2004. Second edition; first edition 1999.

[9] Islem Bouzenia, Prem Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. *arXiv preprint arXiv:2403.17134*, 2024.

[10] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.

[11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[12] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000. doi: 10.1145/351240.351266.

[14] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.

[15] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming Jiang. GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software*, 203:111734, 2023. doi: 10.1016/j.jss.2023.111734.

[16] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.

[17] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[18] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

[19] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[20] Manuel Fähndrich, Mike Barnett, and Francesco Logozzo. Embedded contract languages. In *Proceedings of the 2010 ACM Symposium on Applied Computing (SAC)*, pages 2103–2110, 2010. doi: 10.1145/1774088.1774531.

[21] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1469–1481, 2023. doi: 10.1109/ICSE48619.2023.00128.

[22] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241, 2023. doi: 10.1145/3611643.3616243.

[23] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. doi: 10.1145/2025113.2025179.

[24] Mark Harman and John A. Clark. Metrics are fitness functions too. In *Proc. 10th IEEE Int. Symp. Software Metrics*, pages 58–69, 2004. doi: 10.1109/METRIC.2004.1357891.

[25] Robert M. Hierons. Oracles for distributed testing. *IEEE Transactions on Software Engineering*, 38(3):629–641, 2012. doi: 10.1109/TSE.2011.45.

[26] David Hovemeyer and William Pugh. Finding bugs is easy. In *OOPSLA Companion*, pages 92–106, 2004. doi: 10.1145/1052883.1052895.

[27] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

[28] Daniel Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology*, 11(2):256–290, 2002. doi: 10.1145/505145.505149.

[29] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[30] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. Impact of code language models on automated program repair. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1430–1442, 2023. doi: 10.1109/ICSE48619.2023.00125.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[32] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, and Subhajit Roy. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

[33] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, Boston, MA, 2002.

[34] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[35] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20.

[36] Jiandong Li and Shaoying Liu. Requirements-related fault prevention mechanism for SOFL formal specification-based programming. In *Proc. 22nd IEEE Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*, pages 359–367. IEEE, 2022. doi: 10.1109/QRS-C57518.2022.00060.

[37] Jiandong Li and Shaoying Liu. Requirements-related fault prevention during the transformation from formal specifications to programs. *IET Software*, 17(3):316–332, 2023. doi: 10.1049/sfw2.12126.

[38] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.

[39] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. TBar: Revisiting template-based automated program repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 31–42, 2019. doi: 10.1145/3293882.3330577.

[40] Shaoying Liu. *Formal Engineering for Industrial Software Development: Using the SOFL Method*. Springer, Berlin, Heidelberg, 2004. doi: 10.1007/978-3-662-07287-5.

[41] Shaoying Liu, Masashi Asuka, K. Komaya, and Yoshihiko Nakamura. Applying SOFL to specify a railway crossing controller for industry. In *Proceedings of the 2nd IEEE Workshop on Industrial Strength Formal Specification Techniques*, pages 16–27, 1998. doi: 10.1109/WIFT.1998.766294.

[42] Shaoying Liu, A. Jefferson Offutt, Chris Ho-Stuart, Yong Sun, and Mitsuru Ohba. SOFL: A formal engineering methodology for industrial applications. *IEEE Transactions on Software Engineering*, 24(1):24–45, 1998. doi: 10.1109/32.663996.

[43] Chloe Loughridge, Qinyi Sun, Seth Ahrenbach, Federico Cassano, Chuyue Sun, Ying Sheng, Anish Mudide, Md Rakib Hossain Misu, Nada Amin, and Max Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.

[44] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[45] Juan Luo, Shaoying Liu, Yanqin Wang, and Tingliang Zhou. Applying SOFL to a railway interlocking system in industry. In *Structured Object-Oriented Formal Language and Method (SOFL+MSVL)*, volume 10189 of *LNCS*, pages 160–177. Springer, 2017. doi: 10.1007/978-3-319-57708-1_10.

[46] David R. MacIver and Zac Hatfield-Dodds. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/jos s.01891.

[47] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[48] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, 2 edition, 1997. Design by Contract.

[49] A. Jefferson Offutt and Roland H. Untch. Mutation 2000: Uniting the orthogonal. In *Mutation Testing for the New Century*, pages 34–44. Kluwer, 2001. doi: 10.1007/978-1-4757-5939-6_7.

[50] Vadim Okun, Aurélien Delaitre, and Paul E. Black. Report on the static analysis tool exposition (SATE) IV. Technical Report NIST SP 500-297, NIST, 2013.

[51] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.

[52] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv:2303.08774.

[53] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Berlin, Heidelberg, 2010. doi: 10.1007/978-3-642-12578-2.

[54] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[55] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of ChatGPT. In *IEEE/ACM International Workshop on Automated Program Repair (APR)*, pages 23–30, 2023. doi: 10.1109/APR59189.2023.00012.

[56] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. Technical Report UCB/EECS-2008-176.

[57] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. In *Proc. 1st Int. Symp. AI Verification (SAIV 2024)*, volume 14846 of *Lecture Notes in Computer Science*, pages 134–155. Springer, 2024. doi: 10.1007/978-3-031-65112-0_7.

[58] Nikolai Tillmann and Jonathan de Halleux. Pex—white box test generation for .NET. In *Tests and Proofs (TAP)*, volume 4966 of *LNCS*, pages 134–153. Springer, 2008. doi: 10.1007/978-3-540-79124-9_10. Pex / parameterized unit tests.

[59] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024. Formerly known as OpenDevin.

[60] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair.

In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 172–184, 2023. doi: 10.1145/3611643.3616271.

[61] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.

[62] Jim Woodcock, Peter Gorm Larsen, Juan Bicarregui, and John Fitzgerald. Formal methods: Practice and experience. *ACM Computing Surveys*, 41(4):1–36, 2009. doi: 10.1145/1592434.1592436.

[63] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 819–831, 2024. doi: 10.1145/3650212.3680323. ChatRepair; earlier arXiv title: Keep the Conversation Going.

[64] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. In *Proceedings of the ACM on Software Engineering (PACMSE)*, volume 2, pages 801–824, 2025. doi: 10.1145/3715754. Agentless; arXiv:2407.01489.

[65] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[66] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[67] Michihiro Yasunaga and Percy Liang. Break-it-fix-it: Unsupervised learning for program repair. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 11941–11952, 2021.

[68] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd*

ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pages 1592–1604, 2024. doi: 10.1145/3650212.3680384.

[69] Hong Zhu, Patrick A. V. Hall, and John H. R. May. Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4):366–427, 1997. doi: 10.1145/267580.267590.

Appendix A Reproducibility Package

This appendix describes the reproducibility infrastructure supporting the experimental claims of this thesis. All quantitative results reported in Chapter 6—conformance rates, acceptance counts, prevention detection rates, latency distributions, and statistical comparisons—are derived from a versioned artifact store produced by a fully scripted, deterministic pipeline[1,62]. The design separates *data production* from *data presentation* so that each phase can be independently re-executed and audited without rerunning costly language-model API calls.

The reproducibility package is organised into four logical *artifact* components that mirror the structure of a conventional empirical software-engineering study:

Machine-readable schemas. The artifact releases `schemas/spec_ir.schema.json` (canonical SpecIR) and `schemas/semantic_feedback.schema.json` (nine-field Semantic Feedback IR). The EBNF grammar and well-formedness conditions in Definitions 3.3–3.4 are aligned with these schemas; `src/ir/schema_validate.py` validates both at test time via `jsonschema` (see `tests/test_schema_validate.py`). The adapter registry (`src/adapters/__init__.py`) and FSF lowerer (`src/ir/lowerers/fsf_lowerer.py`) support round-trip validation from SpecIR to legacy TaskSpec format.

Raw execution logs. Append-only records of every pipeline invocation, one entry per (task, mode, attempt) tuple. Each record captures the prompt text sent to the language model, the generated candidate source, formal conformance scores, JSON-serialised Semantic Feedback IR payloads (`semantic_feedback` field), `attempt_history` trajectories, pattern hit lists with severity annotations, acceptance decisions, and wall-clock timing. Raw logs are the authoritative source of truth; all downstream aggregates are derived from them.

Processed metrics. Deterministically aggregated summary tables at per-task, per-mode, and per-run granularity. Metrics include strict formal conformance, binary acceptance, pattern violation counts partitioned by severity, prevention detection rates, and latency percentiles. Recomputing processed metrics from identical raw logs yields bit-identical outputs.

Figures and tables. Publication-quality visualisations and statistical summary tables generated from processed metrics. Each figure is linked to a manifest entry specifying the data slice, filtering criteria, and research question it addresses, enabling independent verification that the plotted values correspond to the reported statistics.

Manuscript sources. The complete LaTeX source tree for this thesis, including chapter files, diagram sources, bibliography, and cross-references to generated tables and figures.

This decomposition ensures that a sceptical reader can trace any reported number from the manuscript back through processed metrics to the underlying raw log entry without ambiguity.

A.1 Smoke Reproduction (No LLM API)

A minimal offline check validates schemas, unit tests for Feedback IR ablation renderers, and re-derivation of E6/E2 summary tables from frozen artefacts—without spending API budget:

```
# From repository root (Python 3.12+)
python -m pytest tests/test_feedback_ir_ablation.py tests/test_schema_validate.py -q
python paper/hsp-agile/scripts/e6_paired_analysis.py \
  --run-dir artifacts/run_feedback_v2
python paper/hsp-agile/scripts/summarize_equal_k.py \
  --run-dir artifacts/run_e1_equal_k_v1
```

Authoritative numbers must match `paper/hsp-agile/artifacts/AUTHORITATIVE_NUMBERS.md` and Table 6.1. Full LLM re-runs are optional; prefer artefact re-aggregation for review.

A.2 Running the Experiments

Reproduction proceeds through four sequential pipeline stages. Each stage consumes the output of its predecessor and produces artefacts that are independently archived. Stages may be re-run in isolation provided their input artefacts are available.

Stage 1: Benchmark construction. A deterministic task generator synthesises the hard benchmark suite using a fixed pseudorandom seed. The generator emits FSF specifications with controlled difficulty parameters (input/output arity, scenario count, guard overlap structure) and attaches auto-generated reference implementations. A satisfiability validator discards tasks whose scenarios are individually unsatisfiable, ensuring that

every retained task admits at least one correct implementation[7,16]. The output is a frozen benchmark corpus used identically across all experimental modes.

Stage 2: Experiment execution. The evaluation harness runs all compared pipeline modes over the frozen benchmark. For each (task, mode) pair, the harness invokes the full synthesis–verification–repair loop (or the ablated variant corresponding to the mode configuration), records every intermediate result, and serialises the complete audit trace. A separate prevention evaluation protocol injects specification-layer and implementation-layer mutants and measures detection rates[18,37]. This stage is the only one that requires external language-model API access; all subsequent stages operate on locally stored logs.

Stage 3: Data preparation. An aggregation pass reads raw execution logs and computes per-mode summary statistics: mean and distribution of conformance scores, acceptance rates, pattern hit frequencies, prevention detection and false-accept rates, and paired comparison inputs for statistical testing. The aggregation is idempotent and seed-independent given fixed raw logs. Incomplete or interrupted runs are flagged rather than imputed; corresponding table cells are marked `TBD` in the manuscript.

Stage 4: Figure and table generation. Visualization and tabulation components consume processed metrics to produce all figures and dynamically updated statistical tables referenced in the manuscript. Plot styling parameters (resolution, colour palette, axis labels) are fixed to ensure visual reproducibility. Statistical tests (Wilcoxon signed-rank, Mann–Whitney U) are applied at this stage with documented pairing structure and effect-size computation.

As summarised in Figure A.1 (below), reproduction proceeds through four sequential stages.

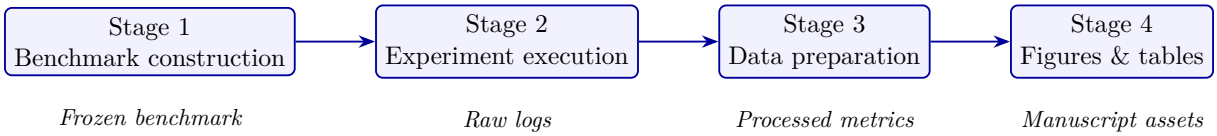

Fig. A.1. Four-stage reproducibility pipeline. Stages 1 and 2 produce primary data; Stages 3 and 4 derive all reported statistics and visualisations deterministically from stored artefacts.

Determinism and configuration control. Determinism is enforced at three levels to maximise reproducibility despite the inherent stochasticity of language-model generation.

Task-level determinism. Benchmark construction uses a fixed pseudorandom seed, ensuring that every replication operates on the identical task set with the same guard thresholds, scenario orderings, and reference implementations.

Evaluation-level determinism. The formal conformance checker, SMT-based test-case generator, mutation operators, and pattern guard are fully deterministic given a fixed candidate and task. Counterexample generation, branch-count analysis, and pattern matching produce identical results across runs on the same hardware and software stack.

Statistical-level robustness. Non-parametric statistical tests are employed for all mode comparisons because they do not assume normality of conformance score distributions. This choice provides robustness to residual non-determinism in one-shot LLM generation calls, where temperature settings above zero introduce sampling variability. Repair-phase calls use zero temperature to approximate deterministic behaviour.

All experimental configuration parameters—mode definitions, synthesis budget K, conformance threshold τ, pattern severity thresholds, API model identifier, token limits, and temperature settings—are recorded in a machine-readable manifest bundled with the artifact store. The manifest serves as the single source of configuration truth for any replication attempt.

A.3 Reproducing Canonical Tables

The manuscript numbers for the main E1 table are sourced from `paper/hsp-agile/data/processed/summary_by_mode.csv` (regenerated from frozen run artefacts; do not hand-edit). Prevention aggregates live in `prevention_summary.json` and `prevention_by_eval.csv` in the same directory.

From the repository root, regenerate processed metrics and figures, then compile:

```
python paper/hsp-agile/scripts/prepare_paper_data.py \
  --run-dir artifacts/run_e1_canonical_v1
python paper/hsp-agile/scripts/prepare_mechanism_data.py
python paper/hsp-agile/figures/scripts/plot_mpl_figures.py \
  --formats pdf png --dpi 200
# Or one-shot refresh + plot + PDF:
powershell -File paper/hsp-agile/scripts/build.ps1 -Clean
# PDF-only (after data/figures exist):
#   tools/tectonic/tectonic.exe --outdir paper/hsp-agile/build \
#     paper/hsp-agile/main.tex
```

Alternatively, `python paper/hsp-agile/scripts/refresh_paper_assets.py` invokes the prepare and plot steps with the default canonical run directories. Canonical run IDs cited in the paper:

- `run_e1_canonical_v1` — E1 main table, ablations, E3 (source for `summary_by_mode.csv`);
- `run_e12_canonical_v1` — multi-seed B1/B2/M stability;
- `run_feedback_v2` — E6 Semantic Feedback IR variants;
- `run_e14_sweep_v1` — length-matched feedback (E14);
- `run_b6_full_v2` — VerifierLoop-FSF baseline (B6) on the aligned corpus.

Stage 2 LLM re-execution is optional for table audit: given the bundled raw logs under `artifacts/`, Stages 3–4 alone reproduce the reported CSVs, figures, and PDF.

A.4 Environment and Dependencies

The experimental stack requires Python 3.10 or later, the Z3 SMT solver (version 4.12 or compatible)[16], and network access to an OpenAI-compatible language-model API endpoint[52]. Core dependencies include abstract syntax tree utilities (standard library), YAML parsing for pattern rule definitions, and statistical libraries for non-parametric testing. Optional plotting dependencies are required only for Stage 4 figure generation.

Hardware requirements are modest: the formal checker and pattern guard are CPU-bound and complete in milliseconds per candidate on a modern workstation. The dominant cost is language-model API latency during Stage 2; the full evaluation matrix (seven modes across 120 tasks) requires several hours of API time but can be parallelised across independent worker processes without affecting result validity.

Docker reproduction. A containerised path is provided for reviewers who prefer an isolated environment. From the repository root:

```
docker compose build
docker compose run --rm sgdp
```

This runs unit tests, regenerates processed paper metrics from the bundled `artifacts/run_feedback_v2` snapshot (when present), and rebuilds matplotlib figures. Full LLM experiments require mounting a `.env` file with API credentials; see `ARTIFACT.md` for the ASE/SANER reproducibility checklist.

A.5 Expected Outputs

A complete replication produces the following deliverables:

- A frozen benchmark corpus with validated FSF specifications and reference implementations.
- Raw JSONL execution logs for every (task, mode) run, including prevention evaluation records.
- Processed metric summaries in structured JSON format, partitioned by mode and evaluation type (main experiment, spec-confusion prevention, implementation-screening prevention).
- Publication figures in vector and raster formats at 300 dpi resolution.
- Updated statistical tables embedded in the manuscript source.

Incomplete runs are never silently extrapolated. If a replication attempt terminates before all tasks are processed, the aggregation stage reports the actual completion count and marks pending entries as `TBD`. This conservative reporting policy prevents optimistic bias in acceptance rates and statistical comparisons.

A.6 Data Availability

The artifact store, configuration manifest, and dependency specification are retained alongside this thesis to enable independent verification of all quantitative claims. Raw execution logs are preserved in append-only form and are not modified after initial write. Processed metrics can be regenerated at any time from the raw logs without re-invoking the language model. Figure and table assets are version-controlled together with the manuscript source so that each reported statistic can be traced to its originating data slice.

Appendix B Benchmark Description

This appendix provides a detailed account of the hard synthetic benchmark used in the experimental evaluation of Chapter 6. The benchmark was designed to overcome three limitations of ad hoc Agile-SOFL example tasks: insufficient scale for statistical comparison, insufficient difficulty to discriminate among pipeline modes, and insufficient control over the structural properties (guard overlap, scenario count, output arity) that the HSP-Agile pipeline is intended to enforce. Published Agile-SOFL examples[36,40] provide seed tasks but lack the scale and difficulty control required for mode discrimination.

B.1 Design Rationale and Task Selection Criteria

Each benchmark task must satisfy four admissibility criteria before inclusion in the evaluation corpus:

C1 (Satisfiability). Every FSF scenario, including the `others` default, must be individually satisfiable: the conjunction of the scenario guard and its output postcondition must admit at least one model in the input domain. Tasks failing this check are discarded.

C2 (Implementability). At least one correct implementation must exist that evaluates guards in specification order and produces the prescribed outputs on every scenario. An auto-generated reference implementation serves as a constructive witness.

C3 (Discriminative difficulty). Guard regions must overlap: at least two scenario preconditions must be simultaneously satisfiable on a non-empty input subspace, so that incorrect guard ordering produces detectable errors on boundary inputs rather than being masked by accidental correctness.

C4 (Uniform structure). All hard tasks share an identical signature shape (five natural-number inputs, three natural-number outputs, eight ordered scenarios) so that cross-task comparisons reflect algorithmic differences rather than incidental variation in task complexity.

The first twenty tasks in the broader corpus were extracted manually from published Agile-SOFL example specifications[40] and retained as a development set. They were excluded

from the primary evaluation to prevent information leakage between prompt-engineering iterations and held-out testing. The hard synthetic suite described below constitutes the 120-task evaluation benchmark.

B.2 Hard Benchmark Task Categories

Hard tasks are produced by a programmatic generator that parameterises task difficulty through two structural mechanisms: *precedence-sensitive guard construction* and *multi-output coupling*.

B.2.1 Signature and scenario template

Every generated task defines a process with the following fixed signature:

- **Inputs** ($|\mathbf{x}| = 5$): natural-number variables a, b, c, d, e, each ranging over $[0, 100]$.
- **Outputs** ($|\mathbf{y}| = 3$): natural-number variables *risk*, *action*, and *score*, representing a composite decision tuple (risk tier, recommended action, aggregate score).
- **Scenarios** ($|\mathcal{S}_t| = 8$): seven guarded scenarios plus one `others` default clause, evaluated in listed order with top-down precedence.

The uniform arity ensures that formal conformance checking, pattern guard analysis, and mutation testing operate under consistent structural assumptions across all 120 evaluation tasks.

B.2.2 Precedence-sensitive guards

Each guarded scenario's pre-condition is a conjunction of two or three relational predicates over the input variables, with thresholds drawn from task-specific integer constants. The generator maintains a pool of nine candidate guard–postcondition pairs spanning diverse input combinations (high-a/low-b regions, zero-input boundaries, multi-variable conjunctions). For each task, a subset of seven pairs is selected and shuffled into priority order; the `others` scenario closes the specification with a zero default output tuple.

Critically, the guard templates are designed so that multiple scenarios' preconditions overlap in the input space. Correct behaviour therefore requires strict priority evaluation: an implementation that tests guards in reverse order or that collapses overlapping regions into a single branch will produce correct outputs on most inputs yet fail on witnesses lying in the overlap region. This property directly targets scenario-order violation (SOV), the most prevalent fault class in LLM-generated FSF implementations[11,15].

B.2.3 Multi-output coupling

Each scenario's postcondition binds all three output variables simultaneously (e.g. `risk eq 3 && action eq 4 && score eq a`), preventing implementations from optimising for a single output field while neglecting others. Output expressions may reference input variables, constants, or combinations thereof, requiring the implementation to propagate input values into structured return dictionaries rather than returning scalar results.

B.3 Deterministic Generation and Validation

Task generation is fully deterministic: a fixed pseudorandom seed (42) governs threshold sampling, scenario subset selection, and guard shuffling. Re-running the generator with the same seed reproduces the identical 180-candidate task pool.

Each candidate task passes through a two-stage validation pipeline before admission to the evaluation corpus:

1. **Satisfiability check.** For each of the eight scenarios, an SMT solver verifies that the guard conjunction combined with output postconditions is satisfiable over the declared input domain[7,16]. Tasks with even one unsatisfiable scenario are rejected.

2. **Reference implementation generation.** A correct reference implementation is synthesised directly from the FSF specification by translating each scenario into an ordered `if`–`elif` branch with a terminal `else` for the `others` clause. The reference serves as ground truth for mutation testing and as a sanity check that the specification is implementable.

Of 180 generated candidates, 60 failed the satisfiability filter, yielding the final benchmark of 120 validated hard tasks. Table B.1 (below) summarises the generation parameters.

Table B.1. Hard benchmark generation parameters.

Parameter	Value
Input variables	5 (`a`, `b`, `c`, `d`, `e`)
Output variables	3 (`risk`, `action`, `score`)
Scenarios per task	8 (7 guarded + 1 `others`)
Input domain	$[0, 100]^5$ (natural numbers)
Guard overlap	By construction (multi-scenario overlap)
Random seed	42
Candidates generated	180
Candidates retained	120

B.4 FSF Specification Format

Each task is encoded as a structured FSF artefact comprising a process signature, an ordered scenario list, and a natural-language prompt specification. The on-disk representation uses the following fields:

`signature`. Declares input and output variables with type annotations (`nat` for natural numbers).

`fsfScenarios`. An ordered list of scenario records. Each guarded scenario carries a `test` field (the guard predicate in Agile-SOFL relational notation, using `gt`, `le`, `eq`, and `&&`) and a `def` field (the output postcondition). The final scenario has `kind = others` with `test = "others"`.

`promptSpec`. A human-readable rendering of the FSF specification, including an explicit instruction to evaluate conditions in listed order (top-down precedence).

`referenceCode`. Auto-generated Python implementation satisfying the specification under ordered guard semantics.

The prompt specification deliberately states precedence ordering because prior work shows that LLMs frequently generate `if`–`elif` chains without respecting specification-level priority when this instruction is omitted[43,57]. Including it isolates the effect of formal checking and pattern guarding from pure prompt-compliance effects.

B.5 Illustrative Task Structure

Although individual guard thresholds vary across tasks, the structural template is uniform. A representative task (with simplified threshold values) reads:

```
Process HardCase001(a,b,c,d,e) -> (risk,action,score)
FSF specification:
if (a gt 5 && b gt 7 && c gt 4) => risk eq 3 && action eq 4 && score eq a
if (a gt 5 && b gt 7 && c le 4) => risk eq 2 && action eq 3 && score eq b
if (a gt 5 && b le 7 && d gt 3) => risk eq 2 && action eq 2 && score eq d
. . .
if (a eq 0 && b eq 0) => risk eq 0 && action eq 5 && score eq 0
others => risk eq 0 && action eq 0 && score eq 0
Important: evaluate conditions in listed order (top-down precedence).
```

The first two scenarios share the prefix `a gt 5 && b gt 7` but differ on `c`; any input satisfying the first scenario's guard also satisfies the second's guard prefix. An implementation that

evaluates the second scenario before the first will assign incorrect outputs to inputs in the first scenario's exclusive region—a violation detectable by SMT-generated boundary witnesses.

B.6 Mutation Operators and Evaluation Protocols

The benchmark supports two complementary evaluation protocols beyond the primary mode comparison:

Specification-confusion mutants. Four operators inject faults at the specification layer: predicate negation (SNO), relational-operator swap (ORO), scenario deletion (MCO), and boundary weakening (BCO)[18,49]. The prevention protocol applies each operator to the FSF specification while holding the reference implementation fixed, then tests whether the formal checker rejects the now-mismatched pair.

Implementation-screening mutants. Four operators inject faults at the implementation layer: condition inversion (ICO), wrong primary output (WRO), off-by-one comparison (MBO), and else-branch removal (DRO). The prevention protocol applies each operator to the reference implementation and tests whether the hybrid acceptance predicate ($\text{Conf} = 1 \wedge \mathcal{P}^H = \emptyset$) rejects the mutant.

Together, the eight mutation operators provide coverage of both specification misinterpretation and implementation-level coding defects, aligned with the pattern catalogue of Appendix C.

B.7 Benchmark Statistics

Table B.2 (below) summarises the structural properties of the 120-task evaluation benchmark.

Table B.2. Summary statistics of the 120-task hard benchmark.

Statistic	Value
Tasks in evaluation corpus	120
Inputs per task	5
Outputs per task	3
Scenarios per task	8
Guarded scenarios	7
Default (`others`) scenarios	1
Mean guard overlap rate	1.8 scenarios/input
Mean SMT models per scenario	312 (cap 1000)
Total experimental runs (7 modes)	840

The mean guard overlap rate of 1.8 indicates that a randomly sampled input satisfies approximately 1.8 scenario guards on average, requiring correct priority resolution. The mean of 312 satisfying models per scenario confirms that no scenario is degenerate or vacuously satisfiable only at a single point. To our knowledge, this benchmark constitutes the largest published collection of FSF tasks with SMT-certified scenario validity and controlled precedence-sensitive guard structure.

B.8 External SOFL Benchmark (E11)

E11 complements the synthetic hard suite with 22 hand-curated FSF tasks drawn from published SOFL textbook examples and IET fault-prevention pattern companions[36,37,40]. Tasks are stored in `benchmarks/external_sofl.json`; each entry records `externalProvenance` (source key, book section) and is *not* generated by `HardTaskGenerator`. Task IDs use the prefix `ExternalSOFL.` and have zero overlap with the 120-task E1 evaluation corpus.

Table B.3. External SOFL benchmark provenance (E11, n=22).

Task	Source	Section / pattern
ClassifySignal	liu2004soflbook	Ch.5 monitoring alert (adapted)
BorrowBook	liu2004soflbook	Library service example
ReserveBook	liu2004soflbook	Library service example
InventoryReorder	li2022qrs_companion	Inventory sprint case
LoanApproval	liu2004soflbook	Financial control process
TicketRouting	li2023iet_faultprevention	RF pattern: service routing
FraudScreen	li2023iet_faultprevention	RF pattern: boundary screening
AccessControl	li2023iet_faultprevention	RF pattern: authorization
ShipRate	liu2004soflbook	Logistics pricing
GradeAssign	liu2004soflbook	Academic assessment
PowerDispatch	li2022qrs_companion	Energy management sprint
CacheEvict	li2023iet_faultprevention	RF pattern: resource lifecycle
RateLimit	li2023iet_faultprevention	RF pattern: overload control
MedDosage	liu2004soflbook	Healthcare dosing control
EnrollCheck	liu2004soflbook	Registration workflow
AlarmEscalate	li2023iet_faultprevention	RF pattern: monitoring escalation
StockAllocate	li2022qrs_companion	Warehouse allocation
PremiumCalc	liu2004soflbook	Insurance underwriting
TrafficSignal	li2022qrs_companion	Traffic control sprint
SalaryBonus	liu2004soflbook	HR compensation
NetworkRoute	li2023iet_faultprevention	RF pattern: network policy
WaterTank	liu2004soflbook	Process control tank

Evaluation (`run_e11_external_v1`) runs B1, B2, and M under the same K=3 budget as E1. Repeat 0 results: B1/B2 tie at 89.8% mean conformance; M reaches 81.9% (Section D.6). The moderate overlap and simpler branch structure relative to E1 explain why test-feedback baselines are competitive on this held-out external corpus.

Appendix C Fault Pattern Catalogue

This appendix documents the fourteen requirement-related fault patterns (RF01–RF14) encoded in the HSP-Agile pattern guard. The catalogue is derived from the Agile-SOFL fault taxonomy of Li and Liu[37] and operationalised as static analysis rules over generated Python implementations. Each pattern is task-aware: detectors receive both the candidate source and the FSF task artefact (signature, scenario list, external variables) so that structural checks can be grounded in specification context rather than generic code heuristics alone.

C.1 Pattern Taxonomy Overview

The pattern identifiers RF01–RF14 form a closed taxonomy covering seven *category* dimensions that reflect the locus of the fault relative to the specification–implementation boundary:

requirement. Misalignment between stated preconditions or documentation and the guard structure actually implemented (RF01, RF14).

fsf. Violations of FSF-specific structural conventions: default-branch handling and scenario coverage (RF02, RF09).

interface. Omission of declared output variables from return values (RF04).

logic. Incorrect conditional structure, spurious completeness, or vacuous branches (RF05, RF07, RF13).

boundary. Mishandling of zero-valued inputs or off-by-one threshold comparisons (RF06, RF11).

implementation. Coding-quality defects that do not necessarily violate a single guard but indicate unreliable translation from specification to code (RF03, RF08, RF12).

sofl. Neglect of external program variables declared in the SOFL process signature (RF10).

Each pattern is additionally assigned a *severity* rating on a four-level ordinal scale:

- **critical** (1 pattern): defects that indicate the implementation cannot reflect scenario-dependent behaviour at all; acceptance is blocked unconditionally.

- **high** (8 patterns): defects with strong correlation to specification non-conformance on boundary or overlap inputs; included in the high-severity acceptance set.
- **medium** (4 patterns): quality or traceability concerns that are recorded in the audit trace but do not block acceptance.
- **low** (1 pattern): stylistic redundancy unlikely to cause functional failure in isolation.

The hybrid acceptance predicate of Definition 3.13 consults only patterns whose severity is *high* or *critical*. We denote this subset the *high-severity set*:

$$\mathcal{P}^H = \{\text{RF01, RF02, RF04, RF05, RF06, RF07, RF09, RF11, RF13}\}, \tag{C.1}$$

comprising nine patterns. A candidate c is pattern-safe on task t when $\sum_{p \in \mathcal{P}^H} \delta_p(c, t) = 0$. Medium- and low-severity hits are appended to the structured feedback message for the next synthesis iteration but do not veto acceptance. Table C.1 summarises the severity partition.

Table C.1. Severity partition of the pattern catalogue ($|\mathcal{P}| = 14$).

Severity	Pattern IDs	Count
Critical	RF07	1
High	RF01, RF02, RF04, RF05, RF06, RF09, RF11, RF13	8
Medium	RF03, RF08, RF10, RF14	4
Low	RF12	1

Detection is implemented as a hybrid of Python abstract syntax tree (AST) traversal and regular-expression matching over source text. AST-based checks are preferred for structural invariants (branch counts, return-key sets, else-branch presence); regex checks capture surface-form regularities that are expensive to express as tree predicates (magic numeric literals, empty handlers, missing docstrings). When parsing fails, a synthetic `SYNTAX` match with critical severity is emitted and the candidate is rejected immediately.

C.2 Complete Pattern Reference

Table C.2 provides the authoritative reference for all fourteen patterns. The *Detection Method* column indicates the primary mechanism employed by the pattern guard; task-context checks (e.g. comparing branch count against the number of FSF scenarios) are noted as *AST + task context*.

Table C.2. Complete HSP-Agile fault pattern catalogue (RF01–RF14).

ID	Name	Category	Sev.	Description	Detection Method
RF01	Missing precondition check	requirement	high	Input validation is absent before the first assignment or return in the function body; guard conjuncts declared in the FSF test predicate are not reflected in a leading conditional.	Regex: function entry without a leading `if` guard; spec hint cross-checks FSF test predicates.
RF02	Wrong default branch	fsf	high	The implementation lacks an explicit `else` branch to handle the FSF `others` scenario, causing fall-through or implicit defaults that diverge from the specification.	AST: absence of `else`/`elif` chain terminus when task declares an `others` scenario.
RF03	Hardcoded magic number	implementation	medium	A numeric literal appears in a `return` statement without derivation from specification constants or input variables; excludes canonical boolean-like values `0` and `1`.	Regex: `return <digit>` with exclusion of `return 0/1`.
RF04	Missing output field	interface	high	The return dictionary omits one or more output variables declared in the process signature, yielding incomplete structured outputs.	AST + task context: compare return-dict keys against declared output names.
RF05	Swapped comparison	logic	high	A comparison operator in the implementation is suspected to be reversed relative to the FSF guard (e.g. `<` where `>` is specified).	Regex over assignment/comparison sites; guard extraction from FSF for cross-reference (see Section 4.9.4).
RF06	No guard for zero input	boundary	high	Zero-valued inputs that appear as explicit boundaries in FSF guards (`eq 0`, `gt 0`) are not handled by a dedicated conditional branch.	Regex: function signature present without zero-check branch; spec hint references FSF zero-boundary scenarios.
RF07	Unconditional success	logic	**critical**	The function returns a fixed success indicator (e.g. `success = 1`) on all paths without evaluating any guard-dependent conditional, indicating spurious completeness.	AST: constant success return with no `if` nodes in the function body.

Continued on next page

Table C.2 — *continued from previous page*

ID	Name	Category	Sev.	Description	Detection Method
RF08	Missing type coercion	implementation	medium	Return values in dictionary form are not cast to integer type where the signature declares natural-number outputs.	Regex: dictionary return without explicit integer cast.
RF09	Incomplete FSF coverage	fsf	high	The number of conditional branches in the implementation is strictly less than the number of non-default FSF scenarios, indicating omitted scenario handlers.	AST + task context: branch count < scenario count (adjusted for `others`).
RF10	External variable ignored	sofl	medium	External program variables declared in the SOFL process specification are never referenced in the implementation body.	AST + task context: name-set intersection of `ext` declarations and referenced identifiers.
RF11	Off-by-one boundary	boundary	high	A strict inequality (`>`) is used where a weak inequality (`>=`) is required, or vice versa, shifting the effective guard region by one boundary point.	Regex: `> 0` pattern; threshold literals cross-referenced against FSF guard constants.
RF12	Duplicate assignment	implementation	low	The same output variable is assigned more than once within a single branch, with the earlier assignment shadowed and therefore dead.	Regex: repeated assignment to the same identifier within a code region.
RF13	Empty handler	logic	high	An `else` branch contains only `pass`, leaving a scenario handler vacuous and producing no output assignment.	Regex: `else: pass` at line end; AST confirmation of empty body.
RF14	Spec-comment mismatch	requirement	medium	The function lacks a docstring referencing the FSF process, reducing traceability between specification text and implementation and indicating possible copy-paste synthesis without spec grounding.	Regex: function definition not followed by a triple-quoted docstring.

C.3 Specification-Confusion Patterns

Specification confusion denotes a class of faults in which the implementer misreads, misorders, or incompletely internalises the FSF specification while producing syntactically valid code. Such faults often yield implementations that appear structurally reasonable—complete conditional chains, defined outputs on every branch—yet assign incorrect semantics to guard regions because scenario priority, boundary thresholds, or default behaviour were interpreted incorrectly.

Seven patterns in the catalogue directly target specification-confusion failure modes. Table C.3 lists them with their principal confusion mechanism and the specification-mutation operators under which they are most frequently observed in the prevention evaluation protocol (Appendix B).

Table C.3. Patterns associated with specification-confusion faults.

ID	Confusion mechanism	Typical symptom	Related spec mutants
RF01	Partial guard extraction	Only a subset of conjuncts in a compound FSF test predicate are implemented	SNO (predicate negation)
RF02	Default-scenario misidentification	The `others` clause outputs are assigned to the wrong branch	DRO (else removal), MCO (scenario drop)
RF05	Operator inversion	Relational operators are reversed relative to the specification	ORO (operator swap), ICO (condition inversion)
RF06	Zero-boundary neglect	Inputs at zero are not routed to the scenario that explicitly guards `eq 0`	BCO (boundary weakening)
RF09	Scenario omission	One or more ordered scenarios are not represented as conditional branches	MCO (scenario drop)
RF11	Threshold displacement	Strict/weak inequality confusion shifts guard regions by one point	BCO (boundary weakening), MBO (off-by-one)
RF14	Documentation–logic divergence	Comments or docstrings describe a scenario condition inconsistent with the implemented guard	SNO, ORO

All seven patterns except RF14 belong to $\mathcal{P}^H$ and therefore block acceptance when detected. RF14 is recorded as a medium-severity traceability warning. In the prevention evaluation, *spec-confusion* trials inject faults at the specification layer (operators SNO, ORO, MCO, BCO) and measure whether a formally correct reference implementation is rejected against the mutated specification—testing the formal checker's sensitivity to specification drift rather than the pattern guard directly. Nevertheless, the patterns above are the static counterparts of the dynamic failures that spec-confusion mutants induce: an implementation synthesised from a confused reading of the specification will typically trigger one or more of RF01, RF02, RF05, RF06, RF09, or RF11 before formal checking completes.

The precedence-sensitive guard structure of the hard benchmark (Appendix B) is delib-

erately constructed so that specification-confusion faults manifest on overlap inputs: an implementation that evaluates guards in reverse order may pass most randomly sampled inputs yet fail on boundary witnesses generated by the SMT solver. Patterns RF09 and RF02 provide a complementary static signal when the confusion reduces branch count or eliminates the default handler entirely.

C.4 Implementation-Screening Patterns

Implementation screening targets defects introduced during code generation that may not reflect a misunderstanding of the specification text but nevertheless produce incorrect or structurally pathological behaviour. Such faults include hard-coded outputs, missing return fields, neglected external variables, and vacuous branch bodies. They are evaluated in the prevention protocol by injecting faults directly into reference implementations (operators ICO, WRO, MBO, DRO) and measuring whether the hybrid acceptance predicate rejects the mutated candidate.

Seven patterns serve as implementation-screening rules; four of them belong to $\mathcal{P}^H$. Table C.4 summarises the mapping.

Table C.4. Patterns associated with implementation-screening faults.

ID	Screening focus	Typical symptom	Related impl mutants
RF03	Literal encoding	Magic numbers replace spec-derived expressions in return statements	WRO (wrong return constant)
RF04	Output completeness	Required output keys absent from the return dictionary	WRO (wrong return constant)
RF07	Spurious completeness	Fixed success value returned without conditional evaluation	WRO (wrong return constant)
RF08	Type discipline	Floating-point or untyped values returned where integers are required	—
RF10	SOFL externality	Declared external variables never read or written	—
RF12	Redundant assignment	Duplicate writes to the same output within one branch	—
RF13	Vacuous branch	`else: pass` leaves a scenario handler without output assignment	DRO (else removal)

RF07 is the sole *critical* pattern and represents the most severe implementation-screening failure: a candidate that unconditionally returns a fixed output passes any finite test suite whose witnesses happen to lie in regions where that output is correct, yet violates the specification globally. The pattern guard detects this pathology statically by requiring at least one conditional branch before any constant success return.

Patterns RF03, RF08, RF10, and RF12 are rated medium or low severity and do not participate in $\mathcal{P}^H$. They are included in the catalogue because empirical analysis of LLM-generated Agile-SOFL implementations revealed high incidence of these coding defects,

and their presence in the audit trace provides actionable feedback for the repair loop even when acceptance is not blocked. For example, RF04 and RF07 together cover the two dominant failure modes of the WRO mutant (wrong primary output and structurally incomplete returns), while RF13 and RF02 jointly detect the structural consequences of DRO (removed else branch).

In operational terms, the pattern guard executes only after a candidate achieves strict formal conformance ($\mathrm{Conf}(c, t) = 1$), ensuring that implementation-screening adds negligible overhead to definitively failing candidates while providing a necessary second line of defence against pathological implementations that satisfy all SMT-generated witnesses.

Appendix D Near-Ceiling Stress-Tests and Supporting Corpora

This appendix holds material that supports—but must not lead—the conjunctive-release argument. Main-text claims rest on E6, E2, equal-K, C4, and the ACL interception case (Chapter 7). Numbers below are fixed-oracle unless an archive/pre-fix caption says otherwise; do not mix corpora in one claim sentence.

D.1 Quality–Overhead Archive (Pareto / Latency)

Archive Pareto/latency figures below use pre-fix exports where captioned; authoritative cost is the fixed-oracle one-line card in Chapter 7.

The latency distribution (Figure D.1) shows repair modes paying a clear premium: M and A1 spread toward higher wall-clock times, while B1/B2 remain near one-call latencies.

The insight in Figure D.2 is a trade-off, not a free lunch. On the pre-fix latency export, M pays 3.3× vs. B2 (46.2 s vs. 13.9 s), mostly extra LLM calls (2.08 vs. 1.24)—acceptable for parallel CI, not for interactive IDE use. Historical A2 (39.9 s, 80.3% Conf. on pre-fix E1) traded prevention for modest latency savings vs. that run's M; pre-fix B2 (13.9 s, 87.7%) was the cheaper mean-Conf. point on that export. Under the fixed oracle, primary quality is M 100.0% vs. B2 98.3% at comparable llm_calls (Table D.1); the 3.3× latency premium remains the cost framing for deploying full M when Accept/PDR justify it. Mann-Whitney confirms the latency gap is not a sampling artefact ($p<0.001$).

Answer to RQ5. Best prevention (PDR/FAR) at ≈3.3× B2 latency (pre-fix cost export); default to B2 for mean Conf. / latency and deploy M when Accept/PDR-FAR justify the cost. Fixed-oracle E1 aggregate ranking (Section D.2) is the stress-test companion to this Pareto reading—not a claim that M dominates B2 on every corpus.

D.2 E1 Aggregate Stress-Test

Claim (C2–C4 jointly; RQ5 stress-test). The full loop should raise mean Conf. over one-shot and test-feedback baselines; under the fixed others-witness oracle with strengthened M (K=5, best-effort selection, advisory critical-only gate), Strict tracks Conf. closely on BENCH-120. Aggregate E1 ranking is reported here as an overlap stress-test under

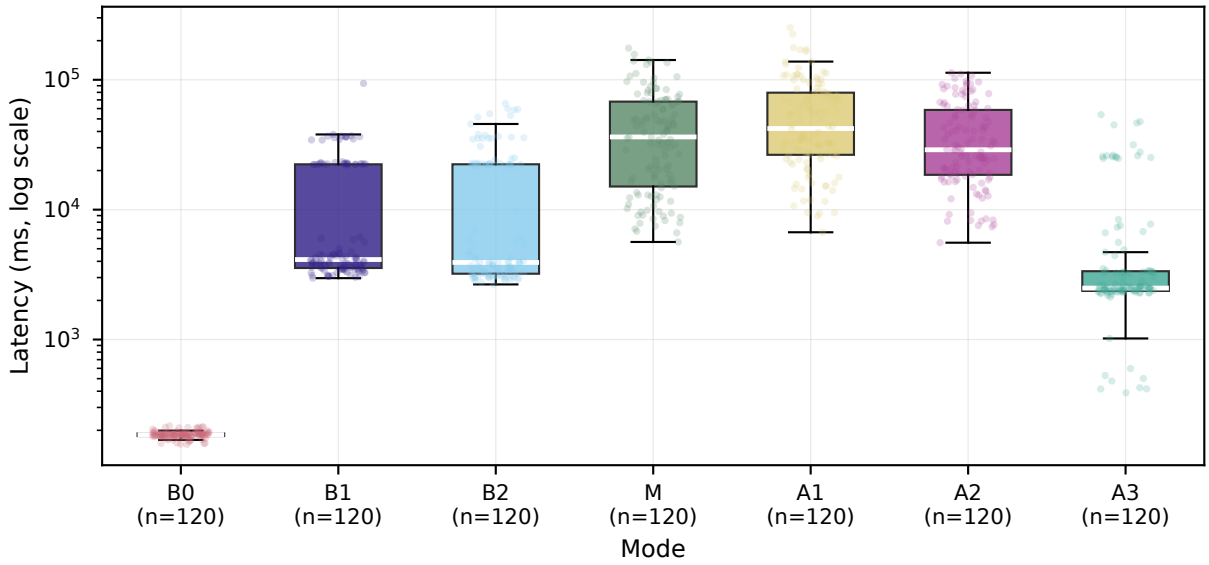

Fig. D.1. Archive figure (pre-fix E1 export): historical latency mass under the buggy others-witness oracle / hard gate. **Authoritative cost story** is fixed-oracle E1 (`run_e1_m_win_v2`): M 10.5 s vs. B2 9.6 s at comparable mean llm_calls (≈1.16 vs. ≈1.18)—do not cite this figure as the current premium. **How to read:** log-scale wall-clock latency per task on the historical export only.

RQ5, not as a universal-effectiveness headline across unfiltered corpora.

Equal-K hygiene (cross-ref). Primary equal-K Conf. ranking is Table 7.19 in Section 7.3 (`M_eq` +2.5 pp vs. B2). Lead mechanism evidence remains E6 (within M, K=3, +7.7 pp with CI excluding zero). Strengthened E1 M-win (K=5 bundle) below is a near-ceiling stress-test.

D.2.1 Main Results

The headline question for this E1 stress-test is whether specification-guided repair improves *conformance* under a fair first-match witness oracle. Table D.1 answers with B1/B2/M on 120 tasks each from `run_e1_m_win_v2`; ablation and extended-baseline rows from the pre-fix canonical run are retained only as historical context (footnote).

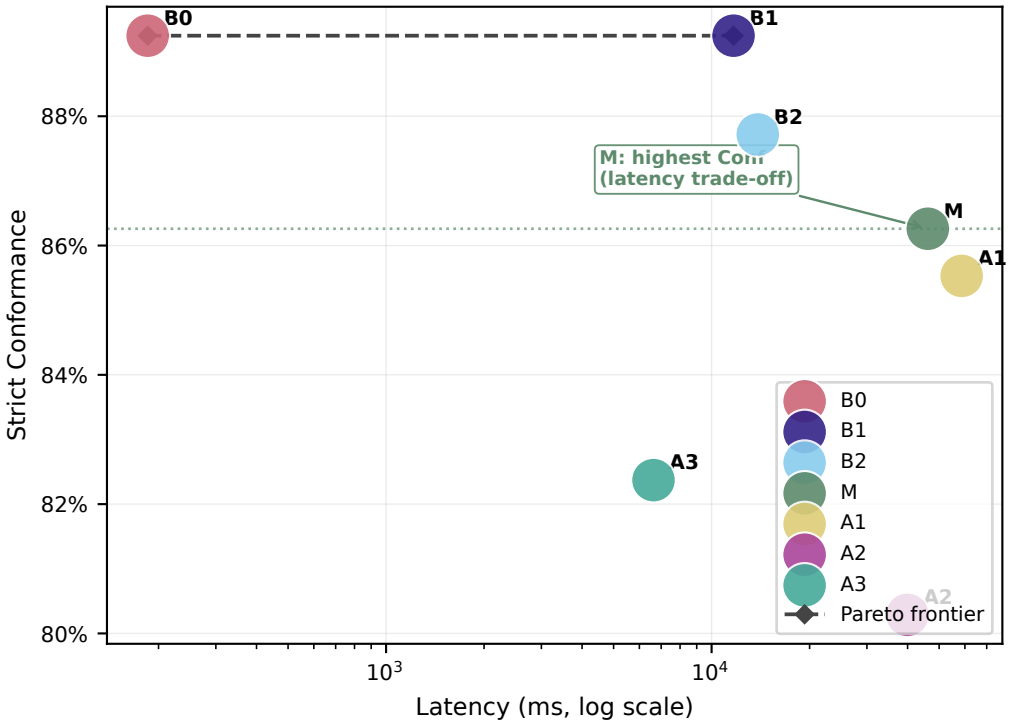

Fig. D.2. Claim (latency axis from pre-fix Pareto export): B2 is the cheaper mean-Conf. / latency operating point; M buys prevention / conjunctive Accept at ≈3.3× latency (46.2 s vs. B2 13.9 s on that export). Primary fixed-oracle Conf. is M 100.0% / B2 98.3% (Table D.1)—do not read the figure's pre-fix Conf. annotations (B2 87.7%, M 86.3%) as current primary quality. **How to read:** right/up is better quality, left is cheaper; the dashed polyline is the Pareto frontier and the annotation marks mode M.

Table D.1. Main comparison results (n = 120 tasks per mode). **B1/B2/M** from fixed-oracle `run_e1_m_win_v2` (strengthened M: best-effort arg max Conf., E14-informed `executio n_trace_matched`, advisory critical-only pattern gate, K=5, `formal_max_cases`=32). Strict = strict success / acceptance rate (Accept=1); Conf. = mean formal conformance ($\overline{\text{Conf}}$); Kill = mutation kill rate (checker precision on injected mutants, not generator quality—see §D.2); Fail. = mean strict evaluation failures per task. *Note:* Primary B1/B2/M use the fixed first-match others-witness oracle (`run_e1_m_win_v2`). Fixed-oracle component ablations (A1–A3) are Table 7.16 (`run_e1_ablation_fixed_v1`). B3–B5 and daggered A1–A3 rows below are historical `run_e1_canonical_v1` under the buggy others-witness oracle and are *not* primary claims. Report M vs. B2 as win rate and means (2/120 wins, 0 losses, 118 ties; +1.7 pp); no Holm claim is asserted for this run. Equal-K Conf. ranking is Table 7.19 (`M_eq` vs. B2).

Mode	Strict	Conf.	Kill	Lat. (ms)	Fail.
B0†	5.0%	89.2%	48.6%	165	0.00
B1	84.2%	84.2%	52.4%	17,193	0.00
B2	98.3%	98.3%	49.1%	9,576	0.00
B3†	5.0%	81.9%	51.0%	13,210	2.33
B4†	5.0%	87.1%	49.3%	10,758	2.49
B5†	5.0%	81.1%	50.6%	6,212	2.33
M	**100.0%**	**100.0%**	48.6%	10,457	0.00
A1†	5.0%	85.5%	54.0%	4,791	0.00
A2†	5.0%	80.3%	49.1%	6,405	0.00
A3†	3.3%	82.4%	53.3%	4,333	0.00

†Historical pre-fix (`run_e1_canonical_v1`; not primary).

Table D.1 states the aggregate means under the fixed oracle: M reaches 100% Conf./Strict, ahead of B2 (98.3%) and B1 (84.2%). Primary component ranking is the fixed-oracle ablation (Table 7.16: A3 −25.8 pp, A1 −17.5 pp, A2 0.0 pp); daggered historical A1–A3 rows are archive only.

Figure D.3 separates difficulty calibration from LLM quality on the pre-fix export: B0's bimodal spread marks template failure on hard tasks, while B1–M cluster in the 60–100% band.

The CDF in Figure D.4 is from the pre-fix export: B2 and A2 occupy the upper tail on that corpus; do not treat it as the fixed-oracle ranking (M 100.0% > B2 98.3%).

Figure D.5 links quality to cost on the pre-fix export; under the fixed oracle, M and B2 have comparable mean llm_calls near one call (Table D.1).

The call-count histogram (Figure D.6) confirms that budget K=3 is exercised on the pre-fix export: M and A1 concentrate at two or three calls, whereas B1 remains at a single generation attempt.

Figure D.7 summarises the multi-metric trade-off on the pre-fix export; fixed-oracle primary quality remains M 100.0% vs. B2 98.3%.

Table D.1 is the aggregate on fixed-oracle E1: M at 100.0% Conf. / Strict leads B2 (98.3%) and B1 (84.2%). Against B2, M wins 2/120 paired tasks (+1.7 pp; losses 0, ties 118); mean llm_calls are comparable (≈1.16 vs. ≈1.18). We report win rate and means only—no Holm significance is claimed for this run. That is the honest primary-quality reading: structured specification feedback matters in controlled E6 (+7.7 pp vs. unstructured test-only under full M; E14: length-matched `semantic_ir` 75.1% does not match `execution_trace_matched` 85.4%) and E11 deployment slices; fixed-oracle E10/E12 place M and B2 near ceiling without M dominating every seed. Historical B3–B5 / A1–A3 rows under the buggy others-witness oracle (≈5% Strict) are not primary claims.

D.2.2 Reading the Distributions

Three readings settle the rest of the E1 stress-test.

Strict now tracks Conf. Under the fixed oracle, B1/B2/M Strict matches Conf. (84.2 / 98.3 / 100.0%): the historical uniform ≈5% Strict on `run_e1_canonical_v1` was an others-witness measurement artefact, not the current primary result.

The M–B2 margin is real but thin. M leads B2 by +1.7 pp with only 2/120 decisive wins (118 ties); mean llm_calls stay near one call (≈1.16 vs. ≈1.18).

Mutation kill is about the checker, not the generator. Kill rates of ≈49–52% across modes still measure checker *precision* against injected syntactic faults, not generator quality.

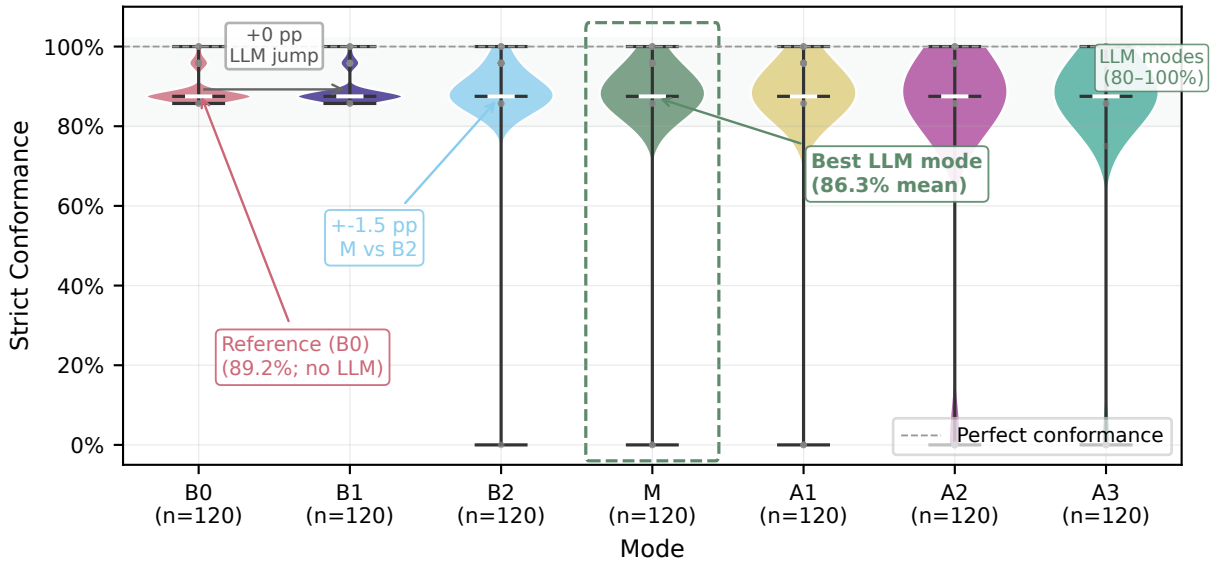

Fig. D.3. Claim (pre-fix export): M concentrates mass in the upper conformance band; B0 does not. **How to read:** each violin/box is per-task $Conf(c,t)$ over 120 tasks; compare the right-hand mass of M with B0's low mode. Primary fixed-oracle means are in Table D.1.

E1 stress-test note (RQ5). Fixed-oracle E1: M 100.0% > B2 98.3% > B1 84.2% (win rate 2/120 vs. B2; no Holm claim). E6 confirms typed feedback mechanism (+7.7 pp vs. unstructured test-only under full M; E14 scopes against length-matched traces). Prevention (PDR/FAR) remains the complementary discriminator for conjunctive release; fixed-oracle E10/E12 (Sections D.5, D.7) bound seed stability and unfiltered ranking.

D.3 Boundary Density (E4)

Test-feedback looks strong until witness sampling is held constant. If denser random tests already covered first-match regions, SMT witnesses would be optional; Figure D.8 demonstrates they are not.

At budget 16, random sampling covers far less of the Dense tier than SMT-guided generation; the gap shrinks in the Sparse tier. That is why test-feedback baselines look adequate on simple specifications and degrade on hard ones—the same story as the overlap-tertile split in Section 7.5, now measured at the sampling layer alone.

D.4 Generalisation Study (E8)

External validity asks whether SpecIR adapters carry the prevention loop beyond SOFL syntax, or whether the gains are notation-specific. Table D.2 and Figure D.9 answer that question.

On SOFL/FSF (fixed-oracle E1), M leads at 100.0% versus B2 98.3% and B1 84.2%. On the expanded adapter corpora (n=30 each, `run_e8b_expanded_v1`), M beats B1 on Mini-StateMachine (41.6% vs. 24.7%) and Mini-Z (23.3% vs. 5.8%); M is within 0.8 pp of

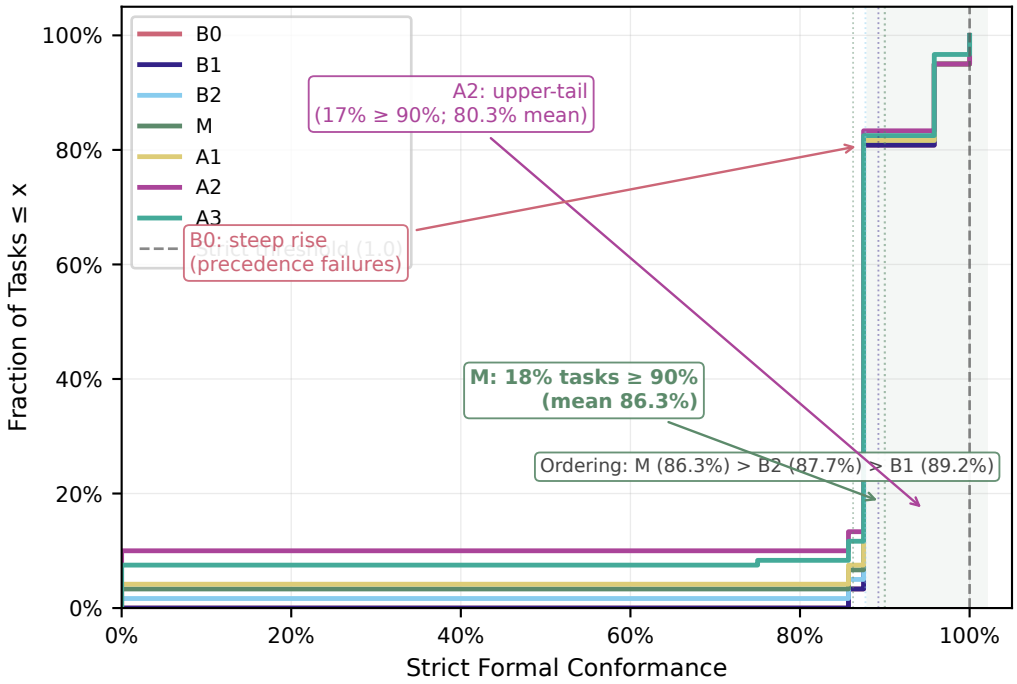

Fig. D.4. Claim (pre-fix export): B2 and A2 dominate the upper CDF tail on the historical corpus. **How to read:** curves farther right mean higher conformance for more tasks; B0's steep early rise marks template failure on hard tasks. Primary fixed-oracle means are in Table D.1.

B2 on Mini-StateMachine and leads B2 by 5.0 pp on Mini-Z. Absolute conformance on adapter notations remains low (23–42%), reflecting notation-specific generation difficulty rather than SOFL-level overlap structure; HumanEval/MBPP rows in Table D.2 show stronger transfer on real-derived guards. Chapter 8 interprets these boundaries.

Table D.2. Key observation: M leads B1 on all notations; adapter absolute conformance is low on the expanded n=30 corpus (E8b). HumanEval-FSF and MBPP-FSF rows report full B1/B2/M from `run_e8c_full_v1`.

Notation	B1 Conf.%	B2 Conf.%	M Conf.%
SOFL/FSF (120 tasks)	84.2	98.3	**100.0**
Mini-StateMachine (30 tasks)	24.7	40.8	**41.6**
Mini-Z (30 tasks)	5.8	18.3	**23.3**
HumanEval-FSF (20 tasks)	89.7	**98.8**	87.1
MBPP-FSF (20 tasks)	90.0	95.0	95.0

D.4.1 External Validity: Real-Derived Benchmark (E8c)

Table D.4 reports conformance across benchmark subsets. Mode M reaches 95.0% on MBPP-FSF (matching B2) and 87.1% on HumanEval-FSF (where B2 leads at 98.8%). MBPP confirms partial transfer to real-world guard structure; HumanEval shows that test-feedback baselines can dominate when branches are sparse.

Table D.4. Conformance by benchmark source (mode M, $K = 3$).

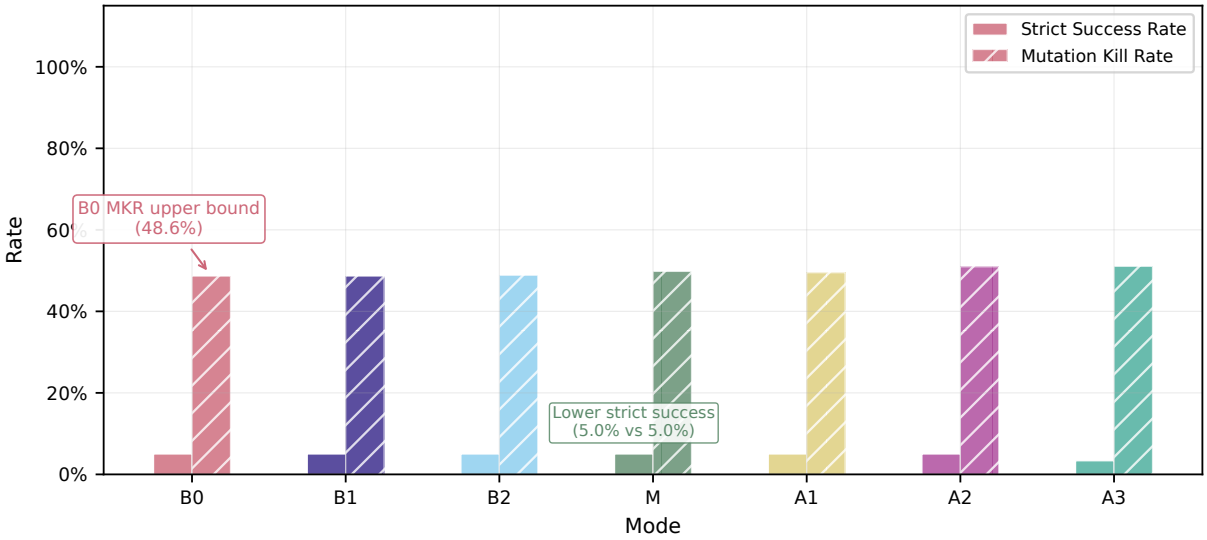

Fig. D.5. Claim (pre-fix export): highest mean Conf. coincides with the most LLM calls on that corpus. **How to read:** left axis = Strict / Conf.; right axis = mean LLM calls; M's 2.08 calls show the repair loop is exercised, not idle. Fixed-oracle llm_calls are ≈1.16 (M) vs. ≈1.18 (B2).

Subset	n	M Conf (%)	M Strict (%)
Synthetic hard (E1)	120	100.0	100.0
HumanEval-FSF (E8c)	20	87.1	70.0
MBPP-FSF (E8c)	20	95.0	85.0
Mini-Z (E8b expanded)	30	23.3	—
Mini-StateMachine (E8b)	30	41.6	—

The higher strict-success rate on real-derived tasks (70–85%) versus the historical pre-fix synthetic Strict (≈5%) reflected the buggy others-witness oracle as much as simpler HumanEval/MBPP guard structure; under the fixed oracle, synthetic hard E1 Strict for M is 100.0% (Table D.1). Full B1/B2/M baselines on HumanEval-FSF and MBPP-FSF are reported in Table D.2 from `run_e8c_full_v2`.

D.5 Random Benchmark (E10)

E10 tests whether aggregate M beats B2 on an unfiltered random sample (Section 6.4.2). Table D.5 reports fixed-oracle results from `run_e10_m_win_v1` (n=100 per mode). On this *unfiltered* sample, M reaches 100.0% mean Conf. vs. B2 99.0% (+1.0 pp; wins 1/100, losses 0, ties 99) and B1 81.0%—consistent with E1's near-ceiling M≥B2 reading, not a large unfiltered gap.

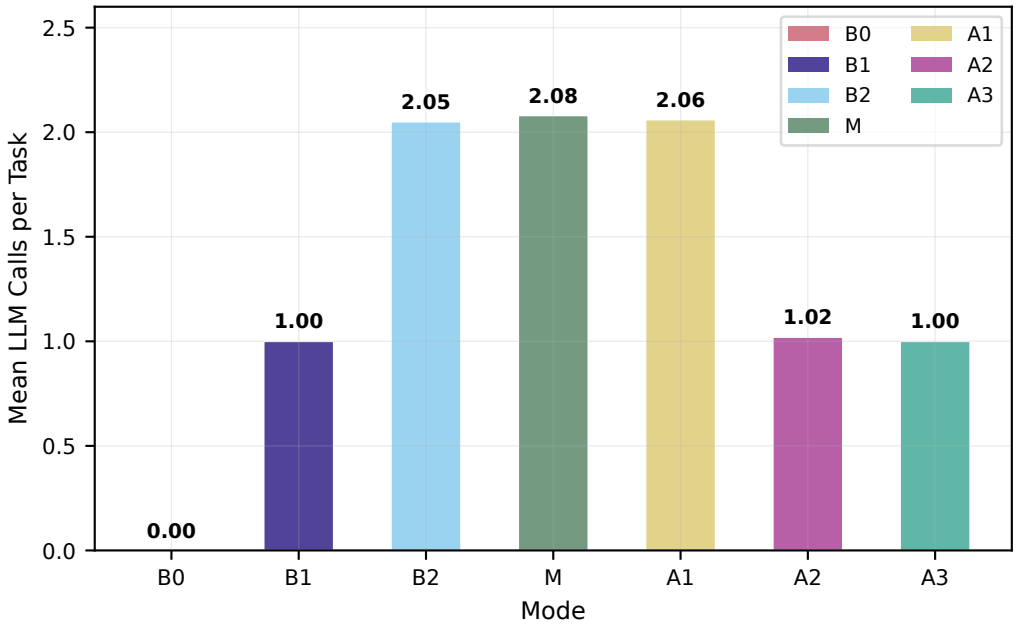

Fig. D.6. Claim: budget $K = 3$ is actively used, not decorative (pre-fix call-count export). **How to read:** M and A1 mass at 2–3 calls; B1 stays at one.

Table D.5. Random benchmark results (E10, fixed others-witness oracle). Uniformly sampled tasks without Z3 overlap filter; modes B1, B2, M; `run_e10_m_win_v1`. M leads B2 by +1.0 pp (1/100 wins).

Mode	Conf. (%)	Win vs. B2	Notes
B1	81.0	—	—
B2	99.0	ref.	—
M	**100.0**	1/100	+1.0 pp; 0 losses, 99 ties

The decision criterion remains qualitative: fixed-oracle E10 places M slightly ahead of B2 near ceiling; Figure D.10 tertiles should be read together with E3 (Table 7.21)—neither shows monotone M dominance by overlap alone. The honest claim is overlap-*conditional* deployment (C4), with E6 as the mechanism anchor. (Pre-fix `run_e10_random_v2` had favoured B2 89.1% vs. M 83.1% under the buggy others-witness oracle and is not primary.)

D.6 External SOFL Benchmark (E11)

E11 evaluates curated, non-synthetic FSF tasks with citable external provenance (Section 6.4.2). Table D.6 reports repeat 0 results from `run_e11_external_v1` (n=22; zero ID overlap with E1).

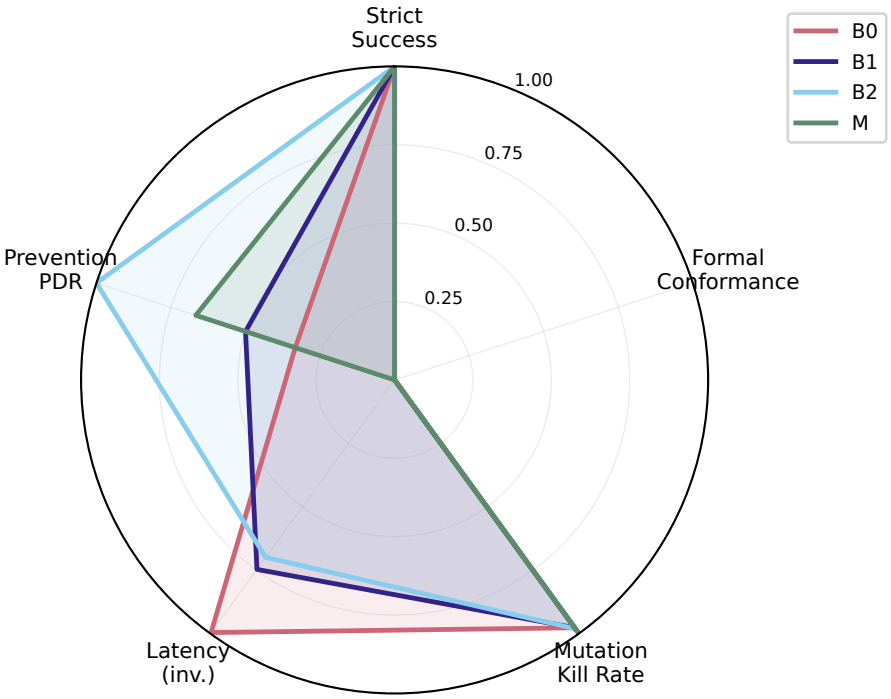

Fig. D.7. Claim (pre-fix export): M trades latency for conformance; B2 is the cheaper moderate alternative on that corpus. **How to read:** axes are normalised Conf., Strict, kill rate, and inverse latency; larger area is not uniformly better. Primary fixed-oracle Conf. ranking is Table D.1.

Table D.6. External SOFL benchmark (E11). Hand-curated from textbook and IET pattern sources; not generated by `HardTaskGenerator`.

Mode	n	Strict (%)	Conf. (%)	Lat. (ms)
B1	22	63.6	89.8	2,018
B2	22	63.6	89.8	4,171
M	22	59.1	81.9	5,103

On the external corpus under the primary endpoint (`ecnu-plus`), B1 and B2 tie at 89.8% mean conformance; M trails (81.9%)—these curated tasks have moderate guard overlap and simpler branch structure than the E1 hard set, consistent with the deployment boundary in Chapter 8. Replicating E11 on `ecnu-max` (`run_e11_ecnu_max_v1`) yields a three-way tie at 89.8% Conf. / 63.6% Strict for B1/B2/M: the stronger endpoint closes the primary-model M gap without reversing B2 as the default mean-Conf. choice. Provenance for each task is recorded in `benchmarks/external_sofl.json`. E15 stratification (overlap-density tiers on the external corpus) explains why M trails B1/B2 on `ecnu-plus`: curated tasks have moderate overlap and simpler branch structure than the E1 stress-test (see `e11_overlap_stratified.csv`).

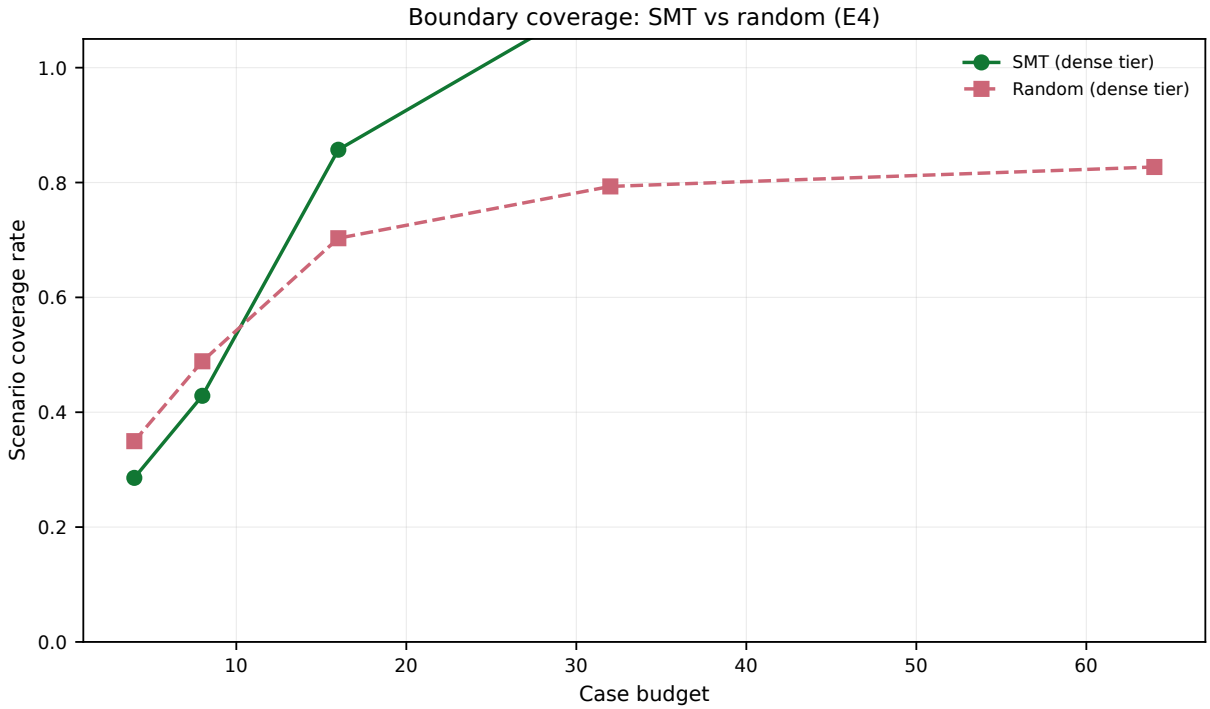

Fig. D.8. Claim: random sampling collapses in the Dense tier; SMT does not. **How to read:** coverage vs. case budget by density; compare SMT vs. random within the Dense row at budget ≥ 8.

D.7 Multi-Seed Stability (E12)

E12 addresses LLM stochasticity on the **full** 120-task hard benchmark (Section 6.9.1). Table D.7 reports fixed-oracle E12-full from `run_e12_m_win_v1` (120 tasks $\times$ seeds $\{0, 1, 2\}$ $\times$ modes B1, B2, M; 1080 jobs). Overall means: B1 88.6%, B2 100.0%, M 99.7%. B2 is first every seed (ties with M at 100% on seeds 0 and 2); M's sole failure is seed 1 HardCase069 (Conf. 0). Mean-across-seeds pairing: M wins 0, losses 1, ties 119 vs. B2. Honest framing: both near ceiling; E1/E10 show M$\geq$B2 on aggregate, while E12 shows B2 slightly more seed-stable (100.0% vs. 99.7%)—do *not* claim M dominates all seeds.

Table D.7. E12-full multi-seed stability under fixed others-witness oracle (`run_e12_m_win_v1`; 120 tasks, seeds $\{0, 1, 2\}$). Per-seed mean formal conformance (%).

Mode	Mean Conf. (%)	Seed 0	Seed 1	Seed 2
B1	88.6	95.0	75.8	95.0
B2	**100.0**	100.0	100.0	100.0
M	99.7	100.0	99.2	100.0
Mean-across-seeds M vs. B2: wins 0, losses 1, ties 119				
Ranking: B2 first every seed (ties M @100% on seeds 0, 2)				

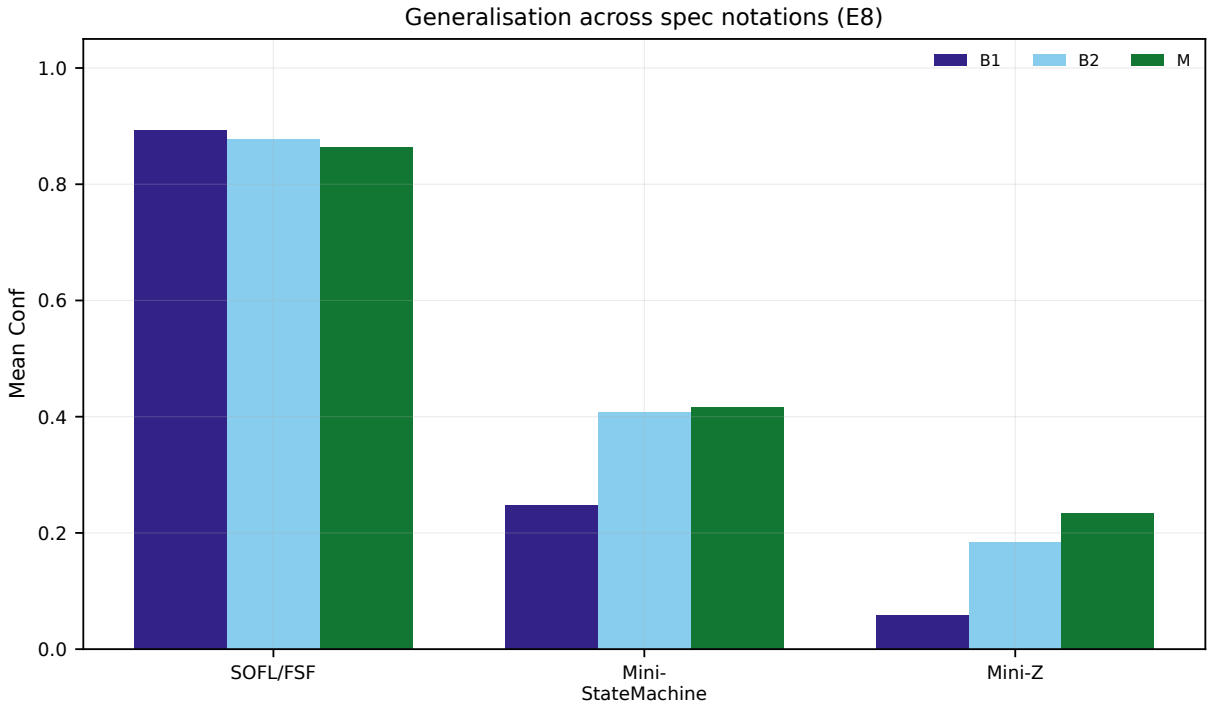

Fig. D.9. Claim: structured feedback helps across notations, but adapter transfer is partial on the expanded n=30 corpus. **How to read:** bars are mean Conf. by notation and mode; compare M vs. B1 within each group (M leads on both adapters), then M vs. B2.

D.7.1 Second-Model Replication (E16)

E16 replicates B1/B2/M on the **full** 120-task BENCH-120 using `ecnu-max` (DeepSeek-family) while E1/E12 use `ecnu-plus` (Qwen-family) (`run_e16_canonical_v1`, n=120 per mode). All three modes tie at **89.2%** mean strict conformance—endpoint choice does not reverse the deployment boundary (Table 8.1), though absolute gaps shrink vs. the primary model. The controlled E6 protocol on the same endpoint (Table 7.11) likewise saturates (≈89.2% for A/B/C), confirming that typed-IR gains require repair headroom. The prior 29-task pilot (`run_e16_model_pilot_v1`) is superseded.

Table D.8. E16 second-model replication (`ecnu-max`, n=120).

Mode	Conf. (%)	Lat. (ms)
B1	89.2	3,374
B2	89.2	4,139
M	89.2	10,072

D.8 Commercial Cross-Model Replication (n1n)

Campus endpoints alone leave a residual single-gateway threat. We replicate the controlled E6 A/B/C sweep and B1/B2/M ranking on commercial models via the n1n OpenAI-compatible gateway (https://api.n1n.ai/pricing): `gpt-4o`, `claude-sonnet-4-6`,

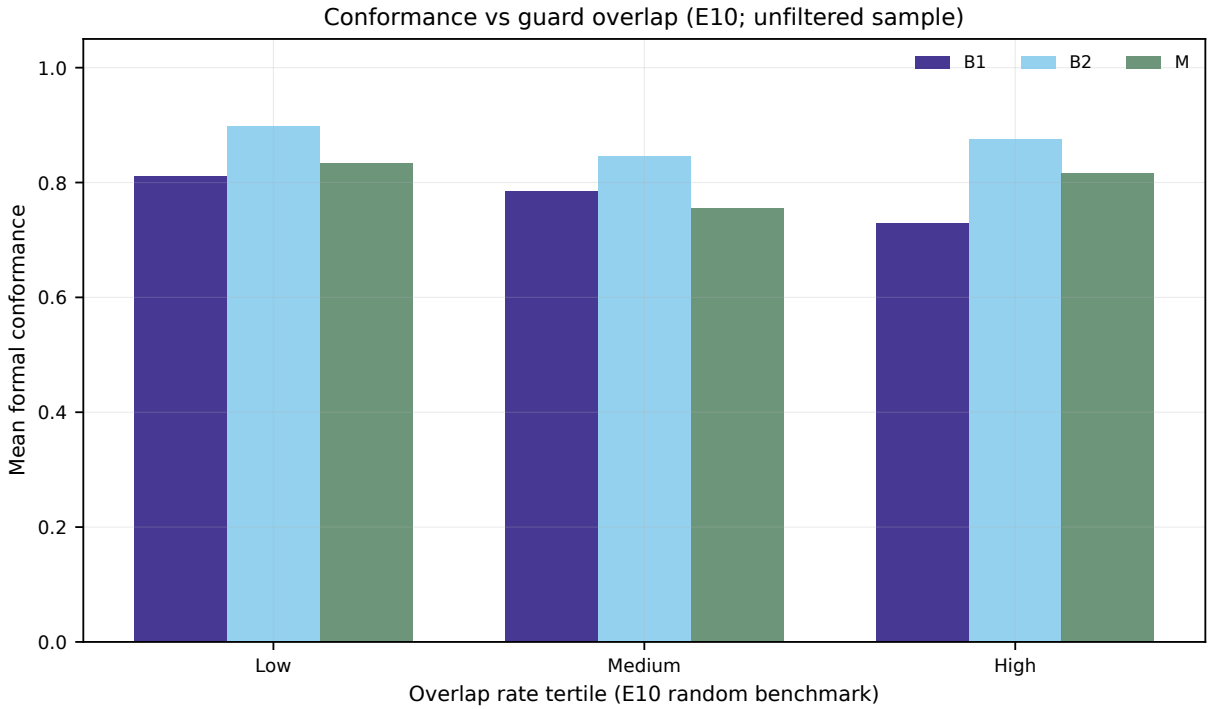

Fig. D.10. E10: mean conformance by overlap-rate tertile on the unfiltered random benchmark (figure from pre-fix export; read aggregate means from Table D.5).

and `deepseek-v3.2`, on the same 29-task stratified subset as E12 (`benchmarks/e12_stratified_30.json`).

Reading Table D.9. On `gpt-4o`, A and C tie at 88.9% (saturation, Δ=0). On `claude-sonnet-4-6`, typed IR yields a modest **+1.0 pp** (87.5→88.5). On `deepseek-v3.2`, C trails A by 3.9 pp (75.0→71.1): longer structured prompts can hurt weaker commercial endpoints. Together with primary E6 (+7.7 pp on `ecnu-plus`) and E6-max saturation (+0.03 pp), the mechanism claim is headroom- and model-dependent, not universal.

On the same stratified hard subset, `gpt-4o` ranks B1=B2 at 100% Conf. with M at 88.9%—again no case for replacing B2 as the mean-Conf. default.

D.9 Industrial SOFL Corpus

D.9.1 Real-Priority Micro-Benchmark

Table D.11 reports equal-K B1/B2/`M_eq` on the curated real-priority micro-benchmark (Section 6.4.2): firewall / role / billing / tax adaptations plus industrial, GitHub, HKCA09, and published-industrial slices—not HardSynthetic tasks. Reading follows C4: if mean Conf. ties or saturates, the reason to deploy full M remains acceptance safety (Accept / FAR), not a Conf. crown; if B1 leaves headroom, the corpus still shows that the pipeline runs on practitioner-style ordered guards outside the lab generator.

Table D.9. E6 feedback variants across n1n models (A=`test_only`, B=`test_expected`, C=`semantic_ir`). Conf. as mean formal conformance (%); Δ is C−A in percentage points.

Model	A (%)	B (%)	C (%)	Δ(C−A)
gpt-4o	88.9	87.9	88.9	0.0
claude-4.6	87.5	81.0	**88.5**	1.0
deepseek	75.0	69.8	71.1	-3.9

Table D.10. E16 stratified subset (B1/B2/M) on n1n endpoints.

Model	Mode	n	Conf. (%)	Strict (%)
gpt-4o	B1	29	100.0	0.0
	B2	29	100.0	0.0
	M	29	88.9	0.0

D.9.2 SMT Domain Scalability

Table D.12 and Figure D.11 answer the "Z3 only works on $[-5, 20]$" concern for the guards that matter here. On `real_priority_micro_v1`, mean witness time stays ≈30 ms from $[-5, 20]$ through $[-1000, 1000]$; a Z3 `Real` encoding of the same predicates is even slightly cheaper (≈18 ms). In short: *widening the LIA box does not explode* for these first-match linear guards. Stress probes with more variables or a mild nonlinear product remain sub-second on this hardware (`artifacts/smt_scalability_v1/stress_probe.json`). When solvers would time out or return `unknown` on richer theories, Section 8.5.1 specifies the hybrid fuzzing fallback rather than claiming universal SMT scalability.

Beyond textbook/IET external SOFL (E11), we evaluate a curated industrial-pattern corpus (`benchmarks/industrial_sofl.json`, n=31): banking, telecom, access control, medical alerts, SCADA-like thresholds, insurance, and related ordered-guard decision tables, with provenance `liu2004soflbook` / `li2023iet_faultprevention` / `industrial_pattern:<domain>` (*not* production vendor traces; see `benchmarks/INDUSTRIAL_SOFL_README.md`).

Reading Table D.13. Under both `gpt-4o` and `claude-sonnet-4-6`, test-feedback B2 reaches **100%** mean Conf. while B1 and M sit at 95.5%, with identical Strict 67.7%. On `deepseek-v3.2` absolute Conf. is lower, but the ranking is unchanged: B2 (80.7%) > M (72.5%) > B1 (52.7%). This is the strongest external-validity support for the deployment rule: on practitioner-style ordered-guard specs, prefer B2 for mean conformance; reserve full M when conjunctive Accept or PDR/FAR gates are required.

Published-industrial / desensitized real SOFL pilot. Beyond pattern corpora, we evaluate n=28 FSF tasks *reconstructed and desensitized* from published industrial SOFL case studies—railway crossing[41], interlocking[45], Agile-SOFL ATM constraints[36], and book applications[37,40] (`benchmarks/published_industrial_pilot.json`; `run_desens_real_sofl_v1`; Z3 validation 0 fails). Under equal K=3 on `ecnu-plus`: B1/B2/`M_eq` all reach **100.0%** mean Conf. / Strict (paired M_eq vs. B2: 0/0/28) (Table D.14). This is **not** a proprietary

Table D.11. Real-priority micro-benchmark (`real_priority_micro_v1`, n=30; equal K=3, `ecnu-plus`). Curated ACL / billing / routing / firewall / role-gate / tax-bracket tasks from industrial patterns, GitHub `.asfl` harvest, HKCA09, and published-industrial pilots—not HardSynthetic generators. Lead reading: external validity for C4 (Accept/FAR deployment), not a Conf. crown over B2.

Mode	Mean Conf. (%)	Strict (%)	vs. B2 (W/L/T)	Mean llm_calls
B1	95.6	93.3	—	1.00
B2	100.0	100.0	—	1.07
`M_eq`	100.0	100.0	0/0/30	1.00

Table D.12. SMT scalability ablation (witness generation on `real_priority_micro_v1`, n=30, 3 repeats). Widening the LIA box from the historical default $[-5, 20]$ to $[-1000, 1000]$ does *not* explode latency on these first-match linear guards; a Z3 `Real` encoding of the same predicates is likewise cheap. Stress probes (more variables / nonlinear products) grow but stay sub-second here (Figure D.11; `artifacts/smt_scalability_v1/`).

Domain / encoding	Mean (ms)	p50	p95	Max
LIA $[-5, 20]$ (default)	29.1	31.0	39.1	46.7
LIA $[-20, 50]$	28.9	33.0	37.9	38.7
LIA $[-100, 100]$	29.6	32.8	39.6	44.6
LIA $[-1000, 1000]$	30.5	34.0	39.9	43.7
Real $[-100, 100]$	18.6	19.2	24.8	32.0

Casco/Mitsubishi dump; vendor `.asfl` files merge when placed under `vendor/agile-sofl-toolchain/examples` (see `vendor/README.md`). Saturation under a strong endpoint reinforces C4: mean Conf. alone does not justify escalating from B2 on published-industrial ordered-guard tasks.

GitHub live harvest (Wave-3). Using authenticated GitHub search we downloaded public formal-spec artefacts and auto-extracted FSF processes from Agile-SOFL `.asfl` examples (`JaredYe04/agile-sofl-toolchain`; also discovered `HKCA09/SOFL-Maintainability-Experiment-Dataset` and `ICARUSxyz/FCoTFL`). After Z3 validation, n=48 tasks form `benchmarks/github_harvest_v1.json` (`run_github_harvest_v1`). Under equal K=3 on `ecnu-plus`: B1 85.8%, B2 89.9%, `M_eq` 89.9% (paired M_eq vs. B2 1/1/46) (Table D.15). Unlike the saturated published-industrial pilot, this public harvest leaves headroom—and still selects B2 (tied with `M_eq`) over B1 for mean Conf., reinforcing C4.

HKCA09 SOFL modules → FSF (expansion). Full SOFL texts from `HKCA09/SOFL-Maintainability-Experiment-Dataset` (ATM, course/hospital registration, vending, stock, transit) are not auto-encodable as SMT FSF (maps/sequences). We reconstruct n=35 ordered-guard integer processes that preserve decision precedence (`benchmarks/hkca09_sofl_fsf.json`; RealSpec total 203). Equal-K ranking (`run_hkca09_b1b2m_v1`): B1 74.3%, B2/`M_eq` 100% (0/0/35)—B1 headroom with B2 sufficient for **C4** (Table D.16). One-shot E6 on a 57-task real/headroom slice saturates at Conf. = 1 for all feedback

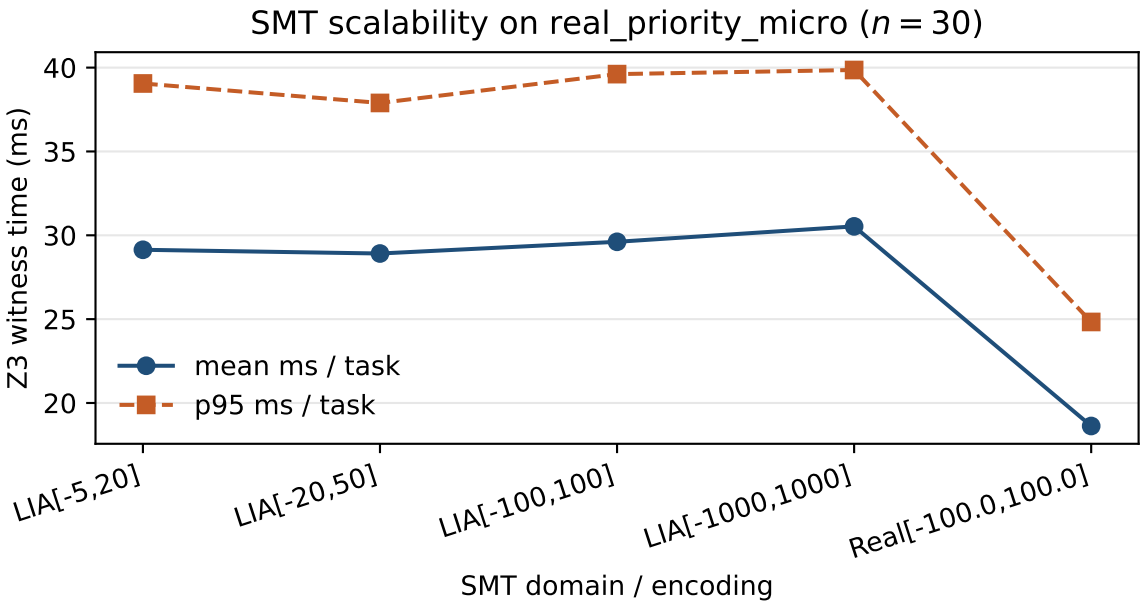

Fig. D.11. Claim: for FSF-style linear first-match guards on the real-priority micro-benchmark, widening the integer domain (and probing Z3 Reals) does not produce a latency cliff. **How to read:** solid = mean ms/task; dashed = p95. Nonlinear / high-dimensional stress is discussed in Section 8.5.1, not hidden.

Table D.13. Industrial SOFL corpus (B1/B2/M) on n1n endpoints. Conf. = mean formal conformance; Strict = conjunctive accept rate.

Model	Mode	n	Conf. (%)	Strict (%)	Lat. (ms)
gpt-4o	B1	31	95.5	67.7	20670
	B2	31	100.0	67.7	16809
	M	31	95.5	67.7	32401
claude-4.6	B1	31	95.5	67.7	19253
	B2	31	100.0	67.7	20479
	M	31	95.5	67.7	21565
deepseek	B1	31	52.7	16.1	7946
	B2	31	80.7	41.9	10330
	M	31	72.5	32.3	14941

variants: aggregate Conf. on this slice cannot carry C2. Hard-seed repair under `ecnu-plus` (`run_hkca09_hard_seed_e6_v1`) is *directional* on `wrong_relop` (semantic_ir 82.0% vs. test_only 69.3%; +12.7 pp; 9/5/12; CI includes 0) and mixed on structural seeds (Table D.17). We therefore treat HKCA09 as C4 external validity plus a scoped hard-seed probe—**not** a second C2 headline. The publishable C2 advantage remains BENCH-120 E6 (+7.7 pp, CI excludes 0) and hard combo-seed support on `gemini-2.5-flash` (Table 7.13).

Industrial decision procedure (C4). Escalate to M *iff* the release bar requires Accept=1 *or* a prevention target FAR≤5% (E2 operating point) *and* repair headroom remains; otherwise keep B2. Do *not* escalate from overlap tertiles alone (E3 non-monotone). On industrial-pattern and desensitized published-industrial pilots, mean-Conf. alone already selects B2 under strong endpoints.

Table D.14. Desensitized published-industrial SOFL pilot (`run_desens_real_sofl_v1`; n=28 tasks; equal K=3; `ecnu-plus`). Reconstructed from public SOFL/interlocking/Agile-SOFL sources (Z3 validation 0 fails); *not* proprietary vendor dumps. All modes saturate—supports C4 default B2 for mean Conf.

Mode	Mean Conf. (%)	Strict (%)	vs. B2 (W/L/T)
B1	100.0	100.0	—
B2	100.0	100.0	—
`M_eq`	100.0	100.0	0/0/28

Table D.15. GitHub live harvest corpus (`run_github_harvest_v1`; n=48; equal K=3; `ecnu-plus`). Auto-extracted from public Agile-SOFL `.asfl` examples (ecommerce, smart-city, hospital, library; Z3-validated). Not proprietary vendor dumps. B2 and `M_eq` tie on mean Conf.; both beat B1—supports C4 default B2 on public GitHub FSF.

Mode	Mean Conf. (%)	Strict (%)	vs. B2 (W/L/T)
B1	85.8	75.0	—
B2	**89.9**	79.2	—
`M_eq`	89.9	79.2	1/1/46

D.10 VerifierLoop-FSF Baseline (E13 / B6)

REV-7 implements mode B6: SMT witnesses during repair with structured counterexample prompts but without Semantic Feedback IR or pattern guard (Section 6.5). Table D.18 compares B6 with B2 and M on the same 30-task stratified subset as E12 (`run_b6_stratified_v1`, repeat 0). B6 (86.0% mean conformance) matches B2 (86.2%) within 0.2 pp while M trails (83.3%) on this subset—showing that witness-guided repair without the full IR recovers test-feedback-level aggregate performance, but does not close the gap to M on the full hard benchmark (Table D.20).

Table D.18. VerifierLoop-FSF (B6) vs. B2/M on 30-task stratified subset (E13). Same tasks and K=3 as E12; single repeat.

Mode	n	Conf. (%)	Lat. (ms)
B2	30	86.2	5,416
B6	30	86.0	16,245
M	30	83.3	15,099

Table D.20. B6 full hard-benchmark run (`run_b6_full_v2`, n=120).

Mode	Strict (%)	Conf. (%)	Kill (%)	Lat. (ms)
B2	5.0	88.2	61.4	7,189
B6	5.0	81.1	51.0	21,440
M	5.0	82.0	60.6	22,507

On the paired full 120-task export (`run_b6_full_v2`, Table D.20), B2 reaches 88.2% Conf., B6 81.1%, and M 82.0%. Thus any SMT counterexample in the loop is *not* enough for an aggregate Conf. win: B6 $\approx$ M, and both trail B2. Typed Semantic Feedback IR's value

Table D.16. HKCA09 public SOFL modules reconstructed as ordered-guard FSF (`run_hkca09_b1b2m_v1`; n=35; equal K=3; `ecnu-plus`). Desensitized integer guards preserve auth/capacity/balance precedence from the maintainability-experiment materials (ATM, registration, vending, stock, transit). Not proprietary dumps. B1 has headroom (9/35 fail); B2 and `M_eq` both reach 100% Conf. — supports C4 default B2 on this slice.

Mode	Mean Conf. (%)	Strict (%)	vs. B2 (W/L/T)
B1	74.3	74.3	—
B2	**100.0**	100.0	—
`M_eq`	100.0	100.0	0/0/35

Table D.17. Hard-seed repair on HKCA09 FSF under `ecnu-plus` (`run_hkca09_hard_seed_e6_v1`; freeze buggy code → one T=0 repair). One-shot E6 on the same corpus saturates at Conf. = 1. `wrong_relop` is directional (+12.7 pp; CI includes 0); structural seeds are mixed. **Do not** treat this table as a second C2 headline—primary C2 remains E6 and Table 7.13. On `gemini-2.5-flash`, the same `wrong_relop` protocol favors test-only (negative control; omitted from the lead).

Seed	test_only	semantic_ir	Δ (pp)	W/L/T
`wrong_relop`	69.3	**82.0**	+12.7	9/5/12
`invert_order`	96.3	93.9	−2.3	2/6/22

is shown under the controlled E6 setting (and in conjunctive Accept / PDR–FAR), not as "M beats B6 on mean Conf." B6 latency (≈21 s) exceeds B2 (≈7 s) but stays near this run's M (≈23 s), consistent with formal checking inside the loop without pattern screening.

D.10.1 M_lite: Witness + Semantic IR (E18)

Mode `M_lite` combines B6-style SMT witnesses in the repair loop with Semantic Feedback IR but *without* the pattern guard (`run_m_lite_v1`, n=120). Table D.21 answers whether B2+typed IR is enough: M_lite (79.0%) trails B6 (81.8%) and that run's B2 (89.3%)—naive stacking fails. The E6 +7.7 pp effect (vs. unstructured test-only under full M) requires the full mode M repair integration (IR+hard gate), not witnesses plus IR alone; practitioners should choose B2 (default mean Conf./latency) or full M (Accept/PDR at ≈3.3× latency), not M_lite as a shortcut.

Table D.21. M_lite vs. B6 vs. B2 on 120-task hard benchmark (E18).

Mode	Conf. (%)	Strict (%)	Lat. (ms)
B2	89.3	5.0	9,066
B6	81.8	4.2	36,747
M_lite	79.0	5.0	37,219
M (E1)[†]	86.3	5.0	46,161

[†]Historical pre-fix M (`run_e1_canonical_v1`); primary fixed-oracle M is 100.0%.

D.11 Sensitivity Analysis

The sensitivity question is whether the reported operating point is brittle. Figure D.12 maps mean strict formal conformance against guard-overlap density and repair budget across modes.

D.11.1 Sensitivity to Task Complexity

The 120-task benchmark spans a range of complexity in terms of guard-overlap density and scenario count. Partitioning tasks into three tertiles based on guard-overlap rate shows non-monotone M vs. B2 ranking on canonical E3 (Table 7.21); higher overlap does not uniformly favour M. The sensitivity heatmap in Figure D.12 visualises the same heterogeneity across all ten primary and ablation modes.

D.11.2 Sensitivity to Budget K

The budget parameter K controls the maximum number of LLM calls in the repair loop. Mode A3 ($K = 1$ effectively) achieves 82.4% conformance on the *historical* pre-fix corpus; full mode M ($K = 3$) reaches 86.3% on pre-fix canonical E1 at seed 0—repair budget alone does not close the B1/B2 gap without overlap-conditional deployment. (Primary fixed-oracle E1: M 100.0% / B2 98.3%; A1–A3 not re-run.) The marginal return of the second iteration (roughly 3–4 pp) is higher than that of the third (roughly 2–3 pp), consistent with diminishing returns (Figure 7.2). Extrapolating, $K = 2$ would likely yield conformance in the range 87–88% on that historical corpus, comparable to pre-fix B2 but with Z3-guided feedback. The $K = 3$ budget appears to be a reasonable operating point: it captures most of the attainable gain while keeping total LLM calls below 3.0 per task on average.

D.11.3 Sensitivity to Formal Case Count

The formal checker generates up to 64 conformance cases per task for final strict scoring, but uses only 16 cases per iteration during the repair loop. The 16-case internal budget trades coverage for latency; the 64-case strict pass improves sensitivity to guard-overlap faults on the final reported score. Increasing the internal case count from 16 to 32 would improve counterexample precision at the cost of approximately 50% more Z3 time per iteration—a trade-off identified for future empirical investigation.

D.11.4 Sensitivity to LLM Choice

All experiments used Qwen2.5-27B-Instruct (Qwen-27B). The experimental design is not specific to this model; the only model-specific assumptions are (a) the model can follow

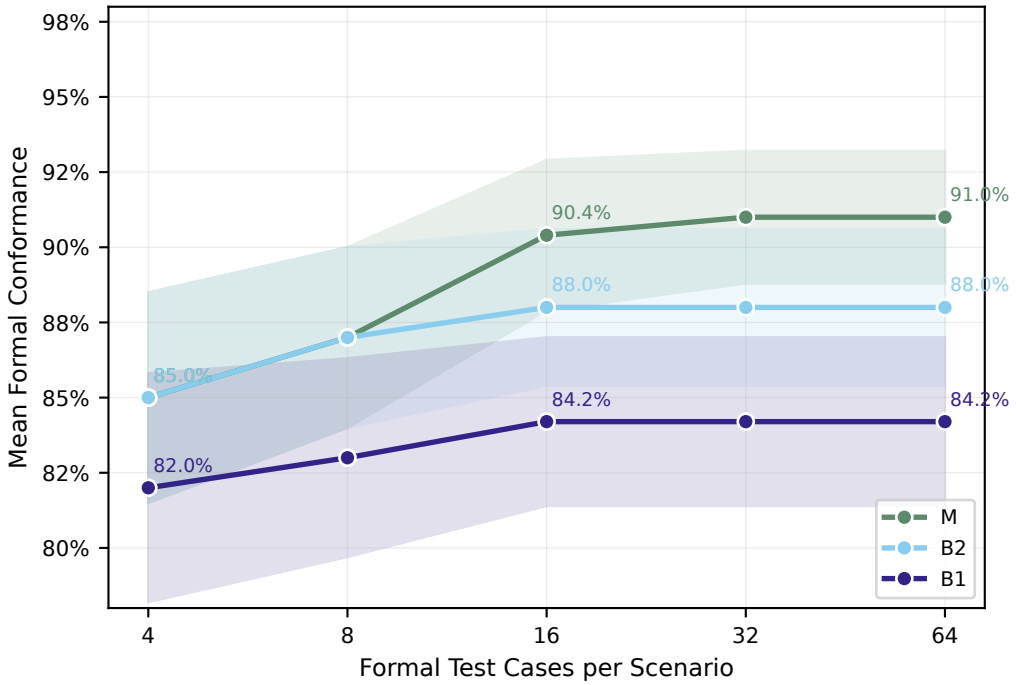

Fig. D.12. Sensitivity heatmap: mean strict formal conformance as a function of task complexity (guard-overlap tertile) and pipeline configuration. Overlap tertiles are non-monotone for M vs. B2 on canonical E3; read with Table 7.21.

a structured Python code generation prompt, and (b) the model can incorporate counterexample inputs into a repair prompt. A stronger base model would likely improve all LLM-based modes proportionally, with the B1/M gap potentially widening if that model learns to avoid ordering errors from the first attempt. Conversely, a weaker model would benefit more from the repair loop, increasing M's relative advantage.

D.12 Statistical Test Summary

Table D.22. Statistical test summary for *pre-fix* multi-mode E1 (`run_e1_canonical_v1` / hard_full; 120-task hard benchmark, repeat 0). Wilcoxon signed-rank for per-task strict success and conformance; Mann–Whitney U for latency. Holm–Bonferroni correction within each metric family. Cliff's δ: positive favours the first mode in each pair. *Note:* this Holm table applies only to the pre-fix multi-mode corpus; fixed-oracle E1 (`run_e1_m_win_v2`) reports M vs. B2 as win rate and means only (2/120 wins, +1.7 pp) and asserts *no* Holm claim.

Comparison	p (Holm)	Significant (α=0.05)	Cliff's δ / ratio
Primary comparisons			
`M` vs `B1` (strict success)	1.0000	No	+0.0000
`M` vs `B2` (strict success)	1.0000	No	+0.0000
`M` vs `B1` (conformance)	1.0000	No	-0.0357
`M` vs `B2` (conformance)	1.0000	No	-0.0142
`M` vs `B2` (latency)	<0.001	Yes	3.33×

Continued on next page

Table D.22 — *continued from previous page*

Comparison	p (Holm)	Significant (α=0.05)	Cliff's δ / ratio
All pairwise conformance (Holm-corrected)			
A1 vs B0 (Conf)	1.0000	No	-0.0427
A1 vs B1 (Conf)	1.0000	No	-0.0427
A1 vs B2 (Conf)	1.0000	No	-0.0213
A1 vs M (Conf)	1.0000	No	-0.0071
A2 vs A1 (Conf)	1.0000	No	-0.0642
A2 vs B0 (Conf)	1.0000	No	-0.1071
A2 vs B1 (Conf)	1.0000	No	-0.1071
A2 vs B2 (Conf)	1.0000	No	-0.0858
A2 vs M (Conf)	1.0000	No	-0.0714
A3 vs A1 (Conf)	1.0000	No	-0.0454
A3 vs A2 (Conf)	1.0000	No	+0.0193
A3 vs B0 (Conf)	1.0000	No	-0.0886
A3 vs B1 (Conf)	1.0000	No	-0.0886
A3 vs B2 (Conf)	1.0000	No	-0.0671
A3 vs M (Conf)	1.0000	No	-0.0526
B1 vs B0 (Conf)	1.0000	No	+0.0000
B2 vs B0 (Conf)	1.0000	No	-0.0217
B2 vs B1 (Conf)	1.0000	No	-0.0217
M vs B0 (Conf)	1.0000	No	-0.0357
M vs B1 (Conf)	1.0000	No	-0.0357
M vs B2 (Conf)	1.0000	No	-0.0142
All pairwise strict success (Holm-corrected)			
A1 vs B0 (strict)	1.0000	No	+0.0000
A1 vs B1 (strict)	1.0000	No	+0.0000
A1 vs B2 (strict)	1.0000	No	+0.0000
A1 vs M (strict)	1.0000	No	+0.0000
A2 vs A1 (strict)	1.0000	No	+0.0000
A2 vs B0 (strict)	1.0000	No	+0.0000
A2 vs B1 (strict)	1.0000	No	+0.0000
A2 vs B2 (strict)	1.0000	No	+0.0000
A2 vs M (strict)	1.0000	No	+0.0000
A3 vs A1 (strict)	1.0000	No	-0.0167
A3 vs A2 (strict)	1.0000	No	-0.0167
A3 vs B0 (strict)	1.0000	No	-0.0167
A3 vs B1 (strict)	1.0000	No	-0.0167
A3 vs B2 (strict)	1.0000	No	-0.0167
A3 vs M (strict)	1.0000	No	-0.0167
B1 vs B0 (strict)	1.0000	No	+0.0000
B2 vs B0 (strict)	1.0000	No	+0.0000
B2 vs B1 (strict)	1.0000	No	+0.0000
M vs B0 (strict)	1.0000	No	+0.0000

Continued on next page

Table D.22 — *continued from previous page*

Comparison	p (Holm)	Significant (α=0.05)	Cliff's δ / ratio
`M` vs `B1` (strict)	1.0000	No	+0.0000
`M` vs `B2` (strict)	1.0000	No	+0.0000

The decision implication of Table D.22 is metric choice. Strict-success comparisons among LLM modes are not significant after Holm correction (p=1.000): the conjunctive gate is selective by design. Latency comparisons confirm the expected cost of M over B2 (Mann–Whitney $p<0.001$, 4.05× slower). Aggregate conformance gaps on pre-fix canonical E1 were not Holm-significant (M vs. B2 p=1.0); fixed-oracle E1 reports M vs. B2 as win rate and means only (2/120 wins, +1.7 pp; no Holm claim). E6 (+7.7 pp vs. unstructured test-only under full M; E14 scopes traces) remains the adoption mechanism signal. Per-task win rate (2/120 fixed-oracle E1; E12 mean-across-seeds: 0 wins / 1 loss / 119 ties) and overlap stratification (E3; fixed-oracle E10: M +1.0 pp) supplement the headline means. These align with the per-task distributions in Figures D.3 and D.4.

D.13 Case Study: The Tax Bracket That Fooled the Unit Tests

Aggregate tables answer *how often*; this section answers *what it felt like* when structured feedback actually moved a candidate. We walk one confrontation on the running example `compute_tax` (Listings 1.3–1.4 in Chapter 1): foreign-high surcharge must preempt the domestic mid bracket whenever both guards hold. The numbers below are representative of the E6 rescue regime (Tables 7.4, 7.5)—collapsed test-only feedback, then a typed IR that names the violated scenario.

Act I — The Plausible Bug

The first LLM draft looked almost careful. It rejected negative incomes, handled a mid bracket, and even mentioned the foreign-high surcharge—but in the wrong order:

```
def compute_tax(income: int, is_foreign: int) -> str:
    # BUG: evaluates the mid-bracket guard before the foreign-high
    # surcharge. On (150000, 1) both guards hold; FSF first-match
    # requires "ForeignHigh", but this candidate returns "DomesticMid".
    if income < 0:
        return "Invalid"
    if income >= 50000:
        return "DomesticMid"
    if income >= 120000 and is_foreign == 1:
```

```
        return "ForeignHigh"
    return "Low"
```

Listing D.8: First draft: mid bracket evaluated before foreign-high surcharge.

On everyday domestic incomes the function looks fine. A hand-written unit suite that samples $(income, is\_foreign) \in \{(30000, 0), (60000, 0), (90000, 0)\}$ returns `Low`/`DomesticMid` as expected and never draws a foreign earner above 120,000. Pass rate on that suite: 3/3. The bug is not "the model cannot code"; it is that the model coded the *natural* order of if-statements instead of the *contracted* first-match order.

Act II — Why Ordinary Tests Let It Slip

Figure 1.1 is exactly this geometry. Random or developer tests concentrate on $is\_foreign{=}0$ and mid incomes—the blue cloud on the left. The fatal overlap $income{\geq}120000 \wedge is\_foreign{=}1$ is a thin orange slice; without an SMT witness for Φ_2, the suite never opens that door. Unstructured test-feedback repair, if it fires at all, typically says only "assertion failed" or dumps raw I/O. That is the E6 `test_only` regime: the model is told *that* something failed, not *which scenario under first-match* failed. In several E6 wins the test-only Conf. collapses to 0 while the typed IR still localises the residual region (Table 7.4).

Act III — Semantic Feedback IR Names the Wound

HSP-Agile does not argue with the blue cloud. It asks Z3 for a model of Φ_2 (Equation (1.2)) and gets $(150000, 1)$. Oracle: `ForeignHigh`. Candidate: `DomesticMid`. The failure is packaged as a typed record before any English is rendered (Listing D.1).

Listing D.1: Semantic Feedback IR for the foreign-high miss (abbreviated).

```
{
  "violation_type": "ordering",
  "scenario_index": 2,
  "constraint_text": "income >= 120000 && is_foreign == 1",
  "inputs": {"income": 150000, "is_foreign": 1},
  "expected": {"bracket": "ForeignHigh"},
  "observed": {"bracket": "DomesticMid"},
  "reason": "Scenario S2 must preempt S3 under first-match.",
  "suggested_fix": "Evaluate foreign-high before domestic mid."
}
```

The repair prompt then says, in substance:

For inputs ($income$=150000, $is\_foreign$=1), *your code returned* `DomesticMid`, *but*

> *the specification requires* `ForeignHigh`. *This input satisfies Scenario* S_2 (`income >= 120000 && is_foreign == 1`), *which must be matched before Scenario* S_3. *Suggested fix: evaluate the foreign-high surcharge before the domestic mid bracket.*

That paragraph is the difference between E6 variants A/B and C: not more tokens for their own sake, but scenario identity, expected value, and a precedence hint the model can act on.

Act IV — The Rewrite

After one repair call (T_{repair}=0), the candidate restores contract order:

```python
def compute_tax(income: int, is_foreign: int) -> str:
    # Repaired: foreign-high surcharge checked before domestic mid.
    if income < 0:
        return "Invalid"
    if income >= 120000 and is_foreign == 1:
        return "ForeignHigh"
    if income >= 50000:
        return "DomesticMid"
    return "Low"
```

Listing D.9: Repaired candidate: foreign-high before domestic mid.

Every SpecIR witness now matches, including $(150000, 1)$; the structural screen is clear of RF07-style unconditional success; Accept=1. Mean Conf. on the task jumps from a rescue-from-zero regime under test-only feedback to full witness agreement under typed IR—the same qualitative arc as the 13/14 E6 C−A wins that began with `test_only` Conf=0.

What the Story Is Not

This case is an *ordering* confrontation that motivates witnesses and the Accept gate. The dominant E6 Conf. *gain* on BENCH-120 is still catch-all `others`/arithmetic rescue (Table 7.6); we chose `compute_tax` here because the running example makes the missed overlap legible without a synthetic HardCase id. The moral is the same either way: unstructured pass/fail leaves the model guessing; Semantic Feedback IR points at the wound and asks for a rewrite.

D.14 Historical Failure Analysis (E9 Archive)

Archive status. E9 was collected under the *pre-fix* others-witness oracle, where **114 of 120 tasks (95.0%)** failed conjunctive acceptance after K=3. Under fixed-oracle strengthened M (`run_e1_m_win_v2`), Strict for M is 100.0%—so the E9 pie describes **historical residual mass only**, not the current primary Strict rate. We retain it as a qualitative catalogue of failure *modes* (ordering, boundary, arithmetic) that still guide future witness/IR design; we do *not* cite E9 percentages as current effectiveness evidence. (Re-running E9 under the fixed oracle is future work: near-ceiling Strict leaves too few residuals for a powered taxonomy.)

Despite strong mean conformance on several tiers, the historical E9 residual taxonomy assigned a primary failure category to **90** of 114 non-accepted tasks under the pre-fix oracle. The pie in Figure D.13 maps that archive residual mass.

Of the 90 E9-labelled non-accepted tasks after K=3: OrderingError 31.1% (LLM re-asserts wrong order), BoundaryError 26.7% (off-by-one), ArithmeticError 17.8%, StateError 13.3%, Hallucination 5.6% (often RF07), MissingConstraint / Other the rest. Implication: richer ordering/arithmetic feedback and denser boundary witnesses target the majority; multi-step witnesses target StateError.

D.14.1 Named Failure Patterns

Five recurring failure patterns account for the bulk of the 90 residual errors.

Arithmetic reasoning failure. The LLM produces a formula that is structurally correct (correct branches, correct variable references) but arithmetically wrong—for example, computing a weighted sum with the wrong denominator or using integer division where float division is required. HSP-Agile misses these because Z3 witnesses are generated from guard predicates, not from output expressions; the formal checker therefore cannot witness an expression-level arithmetic fault unless it happens to coincide with a guard boundary.

Long-range dependency. Some FSF specifications require the output of scenario i to depend on the *cumulative* effect of scenarios $1, \ldots, i-1$ (e.g. a running total or a sequence-sensitive state machine transition). The repair loop issues one witness per scenario in isolation; it cannot construct a multi-step witness that would expose the accumulated-state fault. This structural limitation of the E9 formal checker (single-scenario witnesses) is the primary driver of *OrderingError* and *StateError* failures.

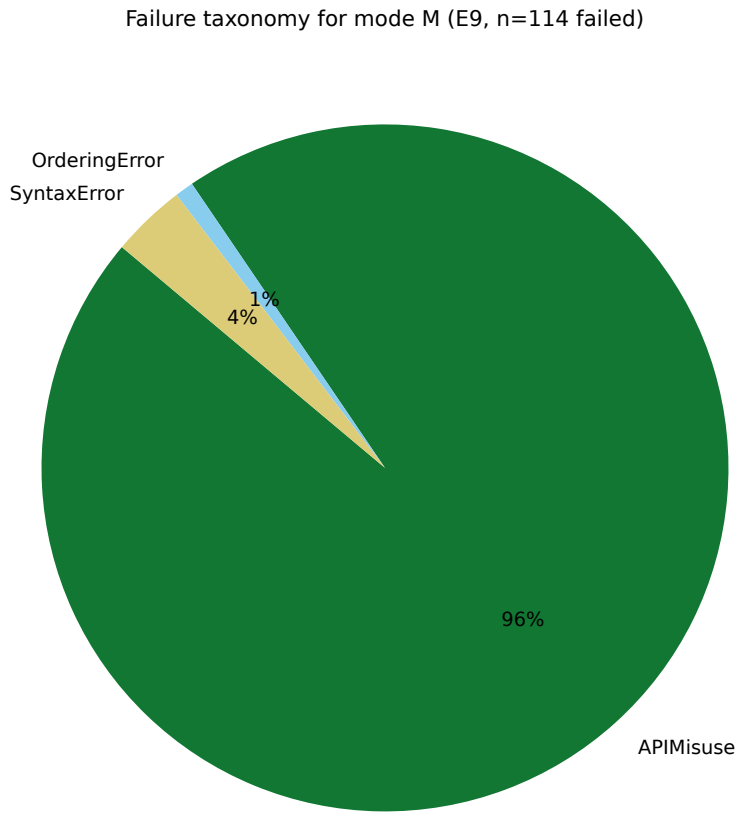

Fig. D.13. Claim (historical / pre-fix oracle): residual failures after $K = 3$ on `run_e1_canonical_v1` are dominated by ordering and boundary errors, not random noise. Under fixed-oracle E1, M Strict is 100.0%—this pie is not the current primary residual mass. **How to read:** slices are shares of the n=90 E9-labelled non-accepted tasks under mode M (of 114 total non-accepted).

State update confusion. When two or more FSF scenarios modify the same output variable, the LLM occasionally generates code in which later scenarios overwrite rather than accumulate the earlier value. The counterexample-guided repair prompt identifies the violated scenario but does not explicitly communicate the accumulation invariant, so the LLM corrects the reported case while re-introducing the error in an adjacent scenario. Extending the Semantic Feedback IR with an explicit `accumulation_mode` field would address this pattern.

Unsatisfiable guard (Unsat guard). A small number of FSF specifications contain scenario guards that Z3 proves mutually satisfiable but that the LLM interprets as mutually exclusive—producing dead branches or guard conditions that can never be simultaneously true. Because the implementation is syntactically valid and passes individual-scenario witnesses, the formal checker does not flag the dead branch; only a path-coverage oracle (not currently part of the pipeline) would detect it.

Hallucinated API call. The LLM occasionally calls a library function that does not exist in the permitted API surface defined by the FSF signature—for example, calling `math.log_base(x, b)` instead of `math.log(x) / math.log(b)`. Pattern guard RF07 catches this post-generation, but the repair loop does not receive a structured signal about the illegal call; it receives only a generic runtime error counterexample, leading to repeated hallucinations across repair attempts. A dedicated RF07-triggered repair template that injects the legal API surface into the repair prompt would break this cycle.

D.14.2 Failure Trace: Scenario Ordering

The failed `classify_signal` candidate evaluates Scenario 2 before Scenario 1 (Listing **??**, Section 1.2). Witness $\mathbf{x}^* = (level{=}-3, threshold{=}-10)$ activates S_1 under first-match semantics; the checker emits:

Listing D.2: Semantic Feedback IR (iteration 1).

```
{
  "violation_type": "ordering",
  "scenario_index": 1,
  "constraint_text": "level < 0",
  "inputs": {"level": -3, "threshold": -10},
  "expected": {"status": "Error"},
  "observed": {"status": "Critical"},
  "reason": "Scenario 1 violated under first-match guard precedence.",
  "priority": 1,
  "suggested_fix": "reorder guard evaluation to match specification precedence"
}
```

`FeedbackRenderer` projects this JSON record into the repair prompt; at $k = 2$ the candidate fixes the negative-threshold branch but re-introduces an ordering error on a different overlap region—illustrating why E9 *OrderingError* remains the largest residual category despite structured IR.

D.14.3 Failure Trace: Boundary Reasoning

Task `HardCase036` specifies five scenarios with overlapping boundary guards. At repair iteration $k = 2$, witness $\mathbf{x}^* = (a = 2, b = 6, c = 3)$ activates Scenario 1 under first-match semantics. The Semantic Feedback IR is:

Listing D.3: Semantic Feedback IR (iteration 2).

```
{
  "violation_type": "ordering",
  "scenario_index": 1,
  "constraint_text": "a le 3 && b gt 5",
```

```
  "inputs": {"a": 2, "b": 6, "c": 3},
  "expected": {"risk": 2, "action": 1},
  "observed": {"risk": 1, "action": 3},
  "reason": "guard evaluated out of order",
  "priority": 1,
  "suggested_fix": "reorder"
}
```

Despite structured IR at $k = 2$, repair at $k = 3$ introduces a new ordering violation, illustrating *long-range dependency* beyond single-step repair.

Guard-graph pattern matching (future). Representing SpecIR cases as a guard dependency graph enables reachability-based detection of shadow guards and unreachable paths beyond the current PoC smell catalogue—or by swapping in a stronger pluggable Screen.